

Introduction

Bruno Siciliano, Oussama Khatib

Robots! Robots on Mars and in oceans, in hospitals and homes, in factories and schools; robots fighting fires, making goods and products, saving time and lives . . . Robots today are making a considerable impact on many aspects of modern life, from industrial manufacturing to healthcare, transportation, and exploration of the deep space and sea. Tomorrow, robots will be as pervasive and personal as today's personal computers. The dream to create machines that are skilled and intelligent has been part of humanity from the beginning of time. This dream is now becoming part of our world's striking reality.

Since the early civilizations, one of man's greatest ambitions has been to create artifacts in their image. The legend of the Titan Prometheus, who molded humankind from clay, or that of the giant Talus, the bronze slave forged by Hephaestus (3500 BC), testify to this quest in Greek mythology. The Egyptians' oracle statues hiding priests inside (2500 BC) were perhaps the precursor of our modern thinking machines. The clepsydra water clock introduced by the Babylonians (1400 BC) was one of the first automated mechanical artifacts. In the following centuries, human creativity has given rise to a host of devices such as the automaton theatre of Hero of Alexandria (100 AD), the hydro-powered water-raising and humanoid machines of Al-Jazari (1200), and Leonardo da Vinci's numerous ingenious designs (1500). The development of automata continued to flourish in the eighteenth century both in Europe and Asia, with creations such as Jacquet-Droz's family of androids (drawer, musician and writer) and the "karakuri-ningyo" mechanical dolls (tea server and archer).

The robot "concept" was clearly established by those many creative historical realizations. Nonetheless, the emergence of the "physical" robot had to await the advent of its underlying technologies during the course of the twentieth century. In 1920, the term *robot* – derived from "robota" which means subordinate labour in Slav languages – was first introduced by the Czech playwright Karel Čapek in his play "Rossum's Universal Robots (R.U.R.)". In 1940, the ethics of the interaction

between robots and humans was envisioned to be governed by the well-known three fundamental laws of Isaac Asimov, the Russian science-fiction writer in his novel "Runaround".

The middle of the twentieth century brought the first explorations of the connection between human intelligence and machines, marking the beginning of an era of fertile research in the field of artificial intelligence (AI). Around that time, the first robots were realized. They benefited from advances in the different technologies of mechanics, controls, computers and electronics. As always, new designs motivate new research and discoveries, which, in turn, lead to enhanced solutions and thus to novel concepts. This virtuous circle over time produced that knowledge and understanding which gave birth to the field of *robotics*, properly referred to as: the science and technology of robots.

The early robots built in the 1960s stemmed from the confluence of two technologies: numerical control machines for precise manufacturing, and teleoperators for remote radioactive material handling. These master-slave arms were designed to duplicate one-to-one the mechanics of the human arm, and had rudimentary control and little perception about the environment. Then, during the mid-to-late twentieth century, the development of integrated circuits, digital computers and miniaturized components enabled computer-controlled robots to be designed and programmed. These robots, termed industrial robots, became essential components in the automation of flexible manufacturing systems in the late 1970s. Further to their wide application in the automotive industry, industrial robots were successfully employed in general industry, such as the metal products, the chemical, the electronics and the food industries. More recently, robots have found new applications outside the factories, in areas such as cleaning, search and rescue, underwater, space, and medical applications.

In the 1980s robotics was defined as the science which studies the *intelligent connection between perception and action*. With reference to this definition, the action of a robotic system is entrusted to a *locomotion*

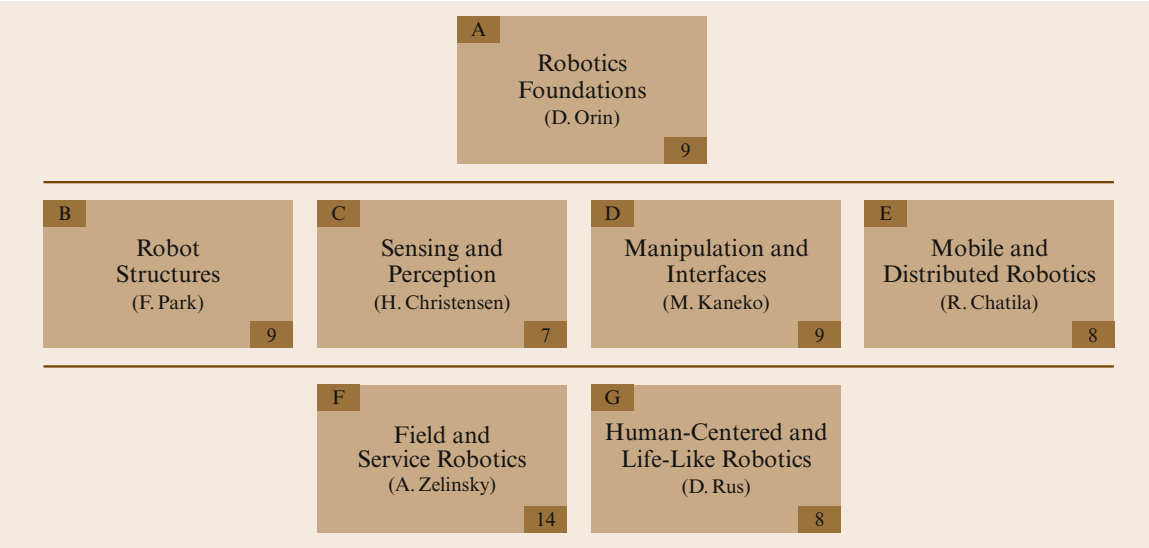


Fig. 1 Organization of the handbook

apparatus to move in the environment (wheels, crawlers, legs, propellers) and/or to a *manipulation* apparatus to operate on objects present in the environment (arms, end effectors, artificial hands), where suitable *actuators* animate the mechanical components of the robot. The perception is extracted from the *sensors* providing information on state of the robot (position and speed) and its surrounding environment (force and tactile, range and vision). The intelligent connection is entrusted to a *programming, planning and control* architecture which relies on the perception and available *models* of the robot and environment and exploits learning and skill acquisition.

In the 1990s research was boosted by the need to resort to robots to address human safety in hazardous environments (*field robotics*), or to enhance the human operator ability and reduce his/her fatigue (*human augmentation*), or else by the desire to develop products with wide potential markets aimed at improving the quality of life (*service robotics*). A common denominator of such application scenarios was the need to operate in a scarcely structured environment which ultimately requires increased abilities and a higher degree of *autonomy*.

By the dawn of the new millennium, robotics has undergone a major transformation in scope and dimensions. This expansion has been brought about by the maturity of the field and the advances in its related technologies. From a largely dominant industrial focus, robotics has been rapidly expanding into the

challenges of the human world (*human-centered and life-like robotics*). The new generation of robots is expected to safely and dependably co-habitat with humans in homes, workplaces, and communities, providing support in services, entertainment, education, healthcare, manufacturing, and assistance.

Beyond its impact on physical robots, the body of knowledge robotics has produced is revealing a much wider range of applications reaching across diverse research areas and scientific disciplines, such as: biomechanics, haptics, neurosciences, virtual simulation, animation, surgery, and sensor networks among others. In return, the challenges of the new emerging areas are proving an abundant source of stimulation and insights for the field of robotics. It is indeed at the intersection of disciplines that the most striking advances happen.

Today, new communities of users and developers are forming, with growing connections to the core of robotics research. A strategic goal for the robotics community is one of outreach and scientific cooperation with these communities. Future developments and expected growth of the field will largely depend on the research community's abilities to achieve this objective.

The dissemination of research results and findings in archival publications and conference presentations has played an important role in the advancement of robotics in the past decades. The extent of scientific activities in robotics has led to the establishment of professional societies and research networks devoted to the field.

The introduction of graduate programs in robotics in many academic institutions around the world is a clear illustration of the level of maturity reached by robotics as a scientific field.

The intensive stream of robotics research documented in the literature culminates into this unique reference, which aims at collecting in one self-contained volume the most significant achievements of our international robotics community. The *Springer Handbook of Robotics* presents a full coverage of the field from its foundations, through the research areas, up to the new emerging applications of robotics. Accordingly, the material is organized in three logical layers reflecting the historical development of the field, as illustrated in Fig. 1.

The foundations of robotics science, laid down in the first layer (Part A with its 9 chapters), address the theory of robot mechanics, sensing, planning, and control. The consolidated methodologies and technologies of robot structures (Part B with its 9 chapters), sensing and perception (Part C with its 7 chapters), manipulation and interfaces (Part D with its 8 chapters) and mobile and distributed robotics (Part E with its 8 chapters) are presented in the second layer. The third layer is devoted to advanced applications, such as in field and service robotics (Part F with its 14 chapters) and human-centered and life-like robotics (Part G with its 9 chapters).

Part A presents the fundamental principles and methods that are used to model, design, and control a robotic system. All of the foundational topics are included in this part: kinematics, dynamics, mechanical design and actuation, sensing and estimation, motion planning, motion control, force control, robotic systems architectures and programming, and **AI** reasoning methods for task planning and learning. A chapter is devoted to each of these topics. The topics are expanded and applied to specific robotic structures and systems in subsequent parts.

Part B is concerned with the design, modeling, motion planning, and control of the actual physical realizations of a robot. Some of the more obvious mechanical structures that come to mind are arms, legs, and hands; to this list can be added wheeled vehicles and platforms, and robot structures at the micro and nano scales. With separate chapters devoted to performance criteria and model identification, the chapters in this part successively examine serial redundant mechanisms, parallel

mechanisms, flexible robots, robot hands, robot legs, wheeled robots, and micro and nano-scale robots.

Part C covers different sensory modalities and integration of sensor data across space and time to generate models of robots and the external environment. Robotics is the intelligent coupling of perception and action and as such Part C complements Part B to build systems. This part of the handbook covers sensing across contact, proprioception to exteroception. The main sensor types such as tactile, odometry, **GPS**, ranging and visual perception are presented. Both basic sensor models, sensor data processing and associated representations are covered. Finally, a chapter on sensor fusion introduces the mathematical tools needed for integration of sensor information across space and time.

Part D is concerned with interaction between robots and objects, or between humans and robots. Manipulation is supposed to handle an object through direct contact by arms or fingers or just a pusher, while interfaces are supposed to make either direct or indirect interaction between humans and robots. For enhancing dexterity in robot manipulation, motion for manipulation tasks, contact modeling and manipulation, grasping, and cooperative manipulators are addressed in the first half of this part. For achieving a skillful manipulation or power increase in a human/robot system, haptics, telerobotics, networked telerobotics, and exoskeletons for human performance augmentation are discussed in the second half of part D.

Part E covers a wide span of topics. This part addresses motion planning and control of wheeled robots with kinematic constraints, perception and world modeling, simultaneous localization and mapping, and the integration of those capacities in a control architecture, as a mobile robot is actually the paradigm of a complex integrated system. This part completes Part A on foundations in the context of mobile robotics, and given the role of perception, is closely related to Part C on sensing. In addition, multi-robot interaction and systems are also discussed, including modular and reconfigurable robots as well as networked robotics.

Part F covers topics related to creating field and service application-based robots that operate in all types of environments. This includes applications ranging from industrial robots, through a diverse array of air, land, sea and space applications to educational robotics. This part

of the handbook, which draws on Parts A, B, C, D and E, describes how robots can be put to work.

Part G covers topics related to creating robots that operate in human-centered environments. This includes the design of robots with humanoid and other biologically inspired morphologies, sensors, actuators, and control architectures. User interfaces, such as programming by demonstration and programming for safety, are also included in this part, and it concludes with the socio-ethical implications of robots.

The handbook was conceived to provide a valuable resource not only for robotics experts, but also for newcomers to this expanding field (engineers, medical doctors, computer scientists, and designers). In particular, it is important to underline the tutorial value of Part A to graduate students and post-docs, the research value of Parts B to E for a wider coverage of research in robotics, and the added value of Parts F and G to engineers and scientists interested in new applications.

The contents of the various chapters have been inspired by a classic cut, i.e. avoiding the inclusion of on-going or not well-established methods. An objective perspective has been taken, while covering multiple approaches, with the goal of ensuring a high archival value to the handbook. Each chapter is preceded by a short summary, and an introductory section providing the state of the art in the area. The core sections are developed at a tutorial level. Lengthy mathematical derivations have been avoided whenever possible, while the equations, tables, and algorithms are illustrated in ready-to-use form. The final section of the chapter provides conclusions and topics for further reading.

From the foundations to the social and ethical implications of robotics, the sixty-four chapters of the handbook provide a comprehensive collection of five decades of progress in robotics. This sum is a testimony to the level of accomplishments in our field, and a premise of further advances towards new frontiers of robotics.

Robotics

Part A

Part A Robotics Foundations

Ed. by David E. Orin

1 Kinematics

Kenneth Waldron, Stanford, USA
James Schmiedeler, Columbus, USA

2 Dynamics

Roy Featherstone, Canberra, Australia
David E. Orin, Columbus, USA

3 Mechanisms and Actuation

Victor Scheinman, Stanford, USA
J. Michael McCarthy, Irvine, USA

4 Sensing and Estimation

Henrik I. Christensen, Atlanta, USA
Gregory D. Hager, Baltimore, USA

5 Motion Planning

Lydia E. Kavrakı, Houston, USA
Steven M. LaValle, Urbana, USA

6 Motion Control

Wankyun Chung, Pohang, Korea
Li-Chen Fu, Taipei, Taiwan, R.O.C.
Su-Hau Hsu†

7 Force Control

Luigi Villani, Napoli, Italy
Joris De Schutter, Leuven-Heverlee, Belgium

8 Robotic Systems Architectures and Programming

David Kortenkamp, Houston, USA
Reid Simmons, Pittsburgh, USA

9 AI Reasoning Methods for Robotics

Joachim Hertzberg, Osnabrück, Germany
Raja Chatila, Toulouse, France

The chapters contained in **Part A**, Robotics Foundations, present the fundamental principles and methods that are used to develop a robotic system. In order to perform the tasks that are envisioned for robots, many challenging problems have been uncovered in kinematics, dynamics, design, actuation, sensing, motion planning, control, programming, and task planning. The chapters in this part address the basic issues in each of these areas. Some of the basic problems in robotics are outlined as follows. Robots often consist of a large number of degrees of freedom so that they can provide the rich set of three-dimensional (3-D) motions that may be required for a range of tasks. The kinematic and dynamic relationships between the joint actuators' motion and torques, and the desired motion and force for a task can be very complex. The design of the link and joint structures, as well as the actuation, to achieve the desired performance is also challenging. The robot is a nonlinear, coupled system which is difficult to control because of its complex dynamics. This is exacerbated when the environment is unstructured, and often sophisticated sensing and estimation techniques are required.

In addition to control of the motion, control of the interaction forces between the robot and environment is needed when manipulating objects or interacting with humans. A fundamental robotics task is to plan collision-free motion for complex bodies from a start to a goal position among a collection of obstacles, and this can become an intractable computational problem. In order to achieve some of the intelligence ascribed to humans, robots need to be equipped with sophisticated task planners that employ symbolic reasoning to move in dynamic, partially known environments. Robot software architectures also have special needs because of these requirements.

While the basic issues outlined in the previous paragraphs are addressed in this part, more depth can be found in other parts of the Handbook. The kinematics, dynamics, mechanical design, and control principles and methods introduced in this part can be applied to robotic structures made up of arms, hands, and legs (Part B) as well as to manipulators (Part D), wheeled mobile robots (Part E), and field and service robots (Part F). Force control is especially important for manipulators and their interfaces (Part D). The basic sensing and estimation techniques presented here are expanded and applied to specific sensing modalities in Part C. Motion planning is an important aspect of manipulation (Part D) and mobile and distributed robotic systems (Part E). Robotic systems architectures and AI reasoning methods are particularly important in mobile and

distributed robotics (Part E) and human-centered and lifelike robotics (Part G).

With this overview of Part A, we now provide a brief synopsis of each chapter.

Chapter 1, Kinematics, provides a number of representations and conventions to describe the motion of the bodies in a robotic mechanism. These include rotation matrices, Euler angles, quaternions, homogeneous transformations, screw transformations, matrix exponential parameterization, and Plücker coordinates. Representations of the kinematics of all common joint types are provided, along with a modified form of the Denavit–Hartenberg convention. These representational tools are applied to compute the workspace, the forward and inverse kinematics, the forward and inverse instantaneous kinematics, the Jacobian, and the static wrench transmission.

Chapter 2, Dynamics, presents the dynamic equations of motion which provide the relationships between actuation and contact forces acting on robot mechanisms, and the acceleration and motion trajectories that result. Efficient algorithms are provided for important dynamics computations which include inverse dynamics, forward dynamics, the joint-space inertia matrix, and the operational-space inertia matrix. The algorithms may be applied to fixed-base robots, mobile robots, and parallel robot mechanisms. Compact formulation of the algorithms results from using six-dimensional (6-D) spatial notation to describe rigid-body velocity, acceleration, inertia, etc.

Chapter 3, Mechanisms and Actuation, focuses on the principles that guide the design and construction of robotic systems. The kinematic equations and Jacobian are used to characterize the work envelope and mechanical advantage, and guide the selection of the robot's size and joint arrangement. The design of both serial and parallel robots is addressed. Practical consideration is given to the design of the link and joint structures along with selection of the actuators and transmission drives to power the movement. Robot performance in terms of speed, acceleration, repeatability, and other measures is also addressed.

Chapter 4, Sensing and Estimation, provides a brief overview of common sensing methods and estimation techniques that have found broad applicability in robotics. These provide information about the state of the environment and robot system. The presentation is structured according to a perception process model that includes sensing, feature extraction, data association, parameter estimation, and model integration. Several common sensing modalities are introduced and charac-

terized. Methods for estimation in linear and nonlinear systems are discussed, including statistical estimation, the Kalman filter, and sample-based methods. The several common representations for estimation are also introduced.

Chapter 5, Motion Planning, addresses the fundamental robotics task to plan collision-free motion for complex bodies from a start to a goal position among a collection of obstacles. The basic geometric path planning problem (the piano mover's problem) is introduced, but the focus of the chapter is on sampling-based planning methods because of their generally wider applicability. Planning with differential constraints is considered and is important for wheeled mobile robots. Extensions and variations to the basic motion planning problem, as well as advanced issues, are discussed at the end of the chapter.

Chapter 6, Motion Control, focuses on the motion control of a rigid manipulator. The main challenges addressed are the complexity of the nonlinear, coupled dynamics and the structured and unstructured uncertainties. The chapter discusses topics ranging from independent-joint control and **PID** control to computed-torque control to manage the complex dynamics. Adaptive and robust control are presented to address the problems related to the uncertainties in the system. The chapter concludes with some practical considerations of digital implementation and learning control for repetitive motions.

Chapter 7, Force Control, focuses on the control of the interaction forces between a robotic system and its

environment. The chapter groups interaction control into two categories: indirect and direct force control, which achieve the control without (indirect) or with (direct) explicit closure of a force feedback loop. Impedance control and hybrid force/motion control are examples of each, respectively. The fundamental problem of interaction tasks modeling is presented to provide the basis for the force control schemes.

Chapter 8, Robotic Systems Architectures, and Programming, presents the software architectures and supporting programming tools and environments that have been developed for robotic systems. Robot architectures have special needs because of the requirements of the robot to interact asynchronously, in real time, with an uncertain, often dynamic, environment. The chapter discusses the major types of architectural components for a layered robot control architecture – behavioral control, executives, and task planners – along with the commonly used techniques for interconnecting these components.

Chapter 9, AI Reasoning Methods for Robotics, sketches the state of the art in artificial intelligence for the symbol-based reasoning methods and applications considered most relevant for robotics. Reasoning on a mobile robot is especially challenging because of its dynamic, partially known environment. The chapter describes knowledge representation and inference, covering both logics- and probability-based approaches. Besides reasoning methods, the chapter considers generic reasoning applications, namely, action planning and robot learning.

Kinematics

1. Kinematics

Kenneth Waldron, James Schmiedeler

Kinematics pertains to the motion of bodies in a robotic mechanism without regard to the forces/torques that cause the motion. Since robotic mechanisms are by their very essence designed for motion, kinematics is the most fundamental aspect of robot design, analysis, control, and simulation. The robotics community has focused on efficiently applying different representations of position and orientation and their derivatives with respect to time to solve foundational kinematics problems.

This chapter will present the most useful representations of the position and orientation of a body in space, the kinematics of the joints most commonly found in robotic mechanisms, and a convenient convention for representing the geometry of robotic mechanisms. These representational tools will be applied to compute the *workspace*, the *forward* and *inverse kinematics*, the *forward* and *inverse instantaneous kinematics*, and the *static wrench transmission* of a robotic mechanism. For brevity, the focus will be on algorithms applicable to open-chain mechanisms.

The goal of this chapter is to provide the reader with general tools in tabulated form and a broader overview of algorithms that can be applied together to solve kinematics problems pertaining to a particular robotic mechanism.

1.1	Overview	9
1.2	Position and Orientation Representation	10
1.2.1	Position and Displacement.....	10
1.2.2	Orientation and Rotation	10
1.2.3	Homogeneous Transformations.....	13
1.2.4	Screw Transformations.....	14
1.2.5	Matrix Exponential Parameterization	16
1.2.6	Plücker Coordinates.....	18
1.3	Joint Kinematics	18
1.3.1	Lower Pair Joints.....	19
1.3.2	Higher Pair Joints.....	21
1.3.3	Compound Joints	22
1.3.4	6-DOF Joint	22
1.3.5	Physical Realization.....	22
1.3.6	Holonomic and Nonholonomic Constraints	23
1.3.7	Generalized Coordinates	23
1.4	Geometric Representation	23
1.5	Workspace	25
1.6	Forward Kinematics	26
1.7	Inverse Kinematics	27
1.7.1	Closed-Form Solutions.....	27
1.7.2	Numerical Methods	28
1.8	Forward Instantaneous Kinematics	29
1.8.1	Jacobian	29
1.9	Inverse Instantaneous Kinematics	30
1.9.1	Inverse Jacobian	30
1.10	Static Wrench Transmission	30
1.11	Conclusions and Further Reading	31
	References	31

1.1 Overview

Unless explicitly stated otherwise, robotic mechanisms are systems of rigid bodies connected by joints. The position and orientation of a rigid body in space are collectively termed the *pose*. Therefore, robot kinemat-

ics describes the pose, velocity, acceleration, and all higher-order derivatives of the pose of the bodies that comprise a mechanism. Since kinematics does not address the forces/torques that induce motion, this chapter

focuses on describing pose and velocity. These descriptions are foundational elements of dynamics (Chap. 2), motion planning (Chap. 5), and motion control (Chap. 6) algorithms.

Among the many possible topologies in which systems of bodies can be connected, two are of particular importance in robotics: serial chains and fully parallel mechanisms. A serial chain is a system of rigid bodies in

which each member is connected to two others, except for the first and last members that are each connected to only one other member. A fully parallel mechanism is one in which there are two members that are connected together by multiple joints. In practice, each *joint* is often itself a serial chain. This chapter focuses almost exclusively on algorithms applicable to serial chains. Parallel mechanisms are dealt with in more detail in Chap. 12.

1.2 Position and Orientation Representation

Spatial, rigid-body kinematics can be viewed as a comparative study of different ways of representing the pose of a body. Translations and rotations, referred to in combination as rigid-body displacements, are also expressed with these representations. No one approach is optimal for all purposes, but the advantages of each can be leveraged appropriately to facilitate the solution of different problems.

The minimum number of coordinates required to locate a body in Euclidean space is six. Many representations of spatial pose employ sets with superabundant coordinates in which auxiliary relationships exist among the coordinates. The number of independent auxiliary relationships is the difference between the number of coordinates in the set and six.

This chapter and those that follow it make frequent use of *coordinate reference frames* or simply *frames*. A coordinate reference frame i consists of an origin, denoted O_i , and a triad of mutually orthogonal basis vectors, denoted $(\hat{x}_i \hat{y}_i \hat{z}_i)$, that are all fixed within a particular body. The pose of a body will always be expressed relative to some other body, so it can be expressed as the pose of one coordinate frame relative to another. Similarly, rigid-body displacements can be expressed as displacements between two coordinate frames, one of which may be referred to as *moving*, while the other may be referred to as *fixed*. This indicates that the observer is located in a stationary position within the fixed reference frame, not that there exists any absolutely fixed frame.

1.2.1 Position and Displacement

The position of the origin of coordinate frame i relative to coordinate frame j can be denoted by the 3×1 vector

$${}^j\mathbf{p}_i = \begin{pmatrix} {}^j p_i^x \\ {}^j p_i^y \\ {}^j p_i^z \end{pmatrix}.$$

The components of this vector are the Cartesian coordinates of O_i in the j frame, which are the projections of the vector ${}^j\mathbf{p}_i$ onto the corresponding axes. The vector components could also be expressed as the spherical or cylindrical coordinates of O_i in the j frame. Such representations have advantages for analysis of robotic mechanisms including spherical and cylindrical joints.

A translation is a displacement in which no point in the rigid body remains in its initial position and all straight lines in the rigid body remain parallel to their initial orientations. (The points and lines are not necessarily contained within the boundaries of the finite rigid body, but rather, any point or line in space can be taken to be rigidly fixed in a body.) The translation of a body in space can be represented by the combination of its positions prior to and following the translation. Conversely, the position of a body can be represented as a translation that takes the body from a position in which the coordinate frame fixed to the body coincides with the fixed coordinate frame to the current position in which the two frames are not coincident. Thus, any representation of position can be used to create a representation of displacement, and vice versa.

1.2.2 Orientation and Rotation

There is significantly greater breadth in the representation of orientation than in that of position. This section does not include an exhaustive summary, but focuses on the representations most commonly applied to robotic mechanisms.

A rotation is a displacement in which at least one point of the rigid body remains in its initial position and not all lines in the body remain parallel to their initial orientations. For example, a body in a circular orbit rotates about an axis through the center of its circular path, and every point on the axis of rotation is a point

in the body that remains in its initial position. As in the case of position and translation, any representation of orientation can be used to create a representation of rotation, and vice versa.

Rotation Matrices

The orientation of coordinate frame i relative to coordinate frame j can be denoted by expressing the basis vectors ($\hat{x}_i \hat{y}_i \hat{z}_i$) in terms of the basis vectors ($\hat{x}_j \hat{y}_j \hat{z}_j$). This yields (${}^j\hat{x}_i {}^j\hat{y}_i {}^j\hat{z}_i$), which when written together as a 3×3 matrix is known as the rotation matrix. The components of ${}^j\mathbf{R}_i$ are the dot products of basis vectors of the two coordinate frames.

$${}^j\mathbf{R}_i = \begin{pmatrix} \hat{x}_i \cdot \hat{x}_j & \hat{y}_i \cdot \hat{x}_j & \hat{z}_i \cdot \hat{x}_j \\ \hat{x}_i \cdot \hat{y}_j & \hat{y}_i \cdot \hat{y}_j & \hat{z}_i \cdot \hat{y}_j \\ \hat{x}_i \cdot \hat{z}_j & \hat{y}_i \cdot \hat{z}_j & \hat{z}_i \cdot \hat{z}_j \end{pmatrix}. \quad (1.1)$$

Because the basis vectors are unit vectors and the dot product of any two unit vectors is the cosine of the angle between them, the components are commonly referred to as direction cosines.

An elementary rotation of frame i about the $\hat{z}_j$ axis through an angle θ is

$$\mathbf{R}_Z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad (1.2)$$

while the same rotation about the $\hat{y}_j$ axis is

$$\mathbf{R}_Y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}, \quad (1.3)$$

and about the $\hat{x}_j$ axis is

$$\mathbf{R}_X(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix}. \quad (1.4)$$

The rotation matrix ${}^j\mathbf{R}_i$ contains nine elements, while only three parameters are required to define the orientation of a body in space. Therefore, six auxiliary relationships exist between the elements of the matrix. Because the basis vectors of coordinate frame i are mutually orthonormal, as are the basis vectors of coordinate frame j , the columns of ${}^j\mathbf{R}_i$ formed from the dot products of these vectors are also mutually orthonormal. A matrix composed of mutually orthonormal vectors is known as an orthogonal matrix and has the property that its inverse is simply its transpose. This property provides the six auxiliary relationships. Three require the

Table 1.1 Equivalent rotation matrices for various representations of orientation, with abbreviations $c_\theta := \cos \theta$, $s_\theta := \sin \theta$, and $v_\theta := 1 - \cos \theta$

Z-Y-X Euler angles (α, β, γ):

$${}^j\mathbf{R}_i = \begin{pmatrix} c_\alpha c_\beta & c_\alpha s_\beta s_\gamma - s_\alpha c_\gamma & c_\alpha s_\beta c_\gamma + s_\alpha s_\gamma \\ s_\alpha c_\beta & s_\alpha s_\beta s_\gamma + c_\alpha c_\gamma & s_\alpha s_\beta c_\gamma - c_\alpha s_\gamma \\ -s_\beta & c_\beta s_\gamma & c_\beta c_\gamma \end{pmatrix}$$

X-Y-Z fixed angles (ψ, θ, ϕ):

$${}^j\mathbf{R}_i = \begin{pmatrix} c_\phi c_\theta & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ s_\phi c_\theta & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{pmatrix}$$

Angle-axis $\theta \hat{\mathbf{w}}$:

$${}^j\mathbf{R}_i = \begin{pmatrix} w_x^2 v_\theta + c_\theta & w_x w_y v_\theta - w_z s_\theta & w_x w_z v_\theta + w_y s_\theta \\ w_x w_y v_\theta + w_z s_\theta & w_y^2 v_\theta + c_\theta & w_y w_z v_\theta - w_x s_\theta \\ w_x w_z v_\theta - w_y s_\theta & w_y w_z v_\theta + w_x s_\theta & w_z^2 v_\theta + c_\theta \end{pmatrix}$$

Unit quaternions ($\epsilon_0 \epsilon_1 \epsilon_2 \epsilon_3$)^T:

$${}^j\mathbf{R}_i = \begin{pmatrix} 1 - 2(\epsilon_2^2 + \epsilon_3^2) & 2(\epsilon_1 \epsilon_2 - \epsilon_0 \epsilon_3) & 2(\epsilon_1 \epsilon_3 + \epsilon_0 \epsilon_2) \\ 2(\epsilon_1 \epsilon_2 + \epsilon_0 \epsilon_3) & 1 - 2(\epsilon_1^2 + \epsilon_3^2) & 2(\epsilon_2 \epsilon_3 - \epsilon_0 \epsilon_1) \\ 2(\epsilon_1 \epsilon_3 - \epsilon_0 \epsilon_2) & 2(\epsilon_2 \epsilon_3 + \epsilon_0 \epsilon_1) & 1 - 2(\epsilon_1^2 + \epsilon_2^2) \end{pmatrix}$$

column vectors to have unit length, and three require the column vectors to be mutually orthogonal. Alternatively, the orthogonality of the rotation matrix can be seen by considering the frames in reverse order. The orientation of coordinate frame j relative to coordinate frame i is the rotation matrix ${}^i\mathbf{R}_j$ whose rows are clearly the columns of the matrix ${}^j\mathbf{R}_i$. Rotation matrices are combined through simple matrix multiplication such that the orientation of frame i relative to frame k can be expressed as

$${}^k\mathbf{R}_i = {}^k\mathbf{R}_j {}^j\mathbf{R}_i.$$

In summary, ${}^j\mathbf{R}_i$ is the rotation matrix that transforms a vector expressed in coordinate frame i to a vector expressed in coordinate frame j . It provides a representation of the orientation of frame i relative to j and thus, can be a representation of rotation from frame i to frame j . Table 1.1 lists the equivalent rotation matrices for the other representations of orientation listed in this section. Table 1.2 contains the conversions from a known rotation matrix to these other representations.

Euler Angles

For a minimal representation, the orientation of coordinate frame i relative to coordinate frame j can be denoted as a vector of three angles (α, β, γ). These angles are known as Euler angles when each represents a rotation about an axis of a moving coordinate frame. In this way, the location of the axis of each successive rotation depends upon the preceding rotation(s), so the order of the

Table 1.2 Conversions from a rotation matrix to various representations of orientation

Rotation matrix:
${}^jR_i = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$
Z-Y-X Euler angles (α, β, γ) :
$\beta = \text{Atan2}(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2})$
$\alpha = \text{Atan2}\left(\frac{r_{21}}{\cos \beta}, \frac{r_{11}}{\cos \beta}\right)$
$\gamma = \text{Atan2}\left(\frac{r_{32}}{\cos \beta}, \frac{r_{33}}{\cos \beta}\right)$
X-Y-Z fixed angles (ψ, θ, ϕ) :
$\theta = \text{Atan2}(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2})$
$\psi = \text{Atan2}\left(\frac{r_{21}}{\cos \theta}, \frac{r_{11}}{\cos \theta}\right)$
$\phi = \text{Atan2}\left(\frac{r_{32}}{\cos \theta}, \frac{r_{33}}{\cos \theta}\right)$
Angle axis $\theta \hat{\mathbf{w}}$:
$\theta = \cos^{-1} \left(\frac{r_{11} + r_{22} + r_{33} - 1}{2} \right)$
$\hat{\mathbf{w}} = \frac{1}{2 \sin \theta} \begin{pmatrix} r_{32} - r_{23} \\ r_{13} - r_{31} \\ r_{21} - r_{12} \end{pmatrix}$
Unit quaternions $(\epsilon_0 \ \epsilon_1 \ \epsilon_2 \ \epsilon_3)^\top$:
$\epsilon_0 = \frac{1}{2} \sqrt{1 + r_{11} + r_{22} + r_{33}}$
$\epsilon_1 = \frac{r_{32} - r_{23}}{4\epsilon_0}$
$\epsilon_2 = \frac{r_{13} - r_{31}}{4\epsilon_0}$
$\epsilon_3 = \frac{r_{21} - r_{12}}{4\epsilon_0}$

rotations must accompany the three angles to define the orientation. For example, the symbols (α, β, γ) are used throughout this handbook to indicate Z-Y-X Euler angles. Taking the moving frame i and the fixed frame j to be initially coincident, α is the rotation about the $\hat{\mathbf{z}}$ axis of frame i , β is the rotation about the rotated $\hat{\mathbf{y}}$ axis of frame i , and finally, γ is the rotation about the twice rotated $\hat{\mathbf{x}}$ axis of frame i . The equivalent rotation matrix jR_i is given in Table 1.1. Z-Y-Z and Z-X-Z Euler angles are other commonly used conventions from among the 12 different possible orders of rotations.

Regardless of the order of rotations, an Euler angle representation of orientation always exhibits a singularity when the first and last rotations both occur about the same axis. This can be readily seen in Table 1.2 wherein the angles α and γ are undefined when $\beta = \pm 90^\circ$. (For Z-Y-Z and Z-X-Z Euler angles, the singularity occurs when the second rotation is 0° or 180° .) This creates a problem in relating the angular velocity vector of a body to the time derivatives of Euler angles, which somewhat lim-

its their usefulness in modeling robotic systems. This velocity relationship for Z-Y-X Euler angles is

$$\begin{pmatrix} \dot{\alpha} \\ \dot{\beta} \\ \dot{\gamma} \end{pmatrix} = \begin{pmatrix} -\sin \beta & 0 & 1 \\ \cos \beta \sin \gamma & \cos \gamma & 0 \\ \cos \beta \cos \gamma & -\sin \beta & 0 \end{pmatrix} \begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}. \quad (1.5)$$

Fixed Angles

A vector of three angles can also denote the orientation of coordinate frame i relative to coordinate frame j when each angle represents a rotation about an axis of a fixed reference frame. Appropriately, such angles are referred to as fixed angles, and the order of the rotations must again accompany the angles to define the orientation. X-Y-Z fixed angles, denoted here as (ψ, θ, ϕ) , are a common convention from among the, again, 12 different possible orders of rotations. Taking the moving frame i and the fixed frame j to be initially coincident, ψ is the yaw rotation about the fixed $\hat{\mathbf{x}}_j$ axis, θ is the pitch rotation about the fixed $\hat{\mathbf{y}}_j$ axis, and ϕ is the roll rotation about the fixed $\hat{\mathbf{z}}_j$ axis. As can be seen by comparing the respective equivalent rotation matrices in Table 1.1 and the respective conversions in Table 1.2, a set of X-Y-Z fixed angles is exactly equivalent to the same set of Z-Y-X Euler angles ($\alpha = \phi$, $\beta = \theta$, and $\gamma = \psi$). This result holds in general such that three rotations about the three axes of a fixed frame define the same orientation as the same three rotations taken in the opposite order about the three axes of a moving frame. Likewise, all fixed-angle representations of orientations suffer from the singularity discussed for Euler angles. Also, the relationship between the time derivatives of fixed angles and the angular velocity vector is similar to the relationship for Euler angles.

Angle-Axis

A single angle θ in combination with a unit vector $\hat{\mathbf{w}}$ can also denote the orientation of coordinate frame i relative to coordinate frame j . In this case, frame i is rotated through the angle θ about an axis defined by the vector $\hat{\mathbf{w}} = (w_x \ w_y \ w_z)^\top$ relative to frame j . The vector $\hat{\mathbf{w}}$ is sometimes referred to as the equivalent axis of a finite rotation. The angle-axis representation, typically written as either $\theta \hat{\mathbf{w}}$ or $(\theta w_x \ \theta w_y \ \theta w_z)^\top$, is superabundant by one because it contains four parameters. The auxiliary relationship that resolves this is the unit magnitude of vector $\hat{\mathbf{w}}$. Even with this auxiliary relationship, the angle-axis representation is not unique because rotation through an angle of $-\theta$ about $-\hat{\mathbf{w}}$ is equivalent to a rotation through θ about $\hat{\mathbf{w}}$. Table 1.3 contains

Table 1.3 Conversions from angle-axis to unit quaternion representations of orientation and vice versaAngle-axis $\theta \hat{\mathbf{w}}$ to unit quaternion $(\epsilon_0 \ \epsilon_1 \ \epsilon_2 \ \epsilon_3)^\top$:

$$\begin{aligned}\epsilon_0 &= \cos \frac{\theta}{2} \\ \epsilon_1 &= w_x \sin \frac{\theta}{2} \\ \epsilon_2 &= w_y \sin \frac{\theta}{2} \\ \epsilon_3 &= w_z \sin \frac{\theta}{2}\end{aligned}$$

Unit quaternion $(\epsilon_0 \ \epsilon_1 \ \epsilon_2 \ \epsilon_3)^\top$ to angle-axis $\theta \hat{\mathbf{w}}$:

$$\begin{aligned}\theta &= 2 \cos^{-1} \epsilon_0 \\ w_x &= \frac{\epsilon_1}{\sin \frac{\theta}{2}} \\ w_y &= \frac{\epsilon_2}{\sin \frac{\theta}{2}} \\ w_z &= \frac{\epsilon_3}{\sin \frac{\theta}{2}}\end{aligned}$$

the conversions from angle-axis representation to unit quaternions and vice versa. The conversions from these two representations to Euler angles or fixed angles can be easily found by using the conversions in Table 1.2 in conjunction with the equivalent rotation matrices in Table 1.1. Velocity relationships are more easily dealt with using the closely related quaternion representation.

Quaternions

The quaternion representation of orientation due to *Hamilton* [1.1], while largely superseded by the simpler vector representations of *Gibbs* [1.2] and *Grassmann* [1.3], is extremely useful for problems in robotics that result in representational singularities in the vector/matrix notation [1.4]. Quaternions do not suffer from singularities as Euler angles do.

A quaternion ϵ is defined to have the form

$$\epsilon = \epsilon_0 + \epsilon_1 i + \epsilon_2 j + \epsilon_3 k,$$

where the components ϵ_0 , ϵ_1 , ϵ_2 , and ϵ_3 are scalars, sometimes referred to as Euler parameters, and i , j , and k are operators. The operators are defined to satisfy the following combinatory rules:

$$\begin{aligned}ii &= jj = kk = -1, \\ ij &= k, \quad jk = i, \quad ki = j, \\ ji &= -k, \quad kj = -i, \quad ik = -j.\end{aligned}$$

Two quaternions are added by adding the respective components separately, so the operators act as separators. The null element for addition is the quaternion $\mathbf{0} = 0 + 0i + 0j + 0k$, and quaternion sums are associative, commutative, and distributive. The null element for multiplication is $\mathbf{1} = 1 + 0i + 0j + 0k$, as can be seen

using $\mathbf{1}\epsilon = \epsilon$ for any quaternion ϵ . Quaternion products are associative and distributive, but not commutative, and following the conventions of the operators and addition, have the form

$$\begin{aligned}\mathbf{ab} &= a_0 b_0 - a_1 b_1 - a_2 b_2 - a_3 b_3 \\ &\quad + (a_0 b_1 + a_1 b_0 + a_2 b_3 - a_3 b_2)i \\ &\quad + (a_0 b_2 + a_2 b_0 + a_3 b_1 - a_1 b_3)j \\ &\quad + (a_0 b_3 + a_3 b_0 + a_1 b_2 - a_2 b_1)k.\end{aligned}\quad (1.6)$$

It is convenient to define the conjugate of a quaternion

$$\tilde{\epsilon} = \epsilon_0 - \epsilon_1 i - \epsilon_2 j - \epsilon_3 k,$$

so that

$$\epsilon \tilde{\epsilon} = \tilde{\epsilon} \epsilon = \epsilon_0^2 + \epsilon_1^2 + \epsilon_2^2 + \epsilon_3^2.$$

A unit quaternion can then be defined such that $\epsilon \tilde{\epsilon} = 1$. Often, ϵ_0 is referred to as the scalar part of the quaternion, and $(\epsilon_1 \ \epsilon_2 \ \epsilon_3)^\top$ is referred to as the vector part.

Unit quaternions are used to describe orientation, and the unit magnitude provides the auxiliary relationship to resolve the use of superabundant (four) coordinates. A vector is defined in quaternion notation as a quaternion with $\epsilon_0 = 0$. Thus, a vector $\mathbf{p} = (p_x \ p_y \ p_z)^\top$ can be expressed as a quaternion $\mathbf{p} = p_x i + p_y j + p_z k$. For any unit quaternion ϵ , the operation $\epsilon \mathbf{p} \tilde{\epsilon}$ performs a rotation of the vector $\mathbf{p}$ about the direction $(\epsilon_1 \ \epsilon_2 \ \epsilon_3)^\top$. This is clearly seen by expanding the operation $\epsilon \mathbf{p} \tilde{\epsilon}$ and comparing the results with the equivalent rotation matrix listed in Table 1.1. Also, as shown in Table 1.3, unit quaternions are closely related to the angle-axis representation of orientation. ϵ_0 corresponds to the angle of rotation, while ϵ_1 , ϵ_2 , and ϵ_3 define the axis of rotation.

For velocity analysis, the time derivative of the quaternion can be related to the angular velocity vector as

$$\begin{pmatrix} \dot{\epsilon}_0 \\ \dot{\epsilon}_1 \\ \dot{\epsilon}_2 \\ \dot{\epsilon}_3 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} -\epsilon_1 & -\epsilon_2 & -\epsilon_3 \\ \epsilon_0 & \epsilon_3 & -\epsilon_2 \\ -\epsilon_3 & \epsilon_0 & \epsilon_1 \\ \epsilon_2 & -\epsilon_1 & \epsilon_0 \end{pmatrix} \begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}. \quad (1.7)$$

While a unit quaternion represents only the orientation of a body, quaternions may be dualized [1.5–7] to create an algebra that provides a description of the position and orientation of a body in space. Other combined representations are discussed in the following sections.

1.2.3 Homogeneous Transformations

The preceding sections have addressed representations of position and orientation separately. With homogeneous transformations, position vectors and rotation

matrices are combined together in a compact notation. Any vector ${}^i\mathbf{r}$ expressed relative to the i coordinate frame can be expressed relative to the j coordinate frame if the position and orientation of the i frame are known relative to the j frame. Using the notation of Sect. 1.2.1, the position of the origin of coordinate frame i relative to coordinate frame j can be denoted by the vector ${}^j\mathbf{p}_i = ({}^j p_i^x \ {}^j p_i^y \ {}^j p_i^z)^\top$. Using the notation of Sect. 1.2.2, the orientation of frame i relative to frame j can be denoted by the rotation matrix ${}^j\mathbf{R}_i$. Thus,

$${}^j\mathbf{r} = {}^j\mathbf{R}_i {}^i\mathbf{r} + {}^j\mathbf{p}_i. \quad (1.8)$$

This equation can be written

$$\begin{pmatrix} {}^j\mathbf{r} \\ 1 \end{pmatrix} = \begin{pmatrix} {}^j\mathbf{R}_i & {}^j\mathbf{p}_i \\ \mathbf{0}^\top & 1 \end{pmatrix} \begin{pmatrix} {}^i\mathbf{r} \\ 1 \end{pmatrix}, \quad (1.9)$$

where

$${}^j\mathbf{T}_i = \begin{pmatrix} {}^j\mathbf{R}_i & {}^j\mathbf{p}_i \\ \mathbf{0}^\top & 1 \end{pmatrix} \quad (1.10)$$

is the 4×4 homogeneous transformation matrix and $({}^j\mathbf{r} \ 1)^\top$ and $({}^i\mathbf{r} \ 1)^\top$ are the homogeneous representations of the position vectors ${}^j\mathbf{r}$ and ${}^i\mathbf{r}$. The matrix ${}^j\mathbf{T}_i$ transforms vectors from coordinate frame i to coordinate frame j . Its inverse ${}^j\mathbf{T}_i^{-1}$ transforms vectors from coordinate frame j to coordinate frame i .

$${}^j\mathbf{T}_i^{-1} = {}^i\mathbf{T}_j = \begin{pmatrix} {}^j\mathbf{R}_i^\top & -{}^j\mathbf{R}_i^\top {}^j\mathbf{p}_i \\ \mathbf{0}^\top & 1 \end{pmatrix}. \quad (1.11)$$

Composition of 4×4 homogeneous transformation matrices is accomplished through simple matrix multiplication, just as in the case of 3×3 rotation matrices. Therefore, ${}^k\mathbf{T}_i = {}^k\mathbf{T}_j {}^j\mathbf{T}_i$. Since matrix multiplications do not commute, the order or sequence is important.

The homogeneous transformation of a simple rotation about an axis is sometimes denoted **Rot** such that a rotation of θ about an axis $\hat{\mathbf{z}}$ is

$$\mathbf{Rot}(\hat{\mathbf{z}}, \theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (1.12)$$

Similarly, the homogeneous transformation of a simple translation along an axis is sometimes denoted **Trans** such that a translation of d along an axis $\hat{\mathbf{x}}$ is

$$\mathbf{Trans}(\hat{\mathbf{x}}, d) = \begin{pmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (1.13)$$

Homogeneous transformations are particularly attractive when compact notation is desired and/or when ease of programming is the most important consideration. This is not, however, a computationally efficient representation since it introduces a large number of additional multiplications by ones and zeros. Although homogeneous transformation matrices technically contain 16 elements, four are defined to be zero or one, and the remaining elements are composed of a rotation matrix and a position vector. Therefore, the only truly superabundant coordinates come from the rotation matrix component, so the relevant auxiliary relationships are those associated with the rotation matrix.

1.2.4 Screw Transformations

The transformation in (1.8) can be viewed as composed of a rotation between coordinate frames i and j and a separate displacement between those frames. To get from frame i to frame j , one could perform the rotation first, followed by the displacement, or vice versa. Alternatively, the spatial displacement between the frames can be expressed, except in the case of a pure translation, as a rotation about a unique line combined with a translation parallel to that line.

Chasles' Theorem

Chasles' theorem, in the form stated by Chirikjian and Kyatkin [1.8], has two parts. The first states that:

Any displacement of a body in space can be accomplished by means of a translation of a designated point from its initial to its final position, followed by a rotation of the whole body about that point to bring it into its final orientation.

The second part states that:

Any displacement of a body in space can be accomplished by means of a rotation of the body about a unique line in space accompanied by a translation of the body parallel to that line.

Such a line is called a screw axis, and it is this second result that is usually thought of as Chasles' theorem.

The first part of the theorem is almost axiomatic. A designated point in a body anywhere in Euclidean space can be displaced from a given initial position to a given final position. By further requiring that all points in the body traverse the same displacement, the body translates so that the designated point moves from its initial position to its final position. The body can then be rotated about that point into any given final orientation.

The second part of the theorem depends on this representation of a spatial displacement and requires a more complex argument. A preliminary theorem due to Euler allows greater specificity about the rotation of the body: *Any displacement of a body in which one point remains fixed is equivalent to a rotation of that body about a unique axis passing through that point.* Geometrically, embedding three points in the moving body and letting one be the fixed point about which rotation occurs, each of the other two will have initial and final positions. The right bisector planes of the lines joining the initial and final positions in each case necessarily contain the fixed point. Any line in the bisector plane can be the axis of a rotation that carries the corresponding point from its initial to its final position. Therefore, the unique line common to the two bisector planes is such that rotation about it will carry any point in the body from its initial to its final position. The rigidity condition requires that all planes in the body that contain that line rotate through the same angle.

For any rotation of a rigid body described by a rotation matrix jR_i , Euler's theorem states that there is a unique eigenvector $\hat{w}$ such that

$${}^jR_i \hat{w} = \hat{w}, \quad (1.14)$$

where $\hat{w}$ is a unit vector parallel to the axis of rotation. This expression requires a unit eigenvalue of jR_i corresponding to the eigenvector $\hat{w}$. The remaining two eigenvalues are $\cos \theta \pm i \sin \theta$, where i is the complex operator and θ is the angle of rotation of the body about the axis.

Combining the first part of Chasles' theorem with Euler's theorem, a general spatial displacement can be expressed as a translation taking a point from its initial to its final position, followed by a unique rotation about a unique axis through that point that carries the body from its initial to its final orientation. Resolving the translation into components in the direction of and orthogonal to the rotation axis, every point in the body has the same displacement component in the direction of the axis because rotation about it does not affect that component. Projected into a plane normal to the rotation axis, the kinematic geometry of the displacement is identical to that of planar motion. Just as there is a unique point in the plane about which a body could be rotated between two given positions, there is a unique point in the projection plane. If the rotation axis is moved to pass through that point, the spatial displacement can be accomplished by a rotation about that axis combined with a translation along it, as stated by the theorem.

The line about which rotation takes place is called the screw axis of the displacement. The ratio of the linear displacement d to the rotation θ is referred to as the pitch h of the screw axis [1.4]. Thus,

$$d = h\theta. \quad (1.15)$$

The screw axis of a pure translation is not unique. Any line parallel to the translation can be regarded as the screw axis, and since the rotation θ is zero, the axis of a translation is said to have infinite pitch.

A screw axis is most conveniently represented in any reference frame by means of a unit vector $\hat{w}$ parallel to it and the position vector ρ of any point lying on it. Additional specification of the pitch h and the angle of rotation θ completely defines the location of a second coordinate frame relative to the reference frame. Thus, a total of eight coordinates define a screw transformation, which is superabundant by two. The unit magnitude of vector $\hat{w}$ provides one auxiliary relationship, but in general, there is no second auxiliary relationship because the same screw axis is defined by all points lying on it, which is to say that the vector ρ contains one free coordinate.

Algebraically, a screw displacement is represented by

$${}^j\mathbf{r} = {}^jR_i({}^i\mathbf{r} - \rho) + d\hat{w} + \rho. \quad (1.16)$$

Comparing this expression to (1.8) yields

$${}^j\mathbf{p}_i = d\hat{w} + (\mathbf{1}_{3 \times 3} - {}^jR_i)\rho. \quad (1.17)$$

$\mathbf{1}_{3 \times 3}$ denotes the 3×3 identity matrix. An expression for d is easily obtained by taking the inner product of both sides of the equation with $\hat{w}$.

$$d = \hat{w}^\top {}^j\mathbf{p}_i. \quad (1.18)$$

The matrix $\mathbf{1}_{3 \times 3} - {}^jR_i$ is singular, so (1.17) cannot be solved to give a unique value of ρ , but since ρ can represent any point on the screw axis, this would not be appropriate. One component of ρ can be arbitrarily chosen, and any two of the component equations can then be solved to find the two other components of ρ . All other points on the screw axis are then given by $\rho + k\hat{w}$, where k can take any value.

Table 1.4 contains the conversions between screw transformations and homogeneous transformations. Note that the equivalent rotation matrix for a screw transformation has the same form as the equivalent rotation matrix for an angle-axis representation of orientation in Table 1.1. Also, the auxiliary relationship that the vector ρ be orthogonal to the screw axis ($\hat{w}^\top \rho = 0$) is used in Table 1.4 to provide a unique conversion to the

Table 1.4 Conversions from a screw transformation to a homogeneous transformation and vice versa, with abbreviations $c_\theta := \cos \theta$, $s_\theta := \sin \theta$, and $v_\theta := 1 - \cos \theta$

Screw transformation to homogeneous transformation:

$${}^j\mathbf{R}_i = \begin{pmatrix} w_x^2 v_\theta + c_\theta & w_x w_y v_\theta - w_z s_\theta & w_x w_z v_\theta + w_y s_\theta \\ w_x w_y v_\theta + w_z s_\theta & w_y^2 v_\theta + c_\theta & w_y w_z v_\theta - w_x s_\theta \\ w_x w_z v_\theta - w_y s_\theta & w_y w_z v_\theta + w_x s_\theta & w_z^2 v_\theta + c_\theta \end{pmatrix}$$

$${}^j\mathbf{p}_i = (\mathbf{1}_{3 \times 3} - {}^j\mathbf{R}_i)\boldsymbol{\rho} + h\theta \hat{\mathbf{w}}$$

Homogeneous transformation to screw transformation:

$$\mathbf{l} = \begin{pmatrix} r_{32} - r_{23} \\ r_{13} - r_{31} \\ r_{21} - r_{12} \end{pmatrix}^\top$$

$$\theta = \text{sign}(\mathbf{l}^\top {}^j\mathbf{p}_i) \left| \cos^{-1} \left(\frac{r_{11} + r_{22} + r_{33} - 1}{2} \right) \right|$$

$$h = \frac{\mathbf{l}^\top {}^j\mathbf{p}_i}{2\theta \sin \theta}$$

$$\boldsymbol{\rho} = \frac{(\mathbf{1}_{3 \times 3} - {}^j\mathbf{R}_i^\top) {}^j\mathbf{p}_i}{2(1 - \cos \theta)}$$

$$\hat{\mathbf{w}} = \frac{\mathbf{l}}{2 \sin \theta}$$

screw transformation. The inverse result, that of finding the rotation matrix ${}^j\mathbf{R}_i$ and the translation ${}^j\mathbf{p}_i$ corresponding to a given screw displacement, is found from Rodrigues' equation.

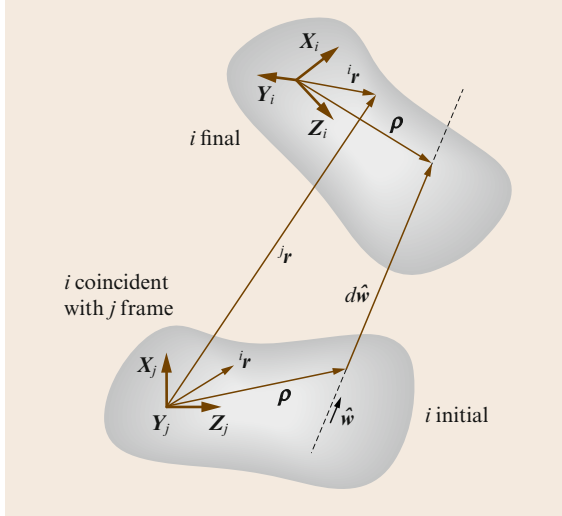


Fig. 1.1 Initial and final positions of an arbitrary point in a body undergoing a screw displacement; ${}^i\mathbf{r}$ is the position of the point relative to the moving frame, which is coincident with the fixed reference frame j in its initial position; ${}^j\mathbf{r}$ is the position of the point relative to the fixed frame after the screw displacement of the moving body

Rodrigues' Equation

Given a screw axis, the angular displacement of a body about it, and the translation of the body along it, the displacement of an arbitrary point in that body can be found. Viewing a matrix transformation as describing the displacement of the body, this is equivalent to finding the matrix transformation equivalent to a given screw displacement.

Referring to Fig. 1.1, the position vectors of a point before and after a screw displacement can be geometrically related as

$${}^j\mathbf{r} = {}^i\mathbf{r} + d\hat{\mathbf{w}} + \sin \theta \hat{\mathbf{w}} \times ({}^i\mathbf{r} - \boldsymbol{\rho}) - (1 - \cos \theta)({}^i\mathbf{r} - \boldsymbol{\rho}) - ({}^i\mathbf{r} - \boldsymbol{\rho}) \cdot \hat{\mathbf{w}} \hat{\mathbf{w}}, \quad (1.19)$$

where ${}^i\mathbf{r}$ and ${}^j\mathbf{r}$ denote the initial and final positions of the point, $\hat{\mathbf{w}}$ and $\boldsymbol{\rho}$ specify the screw axis, and θ and d give the displacement about it. This result is usually referred to as Rodrigues' equation [1.9], which can be written as a matrix transformation [1.10],

$${}^j\mathbf{r} = {}^j\mathbf{R}_i {}^i\mathbf{r} + {}^j\mathbf{p}_i, \quad (1.20)$$

since, when expanded, it gives three linear equations for the components of ${}^j\mathbf{r}$ in terms of those of ${}^i\mathbf{r}$.

$${}^j\mathbf{R}_i = \begin{pmatrix} w_x^2 v_\theta + c_\theta & w_x w_y v_\theta - w_z s_\theta & w_x w_z v_\theta + w_y s_\theta \\ w_x w_y v_\theta + w_z s_\theta & w_y^2 v_\theta + c_\theta & w_y w_z v_\theta - w_x s_\theta \\ w_x w_z v_\theta - w_y s_\theta & w_y w_z v_\theta + w_x s_\theta & w_z^2 v_\theta + c_\theta \end{pmatrix}$$

$${}^j\mathbf{p}_i = (\mathbf{1}_{3 \times 3} - {}^j\mathbf{R}_i)\boldsymbol{\rho} + h\theta \hat{\mathbf{w}},$$

where the abbreviations are $c_\theta := \cos \theta$, $s_\theta := \sin \theta$, and $v_\theta = 1 - \cos \theta$. The rotation matrix ${}^j\mathbf{R}_i$ expressed in this form is also called the screw matrix, and these equations give the elements of ${}^j\mathbf{R}_i$ and ${}^j\mathbf{p}_i$ in terms of the screw parameters.

An exception arises in the case of a pure translation, for which $\theta = 0$ and Rodrigues' equation becomes

$${}^j\mathbf{r} = {}^i\mathbf{r} + d\hat{\mathbf{w}}. \quad (1.21)$$

Substituting for this case, ${}^j\mathbf{R}_i = \mathbf{1}_{3 \times 3}$ and ${}^j\mathbf{p}_i = d\hat{\mathbf{w}}$.

Additional information on screw theory can be found in [1.11–15].

1.2.5 Matrix Exponential Parameterization

The position and orientation of a body can also be expressed in a unified fashion with an exponential mapping. This approach is introduced first with its application to pure rotation and expanded to rigid-body

motion. More details on the approach can be found in [1.16] and [1.17].

Exponential Coordinates for Rotation

The set of all orthogonal matrices with determinant 1, which is the set of all rotation matrices $\mathbf{R}$, is a *group* under the operation of matrix multiplication denoted as $SO(3) \subset \mathbb{R}^{3 \times 3}$ [1.18]. This stands for *special orthogonal* wherein *special* alludes to the $\det \mathbf{R}$ being +1 instead of ± 1 . The set of rotation matrices satisfies the four axioms of a group:

$$\begin{aligned} \text{closure:} & \quad \mathbf{R}_1 \mathbf{R}_2 \in SO(3) \quad \forall \mathbf{R}_1, \mathbf{R}_2 \in SO(3); \\ \text{identity:} & \quad \mathbf{1}_{3 \times 3} \mathbf{R} = \mathbf{R} \mathbf{1}_{3 \times 3} = \mathbf{R} \quad \forall \mathbf{R} \in SO(3); \\ \text{inverse:} & \quad \mathbf{R}^\top \in SO(3) \text{ is the unique inverse of } \mathbf{R} \\ & \quad \forall \mathbf{R} \in SO(3); \\ \text{associativity:} & \quad (\mathbf{R}_1 \mathbf{R}_2) \mathbf{R}_3 = \mathbf{R}_1 (\mathbf{R}_2 \mathbf{R}_3) \quad \forall \mathbf{R}_1, \mathbf{R}_2, \mathbf{R}_3 \\ & \quad \in SO(3). \end{aligned}$$

In the angle-axis representation presented in Sect. 1.2.2, orientation is expressed as an angle θ of rotation about an axis defined by the unit vector $\hat{\mathbf{w}}$. The equivalent rotation matrix found in Table 1.1 can be expressed as the exponential map

$$\begin{aligned} \mathbf{R} = e^{S(\hat{\mathbf{w}})\theta} &= \mathbf{1}_{3 \times 3} + \theta S(\hat{\mathbf{w}}) + \frac{\theta^2}{2!} S(\hat{\mathbf{w}})^2 \\ &+ \frac{\theta^3}{3!} S(\hat{\mathbf{w}})^3 + \dots, \end{aligned} \quad (1.22)$$

where $S(\hat{\mathbf{w}})$ is the unit skew-symmetric matrix

$$S(\hat{\mathbf{w}}) = \begin{pmatrix} 0 & -w_z & w_y \\ w_z & 0 & -w_x \\ -w_y & w_x & 0 \end{pmatrix}. \quad (1.23)$$

Thus, the exponential map transforms a skew-symmetric matrix $S(\hat{\mathbf{w}})$ that corresponds to an axis of rotation $\hat{\mathbf{w}}$ into an orthogonal matrix $\mathbf{R}$ that corresponds to a rotation about the axis $\hat{\mathbf{w}}$ through an angle of θ . It can be shown that the closed-form expression for $e^{S(\hat{\mathbf{w}})\theta}$, which can be efficiently computed, is

$$e^{S(\hat{\mathbf{w}})\theta} = \mathbf{1}_{3 \times 3} + S(\hat{\mathbf{w}}) \sin \theta + S(\hat{\mathbf{w}})^2 (1 - \cos \theta). \quad (1.24)$$

The components of $(\theta w_x \theta w_y \theta w_z)^\top$, which are related to the elements of the rotation matrix $\mathbf{R}$ in Table 1.2, are referred to as the *exponential coordinates* for $\mathbf{R}$.

Exponential Coordinates for Rigid-Body Motion

As indicated in Sect. 1.2.3, the position and orientation of a body can be expressed by the combination of a position vector $\mathbf{p} \in \mathbb{R}^3$ and a rotation matrix $\mathbf{R} \in SO(3)$.

The product space of $\mathbb{R}^3$ with $SO(3)$ is the group known as $SE(3)$, which stands for *special Euclidean*.

$$\begin{aligned} SE(3) &= \{(\mathbf{p}, \mathbf{R}) : \mathbf{p} \in \mathbb{R}^3, \mathbf{R} \in SO(3)\} \\ &= \mathbb{R}^3 \times SO(3). \end{aligned}$$

The set of homogeneous transformations satisfies the four axioms of a group:

- *closure*: $\mathbf{T}_1 \mathbf{T}_2 \in SE(3) \quad \forall \mathbf{T}_1, \mathbf{T}_2 \in SE(3)$;
- *identity*: $\mathbf{1}_{4 \times 4} \mathbf{T} = \mathbf{T} \mathbf{1}_{4 \times 4} = \mathbf{T} \quad \forall \mathbf{T} \in SE(3)$;
- *inverse*: the unique inverse of $\mathbf{T} \quad \forall \mathbf{T} \in SE(3)$ is given in (1.11);
- *associativity*: $(\mathbf{T}_1 \mathbf{T}_2) \mathbf{T}_3 = \mathbf{T}_1 (\mathbf{T}_2 \mathbf{T}_3) \quad \forall \mathbf{T}_1, \mathbf{T}_2, \mathbf{T}_3 \in SE(3)$.

In the screw transformation representation in Sect. 1.2.4, position and orientation are expressed by the angle θ of rotation about a screw axis defined by the unit vector $\hat{\mathbf{w}}$, the point $\mathbf{p}$ on the axis such that $\hat{\mathbf{w}}^\top \mathbf{p} = 0$, and the pitch h of the screw axis. The equivalent homogeneous transformation found in Table 1.4 can be expressed as the exponential map

$$\mathbf{T} = e^{\hat{\xi}\theta} = \mathbf{1}_{4 \times 4} + \hat{\xi}\theta + \frac{(\hat{\xi}\theta)^2}{2!} + \frac{(\hat{\xi}\theta)^3}{3!} + \dots, \quad (1.25)$$

where

$$\hat{\xi} = \begin{pmatrix} S(\hat{\mathbf{w}}) & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{pmatrix} \quad (1.26)$$

is the generalization of the unit skew-symmetric matrix $S(\hat{\mathbf{w}})$ known as a *twist*. The twist coordinates of $\hat{\xi}$ are given by $\xi := (\hat{\mathbf{w}}^\top \mathbf{v}^\top)^\top$. It can be shown that the closed-form expression for $e^{\hat{\xi}\theta}$ is

$$e^{\hat{\xi}\theta} = \begin{pmatrix} e^{S(\hat{\mathbf{w}})\theta} & (\mathbf{1}_{3 \times 3} - e^{S(\hat{\mathbf{w}})\theta})(\hat{\mathbf{w}} \times \mathbf{v}) + \hat{\mathbf{w}}^\top \mathbf{v} \theta \hat{\mathbf{w}} \\ \mathbf{0}^\top & 1 \end{pmatrix}. \quad (1.27)$$

Comparison of this result with the conversion between homogeneous and screw transformations in Table 1.4 yields

$$\mathbf{v} = \mathbf{p} \times \hat{\mathbf{w}} \quad (1.28)$$

and

$$h = \hat{\mathbf{w}}^\top \mathbf{v}. \quad (1.29)$$

Thus, the exponential map for a twist transforms the initial pose of a body into its final pose. It gives the relative rigid-body motion. The vector $\xi\theta$ contains the *exponential coordinates* for the rigid-body transformation.

As for screw transformations, the case of pure translation is unique. In this case, $\hat{\mathbf{w}} = \mathbf{0}$, so

$$\mathbf{e}^{\hat{\xi}\theta} = \begin{pmatrix} \mathbf{1}_{3 \times 3} & \theta \mathbf{v} \\ \mathbf{0}^\top & 1 \end{pmatrix}. \quad (1.30)$$

1.2.6 Plücker Coordinates

A minimum of four coordinates are needed to define a line in space. The Plücker coordinates of a line form a six-dimensional vector, so they are superabundant by two. They can be viewed as a pair of three-dimensional vectors; one is parallel to the line, and the other is the *moment* of that vector about the origin. Thus, if $\mathbf{u}$ is any vector parallel to the line and $\boldsymbol{\rho}$ is the position of any point on the line relative to the origin, the Plücker coordinates (L, M, N, P, Q , and R) are given by

$$(L, M, N) = \mathbf{u}^\top; (P, Q, R) = (\boldsymbol{\rho} \times \mathbf{u})^\top. \quad (1.31)$$

For simply defining a line, the magnitude of $\mathbf{u}$ is not unique, nor is the component of $\boldsymbol{\rho}$ parallel to $\mathbf{u}$. Two auxiliary relationships are imposed to reduce the set to just four independent coordinates. One is that the scalar product of the two three-dimensional vectors is identically zero.

$$LP + MQ + NR \equiv 0. \quad (1.32)$$

The other is the invariance of the line designated when the coordinates are all multiplied by the same scaling factor.

$$(L, M, N, P, Q, R) \equiv (kL, kM, kN, kP, kQ, kR). \quad (1.33)$$

This relationship may take the form of constraining $\mathbf{u}$ to have unit magnitude so that L, M , and N are the direction cosines.

In this handbook, it is often useful to express velocities in Plücker coordinates, wherein unlike the definition of lines, the magnitudes of the two three-dimensional vectors are not arbitrary. This leads to the motor notation of *von Mises* [1.9, 19] and *Everett* [1.20]. For instantaneously coincident coordinate frames, one fixed and the

other embedded in the moving body, $\boldsymbol{\omega}$ is the angular velocity of the body and $\mathbf{v}_O$ is the velocity of the origin O of the body-fixed frame when both are expressed relative to the fixed frame. This provides a Plücker coordinate system for the spatial velocity $\mathbf{v}$ of the body. The Plücker coordinates of $\mathbf{v}$ are simply the Cartesian coordinates of $\boldsymbol{\omega}$ and $\mathbf{v}_O$,

$$\mathbf{v} = \begin{pmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{pmatrix}. \quad (1.34)$$

The transformation from Plücker coordinate system i to Plücker coordinate system j for spatial velocities is achieved with the spatial transform jX_i . If $\mathbf{v}_i$ and $\mathbf{v}_j$ denote the spatial velocities of a body relative to the i and j frames, respectively, and ${}^j\mathbf{p}_i$ and ${}^j\mathbf{R}_i$ denote the position and orientation of frame i relative to frame j ,

$$\mathbf{v}_j = {}^jX_i \mathbf{v}_i, \quad (1.35)$$

where

$${}^jX_i = \begin{pmatrix} {}^j\mathbf{R}_i & \mathbf{0}_{3 \times 3} \\ S({}^j\mathbf{p}_i){}^j\mathbf{R}_i & {}^j\mathbf{R}_i \end{pmatrix}, \quad (1.36)$$

such that

$${}^jX_i^{-1} = {}^iX_j = \begin{pmatrix} {}^i\mathbf{R}_j & \mathbf{0}_{3 \times 3} \\ -{}^i\mathbf{R}_j S({}^j\mathbf{p}_i) & {}^i\mathbf{R}_j \end{pmatrix} \quad (1.37)$$

and

$${}^kX_i = {}^kX_j {}^jX_i, \quad (1.38)$$

and $S({}^j\mathbf{p}_i)$ is the skew-symmetric matrix

$$\begin{pmatrix} 0 & -{}^j p_i^z & {}^j p_i^y \\ {}^j p_i^z & 0 & -{}^j p_i^x \\ -{}^j p_i^y & {}^j p_i^x & 0 \end{pmatrix}. \quad (1.39)$$

Spatial vector notation, which includes the spatial velocities and transforms briefly mentioned here, is treated in greater depth in Sect. 2.2. Specifically, Table 2.1 gives a computationally efficient algorithm for applying a spatial transform.

1.3 Joint Kinematics

Unless explicitly stated otherwise, the kinematic description of robotic mechanisms typically employs a number of idealizations. The links that compose the robotic mechanism are assumed to be perfectly rigid

bodies having surfaces that are geometrically perfect in both position and shape. Accordingly, these rigid bodies are connected together at joints where their idealized surfaces are in ideal contact without any

clearance between them. The respective geometries of these surfaces in contact determine the freedom of motion between the two links, or the *joint kinematics*.

A kinematic joint is a connection between two bodies that constrains their relative motion. Two bodies that are in contact with one another create a simple kinematic joint. The surfaces of the two bodies that are in contact are able to move over one another, thereby permitting relative motion of the two bodies. Simple kinematic joints are classified as lower pair joints if contact occurs over surfaces [1.22] and as higher pair joints if contact occurs only at points or along lines.

A joint model describes the motion of a frame fixed in one body of a joint relative to a frame fixed in the other body. The motion is expressed as a function of the joint's motion variables, and other elements of a joint model include the rotation matrix, position vector, free modes, and constrained modes. The free modes of a joint define the directions in which motion is allowed. They are represented by the $6 \times n_i$ matrix Φ_i whose columns are the Plücker coordinates of the allowable motion. This

matrix relates the spatial velocity vector across the joint $\mathbf{v}_{\text{rel}}$ to the joint velocity vector $\dot{\mathbf{q}}$,

$$\mathbf{v}_{\text{rel}} = \Phi_i \dot{\mathbf{q}}. \quad (1.40)$$

In contrast, the constrained modes of a joint define the directions in which motion is not allowed. They are represented by the $6 \times (6 - n_i)$ matrix Φ_i^c that is complementary to Φ_i . Tables 1.5 and 1.6 contain the formulas of the joint models for all of the joints described in this section. They are used extensively for the dynamic analysis presented in Chap. 2. Additional information on joints can be found in Chap. 3.

1.3.1 Lower Pair Joints

Lower pair joints are mechanically attractive since wear is spread over the whole surface and lubricant is trapped in the small clearance space (in nonidealized systems) between the surfaces, resulting in relatively good lubrication. As can be proved [1.23] from the requirement for surface contact, there are only six possible forms of lower pair: revolute, prismatic, helical, cylindrical, spherical, and planar joints.

Table 1.5 Joint model formulas for one-degree-of-freedom lower pair joints, with abbreviations $c_{\theta_i} := \cos \theta_i$ and $s_{\theta_i} := \sin \theta_i$. (Adapted in part from Table 4.1 in [1.21])

Joint type	Joint rotation matrix ${}^j\mathbf{R}_i$	Position vector ${}^j\mathbf{p}_i$	Free modes Φ_i	Constrained modes Φ_i^c	Pose state vars.	$\dot{\mathbf{q}}_i$
Revolute R	$\begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	θ_i	$\dot{\theta}_i$
Prismatic P	$\mathbf{1}_{3 \times 3}$	$\begin{pmatrix} 0 \\ 0 \\ d_i \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$	d_i	$\dot{d}_i$
Helical H (pitch h)	$\begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ h\theta_i \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ h \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -h \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	θ_i	$\dot{\theta}_i$

Table 1.6 Joint model formulas for higher-degree-of-freedom lower pair joints, universal joint, rolling contact joint, and 6-DOF joint, with abbreviations $c_{\theta_i} := \cos \theta_i$ and $s_{\theta_i} := \sin \theta_i$. (Adapted in part from Table 4.1 in [1.21]) *The Euler angles α_i , β_i , and γ_i could be used in place of the unit quaternion ϵ_i to represent orientation

Joint type	Joint rotation matrix jR_i	Position vector jp_i	Free modes Φ_i	Constrained modes Φ_i^c	Pose Variables	Velocity Variables $\dot{q}_i$
Cylindrical C	$\begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ d_i \end{pmatrix}$	$\begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$	θ_i d_i	$\begin{pmatrix} \dot{\theta}_i \\ \dot{d}_i \end{pmatrix}$
Spherical* S	$\begin{pmatrix} \text{see} \\ \text{Table 1.1} \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	ϵ_i	$\omega_i \text{ rel}$
Planar	$\begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} c_{\theta_i} d_{xi} - s_{\theta_i} d_{yi} \\ s_{\theta_i} d_{xi} + c_{\theta_i} d_{yi} \\ 0 \end{pmatrix}$	$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	θ_i d_{xi} d_{yi}	$\begin{pmatrix} \dot{\theta}_i \\ \dot{d}_{xi} \\ \dot{d}_{yi} \end{pmatrix}$
Flat planar rolling contact (fixed radius r)	$\begin{pmatrix} c_{\theta_i} & -s_{\theta_i} & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} r\theta_i c_{\theta_i} - rs_{\theta_i} \\ -r\theta_i s_{\theta_i} - rc_{\theta_i} \\ 0 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 1 \\ r \\ 0 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & -r & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	θ_i	$\dot{\theta}_i$
Universal U	$\begin{pmatrix} c_{\alpha_i} c_{\beta_i} & -s_{\alpha_i} & c_{\alpha_i} s_{\beta_i} \\ s_{\alpha_i} c_{\beta_i} & c_{\alpha_i} & s_{\alpha_i} s_{\beta_i} \\ -s_{\beta_i} & 0 & c_{\beta_i} \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$	$\begin{pmatrix} -s_{\beta_i} & 0 \\ 0 & 1 \\ c_{\beta_i} & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$	$\begin{pmatrix} c_{\beta_i} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ s_{\beta_i} & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$	α_i β_i	$\begin{pmatrix} \dot{\alpha}_i \\ \dot{\beta}_i \end{pmatrix}$
6-DOF*	$\begin{pmatrix} \text{see} \\ \text{Table 1.1} \end{pmatrix}$	0p_i	$\mathbf{1}_{6 \times 6}$		ϵ_i 0p_i	$\begin{pmatrix} \omega_i \\ v_i \end{pmatrix}$

Revolute

The most general form of a revolute joint, often abbreviated as “R” and sometimes referred to colloquially as a hinge or pin joint, is a lower pair composed of two congruent surfaces of revolution. The surfaces are the same except one of them is an external surface, convex in any plane normal to the axis of revolution, and one is an internal surface, concave in any plane normal to

the axis. The surfaces may not be solely in the form of right circular cylinders, since surfaces of that form do not provide any constraint on axial sliding. A revolute joint permits only rotation of one of the bodies joined relative to the other. The position of one body relative to the other may be expressed as the angle between two lines normal to the joint axis, one fixed in each body. Thus, the joint has one *degree of freedom* (DOF). When

the $\hat{z}$ axis of coordinate frame i is aligned with a revolute joint axis, the formulas in Table 1.5 define the revolute joint model.

Prismatic

The most general form of a prismatic joint, often abbreviated as “P” and sometimes referred to colloquially as a sliding joint, is a lower pair formed from two congruent general cylindrical surfaces. These may not be right circular cylindrical surfaces. A general cylindrical surface is obtained by extruding any curve in a constant direction. Again, one surface is internal and the other is an external surface. A prismatic joint permits only sliding of one of the members joined relative to the other along the direction of extrusion. The position of one body relative to the other is determined by the distance between two points on a line parallel to the direction of sliding, with one point fixed in each body. Thus, this joint also has one degree of freedom. When the $\hat{z}$ axis of coordinate frame i is aligned with a prismatic joint axis, the formulas in Table 1.5 define the prismatic joint model.

Helical

The most general form of a helical joint, often abbreviated as “H” and sometimes referred to colloquially as a screw joint, is a lower pair formed from two helicoidal surfaces formed by extruding any curve along a helical path. The simple example is a screw and nut wherein the basic generating curve is a pair of straight lines. The angle of rotation about the axis of the screw joint θ is directly related to the distance of displacement of one body relative to the other along that axis d by the expression $d = h\theta$, where the constant h is called the pitch of the screw. When the $\hat{z}$ axis of coordinate frame i is aligned with a helical joint axis, the formulas in Table 1.5 define the helical joint model.

Cylindrical

A cylindrical joint, often abbreviated as “C”, is a lower pair formed by contact of two congruent right circular cylinders, one an internal surface and the other an external surface. It permits both rotation about the cylinder axis and sliding parallel to it. Therefore, it is a joint with two degrees of freedom. Lower pair joints with more than one degree of freedom are easily replaced by kinematically equivalent compound joints (see Sect. 1.3.3) that are serial chains of one-degree-of-freedom lower pairs. In the present case, the cylindrical joint can be replaced by a revolute in series with a prismatic joint whose direction of sliding is parallel to the revolute

axis. While simpler to implement using the geometric representation discussed in Sect. 1.4, this approach has disadvantages for dynamic simulation. Modeling a single cylindrical joint as a combination of a prismatic and revolute joint requires the addition of a virtual link between the two with zero mass and zero length. The massless link can create computational problems. When the $\hat{z}$ axis of coordinate frame i is aligned with a cylindrical joint axis, the formulas in Table 1.6 define the cylindrical joint model.

Spherical

A spherical joint, often abbreviated as “S”, is a lower pair formed by contact of two congruent spherical surfaces. Once again, one is an internal surface, and the other is an external surface. A spherical joint permits rotation about any line through the center of the sphere. Thus, it permits independent rotation about axes in up to three different directions and has three degrees of freedom. A spherical joint is easily replaced by a kinematically equivalent compound joint consisting of three revolutes that have concurrent axes. They do not need to be successively orthogonal, but often they are implemented that way. The arrangement is, in general, kinematically equivalent to a spherical joint, but it does exhibit a singularity when the revolute joint axes become coplanar. This is as compared to the native spherical joint that never has such a singularity. Likewise, if a spherical joint is modeled in simulation as three revolutes, computational difficulties again can arise from the necessary inclusion of massless virtual links having zero length. The joint model formulas of a spherical joint are given in Table 1.6.

Planar

A planar joint is formed by planar contacting surfaces. Like the spherical joint, it is a lower pair joint with three degrees of freedom. A kinematically equivalent compound joint consisting of a serial chain of three revolutes with parallel axes can replace a planar joint. As was the case with the spherical joint, the compound joint exhibits a singularity when the revolute axes become coplanar. When the $\hat{z}$ axis of coordinate frame i is aligned with the normal to the plane of contact, the formulas in Table 1.6 define the planar joint model.

1.3.2 Higher Pair Joints

Some higher pair joints also have attractive properties, particularly rolling pairs in which one body rolls without slipping over the surface of the other. This is mechan-

ically attractive since the absence of sliding means the absence of abrasive wear. However, since ideal contact occurs at a point, or along a line, application of a load across the joint may lead to very high local stresses resulting in other forms of material failure and, hence, wear. Higher pair joints can be used to create kinematic joints with special geometric properties, as in the case of a gear pair, or a cam and follower pair.

Rolling Contact

Rolling contact actually encompasses several different geometries. Rolling contact in planar motion permits one degree of freedom of relative motion as in the case of a roller bearing, for example. As noted above, rolling contact has desirable wear properties since the absence of sliding means the absence of abrasive wear. Planar rolling contact can take place along a line, thereby spreading the load and wear somewhat. Three-dimensional rolling contact allows rotation about any axis through the point of contact that is, in principle, unique. Hence, a three-dimensional rolling contact pair permits relative motion with three degrees of freedom. When the $\hat{z}$ axis of coordinate frame i is aligned with the axis of rotation and passes through the center of the roller of fixed radius r , the formulas in Table 1.6 define the planar rolling contact joint model for a roller on a flat surface.

Regardless of whether the joint is planar or three-dimensional, the *no-slip* condition associated with a rolling contact joint requires that the instantaneous relative velocity between the points on the two bodies in contact be zero. If P is the point of rolling contact between bodies i and j ,

$$\mathbf{v}_{P_i/P_j} = 0. \quad (1.41)$$

Likewise, relative acceleration is in the direction of the common normal to the two surfaces at the point of contact. Because the constraint associated with the joint is expressed in terms of velocity, it is nonholonomic, as discussed in Sect. 1.3.6. A more detailed discussion of the kinematic constraints for rolling contact is found in Sect. 17.2.2 of Chap. 17.

1.3.3 Compound Joints

Compound kinematic joints are connections between two bodies formed by chains of other members and simple kinematic joints. A compound joint may constrain the relative motion of the two bodies joined in the same way as a simple joint. In such a case, the two joints are said to be kinematically equivalent.

Universal

A universal joint, often abbreviated as “U” and referred to as a Cardan or Hooke joint, is a compound joint with two degrees of freedom. It consists of a serial chain of two revolute whose axes intersect orthogonally. The joint model for a universal joint, in which, from Euler angle notation, α_i is the first rotation about the Z -axis and then β_i is the rotation about the Y -axis, is given in Table 1.6. This is a joint for which the matrices Φ_i and Φ_i^c are not constant, so in general, $\dot{\Phi}_i \neq 0$ and $\dot{\Phi}_i^c \neq 0$. In this case, the orientation of the outboard reference frame varies with α_i .

1.3.4 6-DOF Joint

The motion of two bodies not jointed together can be modeled as a six-degree-of-freedom *joint* that introduces no constraints. This is particularly useful for mobile robots, such as aircraft, that make at most intermittent contact with the ground, and thus, a body in free motion relative to the fixed frame is termed a *floating base*. Such a free motion joint model enables the position and orientation of a floating base in space to be expressed with six joint variables. The 6-DOF joint model is included in Table 1.6.

1.3.5 Physical Realization

In an actual robotic mechanism, the joints may have physical limits beyond which motion is prohibited. The workspace (see Sect. 1.5) of a robotic manipulator is determined by considering the combined limits and freedom of motion of all of the joints within the mechanism. Revolute joints are easily actuated by rotating motors and are, therefore, extremely common in robotic systems. They may also be present as passive, unactuated joints. Also common, although less so than revolute, prismatic joints are relatively easily actuated by means of linear actuators such as hydraulic or pneumatic cylinders, ball screws, or screw jacks. They always have motion limits since unidirectional sliding can, in principle, produce infinite displacements. Screw joints are most often found in robotic mechanisms as constituents of linear actuators such as screw jacks and ball screws and are seldom used as primary kinematic joints. Joints with more than one degree of freedom are generally used passively in robotic mechanisms because each degree of freedom of an active joint must be separately actuated. Passive spherical joints are quite often found in robotic mechanisms, while passive planar joints are only occasionally found. The effect of an actuated spherical joint

is achieved by employing the kinematically equivalent combination of three revolute and actuating each. Universal joints are used in robotic mechanisms in both active and passive forms.

Serial chains are commonly denoted by the abbreviations for the joints they contain in the order in which they appear in the chain. For example, an RPR chain contains three links, the first jointed to the base with a revolute and to the second with a prismatic, while the second and third are jointed together with another revolute. If all of the joints are identical, the notation consists of the number of joints preceding the joint abbreviation, such as 6R for a six-axis serial-chain manipulator containing only revolute joints.

Joints are realized with hardware that is more complex than the idealizations presented in Sects. 1.3.1 and 1.3.2. For example, a revolute joint may be achieved with a ball bearing composed of a set of bearing balls trapped between two journals. The balls ideally roll without slipping on the journals, thereby taking advantage of the special properties of rolling contact joints. A prismatic joint may be realized by means of a roller-rail assembly.

1.3.6 Holonomic and Nonholonomic Constraints

With the exception of rolling contact, all of the constraints associated with the joints discussed in the preceding sections can be expressed mathematically by equations containing only the joint position variables.

These are called holonomic constraints. The number of equations, and hence the number of constraints, is $6 - n$, where n is the number of degrees of freedom of the joint. The constraints are intrinsically part of the axial joint model.

A nonholonomic constraint is one that cannot be expressed in terms of the position variables alone, but includes the time derivative of one or more of those variables. These constraint equations cannot be integrated to obtain relationships solely between the joint variables. The most common example in robotic systems arises from the use of a wheel or roller that rolls without slipping on another member. Nonholonomic constraints, particularly as they apply to wheeled robots, are discussed in more detail in Chap. 17.

1.3.7 Generalized Coordinates

In a robot manipulator consisting of N bodies, $6N$ coordinates are required to specify the position and orientation of all the bodies relative to a coordinate frame. Since some of those bodies are jointed together, a number of constraint equations will establish relationships between some of these coordinates. In this case, the $6N$ coordinates can be expressed as functions of a smaller set of coordinates $\mathbf{q}$ that are all independent. The coordinates in this set are known as generalized coordinates, and motions associated with these coordinates are consistent with all of the constraints. The joint variables $\mathbf{q}$ of a robot manipulator form a set of generalized coordinates [1.24, 25].

1.4 Geometric Representation

The geometry of a robotic manipulator is conveniently defined by attaching reference frames to each link. While these frames could be located arbitrarily, it is advantageous both for consistency and computational efficiency to adhere to a convention for locating the frames on the links. *Denavit* and *Hartenberg* [1.26] introduced the foundational convention that has been adapted in a number of different ways, one of which is the convention introduced by *Khalil* and *Dombre* [1.27] used throughout this handbook. In all of its forms, the convention requires only four rather than six parameters to locate one reference frame relative to another. The four parameters are the link length a_i , the link twist α_i , the joint offset d_i , and the joint angle θ_i . This parsimony is achieved through judicious placement of the reference

frame origins and axes such that the $\hat{\mathbf{x}}$ axis of one frame both intersects and is perpendicular to the $\hat{\mathbf{z}}$ axis of the preceding reference frame. The convention is applicable to manipulators consisting of revolute and prismatic joints, so when multiple-degree-of-freedom joints are present, they are modeled as combinations of revolute and prismatic joints, as discussed in Sect. 1.3.

There are essentially four different forms of the convention for locating reference frames in a robotic mechanism. Each exhibits its own advantages by managing trade-offs of intuitive presentation. In the original *Denavit* and *Hartenberg* [1.26] convention, joint i is located between links i and $i + 1$, so it is on the outboard side of link i . Also, the joint offset d_i and joint angle θ_i are measured along and about the $i - 1$ joint axis, so the

subscripts of the joint parameters do not match that of the joint axis. *Waldron* [1.28] and *Paul* [1.29] modified the labeling of axes in the original convention such that joint i is located between links $i - 1$ and i in order to make it consistent with the base member of a serial chain being member 0. This places joint i at the inboard side of link i and is the convention used in all of the other modified versions. Furthermore, Waldron and Paul addressed the mismatch between subscripts of the joint parameters and joint axes by placing the $\hat{z}_i$ axis along the $i + 1$ joint axis. This, of course, relocates the subscript mismatch to the correspondence between the joint axis and the $\hat{z}$ axis of the reference frame. *Craig* [1.30] eliminated all of the subscript mismatches by placing the $\hat{z}_i$ axis along joint i , but at the expense of the homogeneous transformation ${}^{i-1}T_i$ being formed with a mixture of joint parameters with subscript i and link parameters with subscript $i - 1$. *Khalil* and *Dombre* [1.27] introduced another variation similar to Craig's except that it defines the link parameters a_i and α_i along and about the $\hat{x}_{i-1}$ axis. In this case, the homogeneous transformation ${}^{i-1}T_i$ is formed only with parameters with subscript i , and the subscript mismatch is such that a_i and α_i indicate the length and twist of link $i - 1$ rather than link i . Thus, in summary, the advantages of the convention used throughout this handbook compared to the alternative conventions are

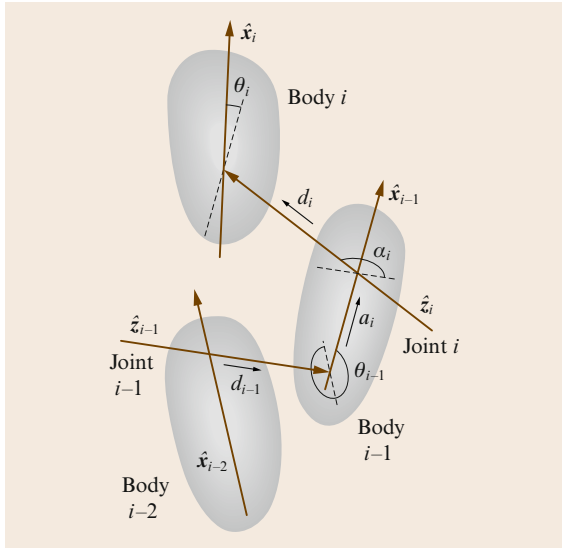


Fig. 1.2 Schematic of the numbering of bodies and joints in a robotic manipulator, the convention for attaching reference frames to the bodies, and the definitions of the four parameters, a_i , α_i , d_i , and θ_i , that locate one frame relative to another

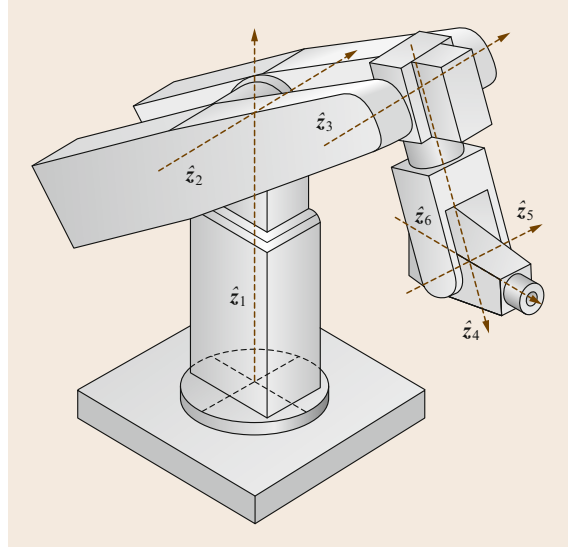


Fig. 1.3 Example six-degree-of-freedom serial chain manipulator composed of an articulated arm with no joint offsets and a spherical wrist

that the $\hat{z}$ axes of the reference frames share the common subscript of the joint axes, and the four parameters that define the spatial transform from reference frame i to reference frame $i - 1$ all share the common subscript i .

In this handbook, the convention for serial chain manipulators is shown in Fig. 1.2 and summarized as follows. The numbering of bodies and joints follows the convention:

- the N moving bodies of the robotic mechanism are numbered from 1 to N . The number of the base is 0.
- the N joints of the robotic mechanism are numbered from 1 to N , with joint i located between members $i - 1$ and i .

With this numbering scheme, the attachment of reference frames follows the convention:

- the $\hat{z}_i$ axis is located along the axis of joint i ,
- the $\hat{x}_{i-1}$ axis is located along the common normal between the $\hat{z}_{i-1}$ and $\hat{z}_i$ axes.

Using the attached frames, the four parameters that locate one frame relative to another are defined as

- a_i is the distance from $\hat{z}_{i-1}$ to $\hat{z}_i$ along $\hat{x}_{i-1}$,
- α_i is the angle from $\hat{z}_{i-1}$ to $\hat{z}_i$ about $\hat{x}_{i-1}$,
- d_i is the distance from $\hat{x}_{i-1}$ to $\hat{x}_i$ along $\hat{z}_i$,
- θ_i is the angle from $\hat{x}_{i-1}$ to $\hat{x}_i$ about $\hat{z}_i$.

Table 1.7 Geometric parameters of the example serial chain manipulator in Fig. 1.3

i	α_i	a_i	d_i	θ_i
1	0	0	0	θ_1
2	$-\frac{\pi}{2}$	0	0	θ_2
3	0	a_3	0	θ_3
4	$-\frac{\pi}{2}$	0	d_4	θ_4
5	$\frac{\pi}{2}$	0	0	θ_5
6	$-\frac{\pi}{2}$	0	0	θ_6

The geometric parameters for the example manipulator shown in Fig. 1.3 are listed in Table 1.7. All of the joints of this manipulator are revolute, and joint 1 has a vertical orientation. Joint 2 is perpendicular to joint 1 and intersects it. Joint 3 is parallel to joint 2, and the length of link 2 is a_3 . Joint 4 is perpendicular to joint 3 and intersects it. Joint 5 likewise intersects joint 4 perpendicularly at an offset of d_4 from joint 3. Finally, joint 6 intersects joint 5 perpendicularly.

1.5 Workspace

Most generally, the workspace of a robotic manipulator is the total volume swept out by the end-effector as the manipulator executes all possible motions. The workspace is determined by the geometry of the manipulator and the limits of the joint motions. It is more specific to define the reachable workspace as the total locus of points at which the end-effector can be placed and the dextrous workspace [1.31] as the subset of those points at which the end-effector can be placed while having an arbitrary orientation. Dexterous workspaces exist only for certain idealized geometries, so real industrial manipulators with joint motion limits almost never possess dextrous workspaces.

Many serial-chain robotic manipulators are designed such that their joints can be partitioned into a regional structure and an orientation structure. The joints in the regional structure accomplish the positioning of the end-effector in space, and the joints in the orientation structure accomplish the orientation of the end-effector. Typically, the inboard joints of a serial chain manipulator comprise the regional structure, while the outboard joints comprise the orientation structure. Also, since prismatic joints provide no capability for rotation, they are generally not employed within the orientation structure.

With this convention, reference frame i can be located relative to reference frame $i-1$ by executing a rotation through an angle α_i about the $\hat{x}_{i-1}$ axis, a translation of distance a_i along $\hat{x}_{i-1}$, a rotation through an angle θ_i about the $\hat{z}_i$ axis, and a translation of distance d_i along $\hat{z}_i$. Through concatenation of these individual transformations,

$$\text{Rot}(\hat{x}_{i-1}, \alpha_i) \text{Trans}(\hat{x}_{i-1}, a_i) \text{Rot}(\hat{z}_i, \theta_i)$$

$$\text{Trans}(\hat{z}_i, d_i),$$

the equivalent homogeneous transformation is,

$${}^{i-1}T_i = \begin{pmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_i \\ \sin \theta_i \cos \alpha_i & \cos \theta_i \cos \alpha_i & -\sin \alpha_i & -\sin \alpha_i d_i \\ \sin \theta_i \sin \alpha_i & \cos \theta_i \sin \alpha_i & \cos \alpha_i & \cos \alpha_i d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (1.42)$$

The identification of geometric parameters is addressed in Chap. 14.

The regional workspace volume can be calculated from the known geometry of the serial-chain manipulator and motion limits of the joints. With three inboard joints comprising the regional structure, the area of workspace for the outer two (joints 2 and 3) is computed first, and then the volume is calculated by integrating over the joint variable of the remaining inboard joint (joint 1). In the case of a prismatic joint, this simply involves multiplying the area by the total length of travel of the prismatic joint. In the more common case of a revolute joint, it involves rotating the area about the joint axis through the full range of motion of the revolute [1.32]. By the theorem of Pappus, the associated volume V is

$$V = A \bar{r} \gamma, \quad (1.43)$$

where A is the area, $\bar{r}$ is the distance from the area's centroid to the axis, and γ is the angle through which the area is rotated. The boundaries of the area are determined by tracing the motion of a reference point in the end-effector, typically the center of rotation of the wrist that serves as the orientation structure. Starting with each of the two joints at motion limits and with joint 2 locked, joint 3 is moved until its second motion limit is reached. Joint 3 is then locked, and joint 2 is freed to move to its second motion limit. Joint 2 is

again locked, while joint 3 is freed to move back to its original motion limit. Finally, joint 3 is locked, and joint 2 freed to move likewise to its original motion limit. In this way, the trace of the reference point is

1.6 Forward Kinematics

The forward kinematics problem for a serial-chain manipulator is to find the position and orientation of the end-effector relative to the base given the positions of all of the joints and the values of all of the geometric link parameters. Often, a frame fixed in the end-effector is referred to as the *tool frame*, and while fixed in the final link N , it in general has a constant offset in both position and orientation from frame N . Likewise, a *station frame* is often located in the base to establish the location of the task to be performed. This frame generally has a constant offset in its pose relative to frame 0, which is also fixed in the base.

A more general expression of the forward kinematics problem is to find the relative position and orientation of any two designated members given the geometric structure of the manipulator and the values of a number of joint positions equal to the number of degrees of freedom of the mechanism. The forward kinematics problem is critical for developing manipulator coordination algorithms because joint positions are typically measured by sensors mounted on the joints and it is necessary to calculate the positions of the joint axes relative to the fixed reference frame.

In practice, the forward kinematics problem is solved by calculating the transformation between a reference frame fixed in the end-effector and another reference frame fixed in the base, i.e., between the tool and station frames. This is straightforward for a serial chain since the transformation describing the position of the end-effector relative to the base is obtained by simply concatenating transformations between frames fixed in adjacent links of the chain. The convention for the geometric representation of a manipulator presented in Sect. 1.4 reduces this to finding an equivalent 4×4 homogeneous transformation matrix that relates the spatial displacement of the end-effector reference frame to the base reference frame.

For the example serial-chain manipulator shown in Fig. 1.3 and neglecting the addition of tool and station frames, the transformation is

$${}^0T_6 = {}^0T_1 {}^1T_2 {}^2T_3 {}^3T_4 {}^4T_5 {}^5T_6. \quad (1.44)$$

a closed curve whose area and centroid can be calculated mathematically.

More details on manipulator workspace can be found in Chaps. 3 and 10.

Table 1.8 contains the elements of 0T_6 that are calculated using Table 1.7 and (1.42).

Once again, homogeneous transformations provide a compact notation, but are computationally inefficient for solving the forward kinematics problem. A reduction in computation can be achieved by separating the

Table 1.8 Forward kinematics of the example serial chain manipulator in Fig. 1.3, with abbreviations $c_{\theta_i} := \cos \theta_i$ and $s_{\theta_i} := \sin \theta_i$

$${}^0T_6 = \begin{pmatrix} r_{11} & r_{12} & r_{13} & {}^0p_6^x \\ r_{21} & r_{22} & r_{23} & {}^0p_6^y \\ r_{31} & r_{32} & r_{33} & {}^0p_6^z \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

$$\begin{aligned} r_{11} &= c_{\theta_1}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3})(s_{\theta_4}s_{\theta_6} - c_{\theta_4}c_{\theta_5}c_{\theta_6}) \\ &\quad - c_{\theta_1}s_{\theta_5}c_{\theta_6}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}) \\ &\quad + s_{\theta_1}(s_{\theta_4}c_{\theta_5}c_{\theta_6} + c_{\theta_4}s_{\theta_6}), \\ r_{21} &= s_{\theta_1}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3})(s_{\theta_4}s_{\theta_6} - c_{\theta_4}c_{\theta_5}c_{\theta_6}) \\ &\quad - s_{\theta_1}s_{\theta_5}c_{\theta_6}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}) \\ &\quad - c_{\theta_1}(s_{\theta_4}c_{\theta_5}c_{\theta_6} + c_{\theta_4}s_{\theta_6}), \\ r_{31} &= (c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3})(s_{\theta_4}s_{\theta_6} - c_{\theta_4}c_{\theta_5}c_{\theta_6}) \\ &\quad + s_{\theta_5}c_{\theta_6}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3}), \\ r_{12} &= c_{\theta_1}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3})(c_{\theta_4}c_{\theta_5}s_{\theta_6} + s_{\theta_4}c_{\theta_6}) \\ &\quad + c_{\theta_1}s_{\theta_5}s_{\theta_6}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}) \\ &\quad + s_{\theta_1}(c_{\theta_4}c_{\theta_6} - s_{\theta_4}c_{\theta_5}s_{\theta_6}), \\ r_{22} &= s_{\theta_1}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3})(c_{\theta_4}c_{\theta_5}s_{\theta_6} + s_{\theta_4}c_{\theta_6}) \\ &\quad + s_{\theta_1}s_{\theta_5}s_{\theta_6}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}) \\ &\quad - c_{\theta_1}(c_{\theta_4}c_{\theta_6} - s_{\theta_4}c_{\theta_5}s_{\theta_6}), \\ r_{32} &= (c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3})(c_{\theta_4}c_{\theta_5}s_{\theta_6} + s_{\theta_4}c_{\theta_6}) \\ &\quad - s_{\theta_5}s_{\theta_6}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3}), \\ r_{13} &= c_{\theta_1}c_{\theta_4}s_{\theta_5}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3}) \\ &\quad - c_{\theta_1}c_{\theta_5}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}) \\ &\quad - s_{\theta_1}s_{\theta_4}s_{\theta_5}, \\ r_{23} &= s_{\theta_1}c_{\theta_4}s_{\theta_5}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3}) \\ &\quad - s_{\theta_1}c_{\theta_5}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}) + c_{\theta_1}s_{\theta_4}s_{\theta_5}, \\ r_{33} &= c_{\theta_4}s_{\theta_5}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}) \\ &\quad + c_{\theta_5}(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3}), \\ {}^0p_6^x &= a_3c_{\theta_1}c_{\theta_2} - d_4c_{\theta_1}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}), \\ {}^0p_6^y &= a_3s_{\theta_1}c_{\theta_2} - d_4s_{\theta_1}(c_{\theta_2}s_{\theta_3} + s_{\theta_2}c_{\theta_3}), \\ {}^0p_6^z &= -a_3s_{\theta_2} + d_4(s_{\theta_2}s_{\theta_3} - c_{\theta_2}c_{\theta_3}). \end{aligned}$$

position and orientation portions of the transformation to eliminate all multiplications by the 0 and 1 elements of the matrices. In Chap. 2, calculations are made using the spatial vector notation briefly introduced here in Sect. 1.2.6 and explained in detail in Sect. 2.2. This approach does not employ homogeneous transformations, but rather separates out the rotation matrices and positions to achieve computation efficiency. Table 2.1 provides the detailed formulas, with the product of

spatial transforms particularly relevant to the forward kinematics problem.

Kinematic trees are the general structure of robot mechanisms that do not contain closed loops, and the forward kinematics of tree structures are addressed in Chap. 2. The forward kinematics problem for closed chains is much more complicated because of the additional constraints present. Solution methods for closed chains are included in Chap. 12.

1.7 Inverse Kinematics

The inverse kinematics problem for a serial-chain manipulator is to find the values of the joint positions given the position and orientation of the end-effector relative to the base and the values of all of the geometric link parameters. Once again, this is a simplified statement applying only to serial chains. A more general statement is: given the relative positions and orientations of two members of a mechanism, find the values of all of the joint positions. This amounts to finding all of the joint positions given the homogeneous transformation between the two members of interest.

In the common case of a six-degree-of-freedom serial chain manipulator, the known transformation is 0T_6 . Reviewing the formulation of this transformation in Sect. 1.6, it is clear that the inverse kinematics problem for serial-chain manipulators requires the solution of nonlinear sets of equations. In the case of a six-degree-of-freedom manipulator, three of these equations relate to the position vector within the homogeneous transformation, and the other three relate to the rotation matrix. In the latter case, these three equations cannot come from the same row or column because of the dependency within the rotation matrix. With these nonlinear equations, it is possible that no solutions exist or multiple solutions exist [1.33]. For a solution to exist, the desired position and orientation of the end-effector must lie in the workspace of the manipulator. In cases where solutions do exist, they often cannot be presented in closed form, so numerical methods are required.

1.7.1 Closed-Form Solutions

Closed-form solutions are desirable because they are faster than numerical solutions and readily identify all possible solutions. The disadvantage of closed-form solutions is that they are not general, but robot dependent. The most effective methods for finding closed-form so-

lutions are ad hoc techniques that take advantage of particular geometric features of specific mechanisms. In general, closed-form solutions can only be obtained for six-degree-of-freedom systems with special kinematic structure characterized by a large number of the geometric parameters defined in Sect. 1.4 being zero-valued. Most industrial manipulators have such structure because it permits more efficient coordination software. Sufficient conditions for a six-degree-of-freedom manipulator to have closed-form inverse kinematics solutions are [1.34–36]:

1. three consecutive revolute joint axes intersect at a common point, as in a spherical wrist;
2. three consecutive revolute joint axes are parallel.

Closed-form solution approaches are generally divided into algebraic and geometric methods.

Algebraic Methods

Algebraic methods involve identifying the significant equations containing the joint variables and manipulating them into a soluble form. A common strategy is reduction to a transcendental equation in a single variable such as,

$$C_1 \cos \theta_i + C_2 \sin \theta_i + C_3 = 0, \quad (1.45)$$

where C_1 , C_2 , and C_3 are constants. The solution to such an equation is

$$\theta_i = 2 \tan^{-1} \left(\frac{C_2 \pm \sqrt{C_2^2 - C_3^2 + C_1^2}}{C_1 - C_3} \right). \quad (1.46)$$

Special cases in which one or more of the constants are zero are also common.

Reduction to a pair of equations having the form,

$$\begin{aligned} C_1 \cos \theta_i + C_2 \sin \theta_i + C_3 &= 0, \\ C_1 \sin \theta_i - C_2 \cos \theta_i + C_4 &= 0, \end{aligned} \quad (1.47)$$

is another particularly useful strategy because only one solution results,

$$\theta_i = \text{Atan2}(-C_1 C_4 - C_2 C_3, C_2 C_4 - C_1 C_3). \quad (1.48)$$

Geometric Methods

Geometric methods involve identifying points on the manipulator relative to which position and/or orientation can be expressed as a function of a reduced set of the joint variables. This often amounts to decomposing the spatial problem into separate planar problems. The resulting equations are solved using algebraic manipulation. The two sufficient conditions for existence of a closed-form solution for a six-degree-of-freedom manipulator that are listed above enable the decomposition of the problem into inverse position kinematics and inverse orientation kinematics. This is the decomposition into regional and orientation structures discussed in Sect. 1.5, and the solution is found by rewriting (1.44),

$${}^0T_6 {}^6T_5 {}^5T_4 {}^4T_3 = {}^0T_1 {}^1T_2 {}^2T_3. \quad (1.49)$$

The example manipulator in Fig. 1.3 has this structure, and this regional structure is commonly known as an articulated or anthropomorphic arm or an elbow manipulator. The solution to the inverse position kinematics problem for such a structure is summarized in Table 1.9. Because there are two solutions for θ_1 and likewise two solutions for both θ_2 and θ_3 corresponding to each θ_1 solution, there are a total of four solutions to the inverse position kinematics problem of the articulated arm manipulator. The orientation structure is simply a spherical wrist, and the corresponding solution to the inverse orientation kinematics problem is summarized in Table 1.10. Two solutions for θ_5 are given in Table 1.10, but only one solution for both θ_4 and θ_6 corresponds to each. Thus, the inverse orientation kinematics problem of a spherical wrist has two solutions. Combining the regional

Table 1.9 Inverse position kinematics of the articulated arm within the example serial chain manipulator in Fig. 1.3

$$\begin{aligned} \theta_1 &= \text{Atan2}({}^0p_6^y, {}^0p_6^x) \\ &\text{or } \text{Atan2}(-{}^0p_6^y, -{}^0p_6^x), \\ \theta_3 &= -\text{Atan2}(D, \pm\sqrt{1-D^2}), \\ \text{where } D &:= \frac{({}^0p_6^x)^2 + ({}^0p_6^y)^2 + ({}^0p_6^z)^2 - a_3^2 - d_4^2}{2a_3d_4}, \\ \theta_2 &= \text{Atan2}({}^0p_6^z, \sqrt{({}^0p_6^x)^2 + ({}^0p_6^y)^2}) \\ &\quad - \text{Atan2}(d_4 \sin \theta_3, a_3 + d_4 \cos \theta_3) \end{aligned}$$

Table 1.10 Inverse orientation kinematics of the spherical wrist within the example serial chain manipulator in Fig. 1.3, with abbreviations $c_{\theta_i} := \cos \theta_i$ and $s_{\theta_i} := \sin \theta_i$

$$\begin{aligned} \theta_5 &= \text{Atan2}(\pm\sqrt{1-(r_{13}s_{\theta_1}-r_{23}c_{\theta_1})^2}, r_{13}s_{\theta_1}-r_{23}c_{\theta_1}), \\ \theta_4 &= \text{Atan2}(\mp(r_{13}c_{\theta_1}+r_{23}s_{\theta_1})s_{(\theta_2+\theta_3)} \mp r_{33}c_{(\theta_2+\theta_3)}, \\ &\quad \pm(r_{13}s_{\theta_1}+r_{23}c_{\theta_1})c_{(\theta_2+\theta_3)} \mp r_{23}s_{(\theta_2+\theta_3)}), \\ \theta_6 &= \text{Atan2}(\pm(r_{12}s_{\theta_1}+r_{22}c_{\theta_1}), \pm(r_{11}s_{\theta_1}-r_{21}c_{\theta_1})), \\ &\text{where the } \pm \text{ choice for } \theta_5 \text{ dictates all of the subsequent } \pm \\ &\text{and } \mp \text{ for } \theta_4 \text{ and } \theta_6. \end{aligned}$$

and orientation structures, the total number of inverse kinematics solutions for the manipulator in Fig. 1.3 is eight.

1.7.2 Numerical Methods

Unlike the algebraic and geometric methods used to find closed-form solutions, numerical methods are not robot dependent, so they can be applied to any kinematic structure. The disadvantages of numerical methods are that they can be slower and in some cases, they do not allow computation of all possible solutions. For a six-degree-of-freedom serial-chain manipulator with only revolute and prismatic joints, the translation and rotation equations can always be reduced to a polynomial in a single variable of degree not greater than 16 [1.37]. Thus, such a manipulator can have as many as 16 real solutions to the inverse kinematics problem [1.38]. Since closed-form solution of a polynomial equation is only possible if the polynomial is of degree four or less, it follows that many manipulator geometries are not solvable in closed form. In general, a greater number of nonzero geometric parameters corresponds to a polynomial of higher degree in the reduction. For such manipulator structures, the most common numerical methods can be divided into categories of symbolic elimination methods, continuation methods, and iterative methods.

Symbolic Elimination Methods

Symbolic elimination methods involve analytical manipulations to eliminate variables from the system of nonlinear equations to reduce it to a smaller set of equations. *Raghavan* and *Roth* [1.39] used dialytic elimination to reduce the inverse kinematics problem of a general six-revolute serial-chain manipulator to a polynomial of degree 16 and to find all possible solutions. The roots provide solutions for one of the joint variables, while the other variables are computed by solving lin-

ear systems. *Manocha* and *Canny* [1.40] improved the numerical properties of this technique by reformulating the problem as a generalized eigenvalue problem. An alternative approach to elimination makes use of Gröbner bases [1.41, 42].

Continuation Methods

Continuation methods involve tracking a solution path from a start system with known solutions to a target system whose solutions are sought as the start system is transformed into the target system. These techniques have been applied to inverse kinematics problems [1.43], and special properties of polynomial systems can be exploited to find all possible solutions [1.44].

Iterative Methods

A number of different iterative methods can be employed to solve the inverse kinematics problem. Most of them converge to a single solution based on an initial guess, so the quality of that guess greatly impacts

the solution time. Newton–Raphson methods provide a fundamental approach that uses a first-order approximation to the original equations. *Pieper* [1.34] was among the first to apply the method to inverse kinematics, and others have followed [1.45, 46]. Optimization approaches formulate the problem as a nonlinear optimization problem and employ search techniques to move from an initial guess to a solution [1.47, 48]. Resolved motion rate control converts the problem to a differential equation [1.49], and a modified predictor–corrector algorithm can be used to perform the joint velocity integration [1.50]. Control-theory-based methods cast the differential equation into a control problem [1.51]. Interval analysis [1.52] is perhaps one of the most promising iterative methods because it offers rapid convergence to a solution and can be used to find all possible solutions. For complex mechanisms, the damped least-squares approach [1.53] is particularly attractive, and more detail is provided in Chap. 11.

1.8 Forward Instantaneous Kinematics

The forward instantaneous kinematics problem for a serial-chain manipulator is: given the positions of all members of the chain and the rates of motion about all the joints, find the total velocity of the end-effector. Here the rate of motion about the joint is the angular velocity of rotation about a revolute joint or the translational velocity of sliding along a prismatic joint. The total velocity of a member is the velocity of the origin of the reference frame fixed to it combined with its angular velocity. That is, the total velocity has six independent components and therefore, completely represents the velocity field of the member. It is important to note that this definition includes an assumption that the pose of the mechanism is completely known. In most situations, this means that either the forward or inverse position kinematics problem must be solved before the forward instantaneous kinematics problem can be addressed. The same is true of the inverse instantaneous kinematics problem discussed in the following section. The forward instantaneous kinematics problem is important when doing acceleration analysis for the purpose of studying dynamics. The total velocities of the members are needed for the computation of Coriolis and centripetal acceleration components.

1.8.1 Jacobian

Differentiation with respect to time of the forward position kinematics equations yields a set of equations of the form,

$$\mathbf{v}_N = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}, \quad (1.50)$$

where $\mathbf{v}_N$ is the spatial velocity of the end-effector, $\dot{\mathbf{q}}$ is an N -dimensional vector composed of the joint rates, and $\mathbf{J}(\mathbf{q})$ is a $6 \times N$ matrix whose elements are, in gen-

Table 1.11 Algorithm for computing the columns of the Jacobian from the free modes of the joints

n_i	Number of degrees of freedom of joint i
$\mathbf{J}$	${}^k\mathbf{v}_N = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$, where k is any frame
$\mathbf{J}_{n_i}$	n_i column(s) of $\mathbf{J}$ associated with $\dot{\mathbf{q}}_i$,
Φ_ω	first three rows of Φ ,
Φ_v	last three rows of Φ ,
	$\mathbf{J}_{n_i} = {}^k\mathbf{X}_i\Phi_i$,
Expression	Computed value
$\mathbf{X}_1\mathbf{X}_2$	$(\mathbf{R}_1\mathbf{R}_2; \mathbf{p}_2 + \mathbf{R}_2^T\mathbf{p}_1)$
$\mathbf{X}\Phi$	$(\mathbf{R}\Phi_\omega; \mathbf{R}(\Phi_v - \mathbf{p} \times \Phi_\omega))$
$\mathbf{X}^{-1}$	$(\mathbf{R}^T; -\mathbf{R}\mathbf{p})$
$\mathbf{X}^{-1}\Phi$	$(\mathbf{R}^T\Phi_\omega; \mathbf{R}^T\Phi_v + \mathbf{p} \times \mathbf{R}^T\Phi_\omega)$

eral, nonlinear functions of $q_1, \dots, q_N$. $J(q)$ is called the Jacobian matrix of this algebraic system and is expressed relative to the same reference frame as the spatial velocity v_N [1.54]. If the joint positions are known, (1.50) yields six linear algebraic equations in the joint rates. If the joint rates are given, a solution of (1.50) is a solution of the forward instantaneous kinematics problem. Note that $J(q)$ can be regarded as a known matrix for this purpose provided all the joint positions are known.

Using the spatial vector notation briefly introduced in Sect. 1.2.6 and explained in detail in Sect. 2.2, the

Jacobian can be easily computed from the free modes Φ_i of the joints and the associated spatial transforms jX_i . The column(s) of $J(q)$ associated with joint rate(s) $\dot{q}_i$ is (are)

$${}^kX_i\Phi_i,$$

where k denotes any reference frame relative to which v_N is expressed. Table 1.11 contains an algorithm for efficiently computing the columns of the Jacobian in this manner. Additional information about Jacobians can be found in Chap. 11.

1.9 Inverse Instantaneous Kinematics

The important problem from the point of view of robotic coordination is the inverse instantaneous kinematics problem. More information on robot coordination can be found in Chaps. 5 and 6. The inverse instantaneous kinematics problem for a serial chain manipulator is: given the positions of all members of the chain and the total velocity of the end-effector, find the rates of motion of all joints. When controlling a movement of an industrial robot which operates in the point-to-point mode, it is not only necessary to compute the final joint positions needed to assume the desired final hand position. It is also necessary to generate a smooth trajectory for motion between the initial and final positions. There are, of course, an infinite number of possible trajectories for this purpose. However, the most straightforward and successful approach employs algorithms based on the solution of the inverse instantaneous kinematics problem. This technique originated in the work of Whitney [1.55] and of Pieper [1.34].

1.9.1 Inverse Jacobian

In order to solve the linear system of equations in the joint rates obtained by decomposing (1.50) into its com-

ponent equations when v_N is known, it is necessary to invert the Jacobian matrix. The equation becomes

$$\dot{q} = J^{-1}(q)v_N. \quad (1.51)$$

Since J is a 6×6 matrix, numerical inversion is not very attractive in real-time software which must run at computation cycle rates on the order of 100 Hz or more. Worse, it is quite possible for J to become singular ($|J| = 0$). The inverse does not then exist. More information on singularities can be found in Chaps. 3 and 12. Even when the Jacobian matrix does not become singular, it may become ill-conditioned, leading to degraded performance in significant portions of the manipulator's workspace. Most industrial robot geometries are simple enough that the Jacobian matrix can be inverted analytically, leading to a set of explicit equations for the joint rates [1.56–58]. This greatly reduces the number of computation operations needed as compared to numerical inversion. For more complex manipulator geometries, though, numerical inversion is the only solution option. The Jacobian of a redundant manipulator is not square, so it cannot be inverted. Chapter 11 discusses how various pseudoinverses can be used in such cases.

1.10 Static Wrench Transmission

Static wrench analysis of a manipulator establishes the relationship between wrenches applied to the end-effector and forces/torques applied to the joints. This is essential for controlling a manipulator's interactions with its environment. Examples include tasks involving fixed or quasi-fixed workpieces such as inserting a component in place with a specified force and tightening a nut to a prescribed torque. More information can be

found in Chaps. 7 and 27. Through the principle of virtual work, the relationship between wrenches applied to the end-effector and forces/torques applied to the joints can be shown to be

$$\tau = J^T f, \quad (1.52)$$

where τ is the n -dimensional vector of applied joint forces/torques for an n -degree-of-freedom manipulator

and $\mathbf{f}$ is the spatial force vector

$$\mathbf{f} = \begin{pmatrix} \mathbf{n} \\ \mathbf{f} \end{pmatrix} \quad (1.53)$$

in which $\mathbf{n}$ and $\mathbf{f}$ are the vectors of torques and forces, respectively, applied to the end-effector, both expressed in the reference frame relative to which

the Jacobian is also expressed. Thus, in the same way the Jacobian maps the joint rates to the spatial velocity of the end-effector, its transpose maps the wrenches applied to the end-effector to the equivalent joint forces/torques. As in the velocity case, when the Jacobian is not square, the inverse relationship is not uniquely defined.

1.11 Conclusions and Further Reading

This chapter presents an overview of how the fundamentals of kinematics can be applied to robotic mechanisms. The topics include various representations of the position and orientation of a rigid body in space, the freedom of motion and accompanying mathematical models of joints, a geometric representation that describes the bodies and joints of a robotics mechanism, the workspace of a manipulator, the problems of forward and inverse kinematics, the problems of forward and inverse instantaneous kinematics including the definition of the Jacobian, and finally the transmission of static wrenches. This chapter is certainly not a comprehensive account of robot kinematics. Fortunately, a number of excellent texts provide a broad introduction to robotics with significant focus on kinematics [1.17,27,29,30,51,59–63].

From a historical perspective, robotics fundamentally changed the nature of the field of mechanism kinematics. Before the first work on the generation of coordination equations for robots [1.34,55], the focus of the field was almost entirely on single-degree-of-freedom mechanisms. This is why robotics, following on from the advent of digital computing, led to a renaissance of work in mechanism kinematics. More details can be found in Chap. 3. The evolution of the field has continued as it has broadened from the study of simple serial chains for industrial robots, the focus of the analysis in this chapter, to parallel machines (see Chap. 12), human-like grippers (Chap. 15), robotic vehicles (Chap. 16 and Chap. 17), and even small-scale robots (see Chap. 18).

References

- 1.1 W. R. Hamilton: On quaternions, or on a new system of imaginaries in algebra, *Philos. Mag.* **18**, installments July 1844 – April 1850, ed. by D. E. Wilkins (2000)
- 1.2 E.B. Wilson: *Vector Analysis* (Dover, New York 1960), based upon the lectures of J. W. Gibbs, (reprint of the second edn. published by Charles Scribner's Sons, 1909)
- 1.3 H. Grassman: *Die Wissenschaft der extensiven Grösse oder die Ausdehnungslehre* (Wigand, Leipzig 1844)
- 1.4 J.M. McCarthy: *Introduction to Theoretical Kinematics* (MIT Press, Cambridge 1990)
- 1.5 W.K. Clifford: Preliminary sketch of bi-quaternions, *Proc. London Math. Soc.*, Vol. 4 (1873) pp. 381–395
- 1.6 A. P. Kotelnikov: Screw calculus and some applications to geometry and mechanics, *Annal. Imp. Univ. Kazan* (1895)
- 1.7 E. Study: *Geometrie der Dynamen* (Teubner, Leipzig 1901)
- 1.8 G.S. Chirikjian, A.B. Kyatkin: *Engineering Applications of Noncommutative Harmonic Analysis* (CRC, Boca Raton 2001)
- 1.9 R. von Mises: Anwendungen der Motorrechnung, *Z. Angew. Math. Mech.* **4**(3), 193–213 (1924)
- 1.10 J.E. Baker, I.A. Parkin: *Fundamentals of Screw Motion: Seminal Papers by Michel Chasles and Olinde Rodrigues* (School of Information Technologies, The University of Sydney, Sydney 2003), translated from O. Rodrigues: Des lois géométriques qui régissent les déplacements d'un système dans l'espace, *J. Math. Pures Applicqu. Liouville* **5**, 380–440 (1840)
- 1.11 R.S. Ball: *A Treatise on the Theory of Screws* (Cambridge Univ Press, Cambridge 1998)
- 1.12 J.K. Davidson, K.H. Hunt: *Robots and Screw Theory: Applications of Kinematics and Statics to Robotics* (Oxford Univ Press, Oxford 2004)
- 1.13 K.H. Hunt: *Kinematic Geometry of Mechanisms* (Clarendon, Oxford 1978)
- 1.14 J.R. Phillips: *Freedom in Machinery: Volume 1. Introducing Screw Theory* (Cambridge Univ Press, Cambridge 1984)
- 1.15 J.R. Phillips: *Freedom in Machinery: Volume 2. Screw Theory Exemplified* (Cambridge Univ Press, Cambridge 1990)

- 1.16 G.S. Chirikjian: Rigid-body kinematics. In: *Robotics and Automation Handbook*, ed. by T. Kurfess (CRC, Boca Raton 2005), Chapt. 2
- 1.17 R.M. Murray, Z. Li, S.S. Sastry: *A Mathematical Introduction to Robotic Manipulation* (CRC, Boca Raton 1994)
- 1.18 A. Karger, J. Novak: *Space Kinematics and Lie Groups* (Routledge, New York 1985)
- 1.19 R. von Mises: Motorrechnung, ein neues Hilfsmittel in der Mechanik, *Z. Angew. Math. Mech.* **2**(2), 155–181 (1924), [transl. J. E. Baker, K. Wohlhart, Inst. for Mechanics, T. U. Graz (1996)]
- 1.20 J.D. Everett: On a new method in statics and kinematics, *Mess. Math.* **45**, 36–37 (1875)
- 1.21 R. Featherstone: *Rigid Body Dynamics Algorithms* (Kluwer Academic, Boston 2007)
- 1.22 F. Reuleaux: *Kinematics of Machinery* (Dover, New York 1963), (reprint of *Theoretische Kinematik*, 1875, in German).
- 1.23 K.J. Waldron: A method of studying joint geometry, *Mechan. Machine Theory* **7**, 347–353 (1972)
- 1.24 T.R. Kane, D.A. Levinson: *Dynamics, Theory and Applications* (McGraw-Hill, New York 1985)
- 1.25 J.L. Lagrange: *Oeuvres de Lagrange* (Gauthier-Villars, Paris 1773)
- 1.26 J. Denavit, R.S. Hartenberg: A kinematic notation for lower-pair mechanisms based on matrices, *J. Appl. Mech.* **22**, 215–221 (1955)
- 1.27 W. Khalil, E. Dombre: *Modeling, Identification and Control of Robots* (Taylor Francis, New York 2002)
- 1.28 K.J. Waldron: A study of overconstrained linkage geometry by solution of closure equations, Part I: a method of study, *Mech. Machine Theory* **8**(1), 95–104 (1973)
- 1.29 R. Paul: *Robot Manipulators: Mathematics, Programming and Control* (MIT Press, Cambridge 1982)
- 1.30 J.J. Craig: *Introduction to Robotics: Mechanics and Control* (Addison-Wesley, Reading 1986)
- 1.31 K.J. Waldron, A. Kumar: The Dextrous workspace, *ASME Mech. Conf.* (Los Angeles 1980), ASME paper No. 80-DETC-108
- 1.32 R. Vijaykumar, K.J. Waldron, M.J. Tsai: Geometric optimization of manipulator structures for working volume and dexterity, *Int. J. Robot. Res.* **5**(2), 91–103 (1986)
- 1.33 J. Duffy: *Analysis of Mechanisms and Robot Manipulators* (Wiley, New York 1980)
- 1.34 D. Pieper: The Kinematics of Manipulators Under Computer Control. Ph.D. Thesis (Stanford University, Stanford 1968)
- 1.35 C.S.G. Lee: Robot arm kinematics, dynamics, and control, *Computer* **15**(12), 62–80 (1982)
- 1.36 M.T. Mason: *Mechanics of Robotic Manipulation* (MIT Press, Cambridge 2001)
- 1.37 H.Y. Lee, C.G. Liang: A new vector theory for the analysis of spatial mechanisms, *Mechan. Machine Theory* **23**(3), 209–217 (1988)
- 1.38 R. Manseur, K.L. Doty: A robot manipulator with 16 real inverse kinematic solutions, *Int. J. Robot. Res.* **8**(5), 75–79 (1989)
- 1.39 M. Raghavan, B. Roth: Kinematic analysis of the 6R manipulator of general geometry, 5th Int. Symp. Robot. Res. (1990)
- 1.40 D. Manocha, J. Canny: *Real Time Inverse Kinematics for General 6R Manipulators* Tech. rep. (University of California, Berkeley 1992)
- 1.41 B. Buchberger: Applications of Gröbner bases in non-linear computational geometry. In: *Trends in Computer Algebra*, Lect. Notes Comput. Sci., Vol. 296, ed. by R. Janen (Springer, Berlin 1989) pp. 52–80
- 1.42 P. Kovacs: Minimum degree solutions for the inverse kinematics problem by application of the Buchberger algorithm. In: *Advances in Robot Kinematics*, ed. by S. Stifter, J. Lenarcic (Springer, New York 1991) pp. 326–334
- 1.43 L.W. Tsai, A.P. Morgan: Solving the kinematics of the most general six- and five-degree-of-freedom manipulators by continuation methods, *ASME J. Mechan. Transmission Autom. Design* **107**, 189–195 (1985)
- 1.44 C.W. Wampler, A.P. Morgan, A.J. Sommese: Numerical continuation methods for solving polynomial systems arising in kinematics, *ASME J. Mech. Des.* **112**, 59–68 (1990)
- 1.45 R. Manseur, K.L. Doty: Fast inverse kinematics of 5-revolute-axis robot manipulators, *Mechan. Machine Theory* **27**(5), 587–597 (1992)
- 1.46 S.C.A. Thomopoulos, R.Y.J. Tam: An iterative solution to the inverse kinematics of robotic manipulators, *Mechan. Machine Theory* **26**(4), 359–373 (1991)
- 1.47 J.J. Uicker Jr., J. Denavit, R.S. Hartenberg: An interactive method for the displacement analysis of spatial mechanisms, *J. Appl. Mech.* **31**, 309–314 (1964)
- 1.48 J. Zhao, N. Badler: Inverse kinematics positioning using nonlinear programming for highly articulated figures, *Trans. Comput. Graph.* **13**(4), 313–336 (1994)
- 1.49 D.E. Whitney: Resolved motion rate control of manipulators and human prostheses, *IEEE Trans. Man Mach. Syst.* **10**, 47–63 (1969)
- 1.50 H. Cheng, K. Gupta: A study of robot inverse kinematics based upon the solution of differential equations, *J. Robot. Syst.* **8**(2), 115–175 (1991)
- 1.51 L. Sciavicco, B. Siciliano: *Modeling and Control of Robot Manipulators* (Springer, London 2000)
- 1.52 R.S. Rao, A. Asaithambi, S.K. Agrawal: Inverse Kinematic Solution of Robot Manipulators Using Interval Analysis, *ASME J. Mech. Des.* **120**(1), 147–150 (1998)
- 1.53 C.W. Wampler: Manipulator inverse kinematic solutions based on vector formulations and damped least squares methods, *IEEE Trans. Syst. Man Cybern.* **16**, 93–101 (1986)
- 1.54 D.E. Orin, W.W. Schrader: Efficient computation of the jacobian for robot manipulators, *Int. J. Robot. Res.* **3**(4), 66–75 (1984)

- 1.55 D. E. Whitney: The mathematics of coordinated control of prosthetic arms and manipulators J. Dynamic Sys. Meas. Contr. **122**, 303–309 (1972)
- 1.56 R.P. Paul, B.E. Shimano, G. Mayer: Kinematic control equations for simple manipulators, IEEE Trans. Syst. Man Cybern. **SMC-11**(6), 339–455 (1981)
- 1.57 R.P. Paul, C.N. Stephenson: Kinematics of robot wrists, Int. J. Robot. Res. **20**(1), 31–38 (1983)
- 1.58 R.P. Paul, H. Zhang: Computationally efficient kinematics for manipulators with spherical wrists based on the homogeneous transformation representation, Int. J. Robot. Res. **5**(2), 32–44 (1986)
- 1.59 H. Asada, J.J.E. Slotine: *Robot Analysis and Control* (Wiley, New York 1986)
- 1.60 F.L. Lewis, C.T. Abdallah, D.M. Dawson: *Control of Robot Manipulators* (Macmillan, New York 1993)
- 1.61 R.J. Schilling: *Fundamentals of Robotics: Analysis and Control* (Prentice-Hall, Englewood Cliffs 1990)
- 1.62 M.W. Spong, M. Vidyasagar: *Robot Dynamics and Control* (Wiley, New York 1989)
- 1.63 T. Yoshikawa: *Foundations of Robotics* (MIT Press, Cambridge 1990)

Dynamics

2. Dynamics

Roy Featherstone, David E. Orin

The dynamic equations of motion provide the relationships between actuation and contact forces acting on robot mechanisms, and the acceleration and motion trajectories that result. Dynamics is important for mechanical design, control, and simulation. A number of algorithms are important in these applications, and include computation of the following: *inverse dynamics*, *forward dynamics*, the *joint-space inertia matrix*, and the *operational-space inertia matrix*. This chapter provides efficient algorithms to perform each of these calculations on a rigid-body model of a robot mechanism. The algorithms are presented in their most general form and are applicable to robot mechanisms with general connectivity, geometry, and joint types. Such mechanisms include fixed-base robots, mobile robots, and parallel robot mechanisms.

In addition to the need for computational efficiency, algorithms should be formulated with a compact set of equations for ease of development and implementation. The use of spatial notation has been very effective in this regard, and is used in presenting the dynamics algorithms. Spatial vector algebra is a concise vector notation for describing rigid-body velocity, acceleration, inertia, etc., using six-dimensional (6-D) vectors and tensors.

The goal of this chapter is to introduce the reader to the subject of robot dynamics and to provide the reader with a rich set of algorithms, in a compact form, that they may apply to their particular robot mechanism. These algorithms are presented in tables for ready access.

2.1	Overview	36
2.1.1	Spatial Vector Notation	36
2.1.2	Canonical Equations	36
2.1.3	Dynamic Models of Rigid-Body Systems	36
2.1.4	Kinematic Trees	37
2.1.5	Kinematic Loops	37
2.2	Spatial Vector Notation	37
2.2.1	Motion and Force	38
2.2.2	Basis Vectors	38
2.2.3	Spatial Velocity and Force	38
2.2.4	Addition and Scalar Multiplication	39
2.2.5	Scalar Product	39
2.2.6	Coordinate Transforms	39
2.2.7	Vector Products	39
2.2.8	Differentiation	40
2.2.9	Acceleration	40
2.2.10	Spatial Momentum	40
2.2.11	Spatial Inertia	41
2.2.12	Equation of Motion	41
2.2.13	Computer Implementation	42
2.2.14	Summary	43
2.3	Canonical Equations	43
2.3.1	Joint-Space Formulation	44
2.3.2	Lagrange Formulation	44
2.3.3	Operational-Space Formulation	44
2.3.4	Impact Model	45
2.4	Dynamic Models of Rigid-Body Systems ..	45
2.4.1	Connectivity	45
2.4.2	Link Geometry	47
2.4.3	Link Inertias	48
2.4.4	Joint Models	48
2.4.5	Example System	48
2.5	Kinematic Trees	50
2.5.1	The Recursive Newton-Euler Algorithm	50
2.5.2	The Articulated-Body Algorithm	53
2.5.3	The Composite-Rigid-Body Algorithm	54
2.5.4	Operational-Space Inertia Matrix ..	55
2.6	Kinematic Loops	57
2.6.1	Formulation of Closed-Loop Algorithm	57
2.6.2	Closed-Loop Algorithm	58
2.7	Conclusions and Further Reading	60
2.7.1	Multibody Dynamics	61
2.7.2	Alternative Representations	61
2.7.3	Alternative Formulations	61

2.7.4 Efficiency	61	2.7.8 Algorithms for Parallel Computers ..	62
2.7.5 Accuracy.....	61	2.7.9 Topologically-Varying Systems	62
2.7.6 Software Packages.....	62		
2.7.7 Symbolic Simplification.....	62	References	62

2.1 Overview

Robot dynamics provides the relationships between actuation and contact forces, and the acceleration and motion trajectories that result. The dynamic equations of motion provide the basis for a number of computational algorithms that are useful in mechanical design, control, and simulation. A growing area of their use is in computer animation of mobile systems, especially using human and humanoid models. In this Chapter, the fundamental dynamic relationships for robot mechanisms are presented, along with efficient algorithms for the most common computations. Spatial vector notation, a concise representation which makes use of 6-D vectors and tensors, is used in the algorithms.

This chapter presents efficient low-order algorithms for four major computations:

1. inverse dynamics, in which the required joint actuator torques/forces are computed from a specification of the robot's trajectory (position, velocity, and acceleration),
2. forward dynamics in which the applied joint actuator torques/forces are specified and the joint accelerations are to be determined,
3. the joint-space inertia matrix, which maps the joint accelerations to the joint torques/forces, and
4. the operational-space inertia matrix, which maps task accelerations to task forces in operational or Cartesian space.

Inverse dynamics is used in feedforward control. Forward dynamics is required for simulation. The joint-space inertia (mass) matrix is used in analysis, in feedback control to linearize the dynamics, and is an integral part of many forward dynamics formulations. The operational-space inertia matrix is used in control at the task or end-effector level.

2.1.1 Spatial Vector Notation

Section 2.2 presents the spatial vector notation, which is used to express the algorithms in this chapter in a clear and concise manner. It was originally developed by *Featherstone* [2.1] to provide a concise vector no-

tation for describing rigid-body velocity, acceleration, inertia, etc., using 6-D vectors and tensors. Section 2.2 explains the meanings of spatial vectors and operators, and provides a detailed tabulation of the correspondence between spatial and standard three-dimensional (3-D) quantities and operators, so that the algorithms in the later sections can be understood. Formulae for efficient computer implementation of spatial arithmetic are also provided. Effort is taken in the discussion of spatial vectors to distinguish between the coordinate vectors and the quantities they represent. This illuminates some of the important characteristics of spatial vectors.

2.1.2 Canonical Equations

The dynamic equations of motion are provided in Sect. 2.3 in two fundamental forms: the joint-space formulation and the operational-space formulation. The terms in the joint-space formulation have traditionally been derived using a Lagrangian approach in which they are developed independently of any reference coordinate frame. The Lagrange formulation provides a description of the relationship between the joint actuator forces and the motion of the mechanism, and fundamentally operates on the kinetic and potential energy in the system. The resulting joint-space formulation has a number of notable properties that have proven useful for developing control algorithms. The equations to relate the terms in the joint-space and operational-space formulations, along with an impact model, are also provided in this section.

2.1.3 Dynamic Models of Rigid-Body Systems

The algorithms in this chapter are model-based and require a data structure describing a robot mechanism as one of their input arguments. Section 2.4 gives a description of the components of this model: a connectivity graph, link geometry parameters, link inertia parameters, and a set of joint models. The description of the connectivity is general so that it covers both kinematic

trees and closed-loop mechanisms. Kinematic trees and the spanning tree for a closed-loop mechanism share a common notation. In order to describe the link and joint geometry, two coordinate frames are associated with each joint, one each attached to the *predecessor* and *successor* links. The successor frame is defined to be compatible with the modified Denavit–Hartenberg convention of Craig [2.2] for single-degree-of-freedom (DOF) joints. The predecessor frame may be defined in a convenient manner to describe a general multi-DOF joint. The relationship between connected links is described using the general joint model of Roberson and Schwertassek [2.3]. A humanoid robot is given as an example to illustrate the link and joint numbering scheme, as well as the assignment of coordinate frames to describe the links and joints. The example includes a floating base, and revolute, universal, and spherical joints.

2.1.4 Kinematic Trees

The algorithms presented in Sect. 2.5 calculate the inverse dynamics, forward dynamics, joint-space inertia matrix, and operational-space inertia matrix for any robot mechanism that is a kinematic tree. An $O(n)$ algorithm for inverse dynamics is provided, where n is the number of degrees of freedom in the mechanism. It uses a Newton–Euler formulation of the problem, and is based on the very efficient recursive Newton–Euler algorithm (RNEA) of Luh, Walker, and Paul [2.4]. Two algorithms are provided for forward dynamics. The first is the $O(n)$ articulated-body algorithm (ABA) which was developed by Featherstone [2.1]. The second is the $O(n^2)$ composite-rigid-body algorithm (CRBA), developed by Walker and Orin [2.5], to compute the joint-space inertia matrix (JSIM). This matrix, together with a vector computed using the RNEA, provide the coefficients of the equation of motion, which can then be solved directly for the accelerations [2.5]. The

operational-space inertia matrix (OSIM) is a kind of articulated-body inertia, and two algorithms are given to calculate it. The first uses the basic definition of the OSIM, and the second is a straightforward $O(n)$ algorithm which is based on efficient solution of the forward dynamics problem. The inputs, outputs, model data, and pseudocode for each algorithm are summarized in tables for ready access.

2.1.5 Kinematic Loops

The above algorithms apply only to mechanisms having the connectivity of kinematic trees, including unbranched kinematic chains. A final algorithm is provided in Sect. 2.6 for the forward dynamics of closed-loop systems, including parallel robot mechanisms. The algorithm makes use of the dynamic equations of motion for a spanning tree of the closed-loop system, and supplements these with loop-closure constraint equations. Three different methods are outlined to solve the resulting linear system of equations. Method 2 is particularly useful if $n \gg n^c$, where n^c is the number of constraints due to the loop-closing joints. This method offers the opportunity to use $O(n)$ algorithms on the spanning tree [2.6]. The section ends with an efficient algorithm to compute the loop-closure constraints by transforming them to a single coordinate system. Since the loop-closure constraint equations are applied at the acceleration level, standard *Baumgarte* stabilization [2.7] is used to prevent the accumulation of position and velocity errors in the loop-closure constraints.

The final section in this chapter provides a conclusion and suggestions for further reading. The area of robot dynamics has been, and continues to be, a very rich area of investigation. This section outlines the major contributions that have been made in the area and the work most often cited. Unfortunately, space does not permit us to provide a comprehensive review of the extensive literature in the area.

2.2 Spatial Vector Notation

There is no single standard notation for robot dynamics. The notations currently in use include 3-D vectors, 4×4 matrices, and several types of 6-D vector: screws, motors, Lie algebra elements, and spatial vectors. Six-dimensional vector notations are generally the best, being more compact than 3-D vectors, and more powerful than 4×4 matrices. We therefore use 6-D vectors throughout this chapter. In particular, we shall use the

spatial vector algebra described in [2.8]. This section provides a brief summary of spatial vectors. Descriptions of 4×4 matrix notations can be found in [2.2, 9], and descriptions of other 6-D vector notations can be found in [2.10–12].

In this handbook, vectors are usually denoted by bold italic letters (e.g., $\mathbf{f}$, $\mathbf{v}$). However, to avoid a few name clashes, we shall use upright bold letters to denote spatial

vectors (e.g., $\mathbf{f}$, $\mathbf{v}$). Note that this applies only to vectors, not tensors. Also, *in this section only*, we will underline coordinate vectors to distinguish them from the vectors that they represent (e.g., $\underline{v}$ and $\underline{v}$, representing $\mathbf{v}$ and $\mathbf{v}$).

2.2.1 Motion and Force

For mathematical reasons, it is useful to distinguish between those vectors that describe the motions of rigid bodies, and those that describe the forces acting upon them. We therefore place motion vectors in a vector space called $\mathbf{M}^6$, and force vectors in a space called $\mathbf{F}^6$. (The superscripts indicate the dimension.) Motion vectors describe quantities like velocity, acceleration, infinitesimal displacement, and directions of motion freedom; force vectors describe force, momentum, contact normals, and so on.

2.2.2 Basis Vectors

Suppose that $\mathbf{v}$ is a 3-D vector, and that $\underline{v} = (v_x, v_y, v_z)^T$ is the Cartesian coordinate vector that represents $\mathbf{v}$ in the orthonormal basis $\{\hat{x}, \hat{y}, \hat{z}\}$. The relationship between $\mathbf{v}$ and $\underline{v}$ is then given by the formula

$$\mathbf{v} = \hat{x}v_x + \hat{y}v_y + \hat{z}v_z.$$

This same idea applies also to spatial vectors, except that we use Plücker coordinates instead of Cartesian coordinates, and a Plücker basis instead of an orthonormal basis.

Plücker coordinates were introduced in Sect. 1.2.6, but the basis vectors are shown in Fig. 2.1. There are 12 basis vectors in total: six for motion vectors and six for forces. Given a Cartesian coordinate frame, O_{xyz} , the Plücker basis vectors are defined as follows: three unit rotations about the directed lines Ox , Oy , and Oz , denoted by $\mathbf{d}_{Ox}$, $\mathbf{d}_{Oy}$, and $\mathbf{d}_{Oz}$, three unit translations in the directions x , y , and z , denoted by $\mathbf{d}_x$, $\mathbf{d}_y$, and $\mathbf{d}_z$, three unit couples about the x , y , and z directions,

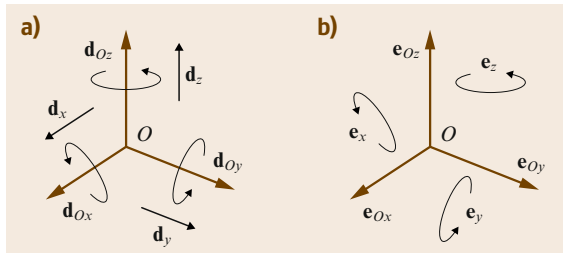


Fig. 2.1a,b Plücker basis vectors for motions (a) and forces (b)

denoted by $\mathbf{e}_x$, $\mathbf{e}_y$, and $\mathbf{e}_z$, and three unit forces along the lines Ox , Oy , and Oz , denoted by $\mathbf{e}_{Ox}$, $\mathbf{e}_{Oy}$, and $\mathbf{e}_{Oz}$.

2.2.3 Spatial Velocity and Force

Given any point O , the velocity of a rigid body can be described by a pair of 3-D vectors, $\boldsymbol{\omega}$ and $\mathbf{v}_O$, which specify the body's angular velocity and the linear velocity of the body-fixed point currently at O . Note that $\mathbf{v}_O$ is not the velocity of O itself, but the velocity of the body-fixed point that happens to coincide with O at the current instant.

The velocity of this same rigid body can also be described by a single spatial motion vector, $\mathbf{v} \in \mathbf{M}^6$. To obtain $\mathbf{v}$ from $\boldsymbol{\omega}$ and $\mathbf{v}_O$, we first introduce a Cartesian frame, O_{xyz} , with its origin at O . This frame defines a Cartesian coordinate system for $\boldsymbol{\omega}$ and $\mathbf{v}_O$, and also a Plücker coordinate system for $\mathbf{v}$. Given these coordinate systems, it can be shown that

$$\mathbf{v} = \mathbf{d}_{Ox}\omega_x + \mathbf{d}_{Oy}\omega_y + \mathbf{d}_{Oz}\omega_z + \mathbf{d}_x v_{Ox} + \mathbf{d}_y v_{Oy} + \mathbf{d}_z v_{Oz}, \quad (2.1)$$

where $\omega_x, \dots, v_{Oz}$ are the Cartesian coordinates of $\boldsymbol{\omega}$ and $\mathbf{v}_O$ in O_{xyz} . Thus, the Plücker coordinates of $\mathbf{v}$ are the Cartesian coordinates of $\boldsymbol{\omega}$ and $\mathbf{v}_O$. The coordinate vector representing $\mathbf{v}$ in O_{xyz} can be written

$$\underline{v}_O = \begin{pmatrix} \omega_x \\ \vdots \\ v_{Oz} \end{pmatrix} = \begin{pmatrix} \boldsymbol{\omega} \\ \underline{v}_O \end{pmatrix}. \quad (2.2)$$

The notation on the far right of this equation is simply a convenient abbreviation of the list of Plücker coordinates.

The definition of spatial force is very similar. Given any point O , any system of forces acting on a single rigid body is equivalent to a single force $\mathbf{f}$ acting on a line passing through O , together with a pure couple, $\mathbf{n}_O$, which is the moment of the force system about O . Thus, the two vectors $\mathbf{f}$ and $\mathbf{n}_O$ describe the force acting on a rigid body in much the same way that $\boldsymbol{\omega}$ and $\mathbf{v}_O$ describe its velocity. This same force can also be described by a single spatial force vector, $\mathbf{f} \in \mathbf{F}^6$. Introducing the frame O_{xyz} , as before, it can be shown that

$$\mathbf{f} = \mathbf{e}_x n_{Ox} + \mathbf{e}_y n_{Oy} + \mathbf{e}_z n_{Oz} + \mathbf{e}_{Ox} f_x + \mathbf{e}_{Oy} f_y + \mathbf{e}_{Oz} f_z, \quad (2.3)$$

where $n_{Ox}, \dots, f_z$ are the Cartesian coordinates of $\mathbf{n}_O$ and $\mathbf{f}$ in O_{xyz} . The coordinate vector representing $\mathbf{f}$ in

O_{xyz} can then be written

$$\underline{\mathbf{f}}_O = \begin{pmatrix} n_{Ox} \\ \vdots \\ f_z \end{pmatrix} = \begin{pmatrix} \underline{n}_O \\ \underline{f} \end{pmatrix}. \quad (2.4)$$

Again, these are the Plücker coordinates of $\mathbf{f}$ in O_{xyz} , and the notation on the far right is simply a convenient abbreviation of the list of Plücker coordinates.

2.2.4 Addition and Scalar Multiplication

Spatial vectors behave in the obvious way under addition and scalar multiplication. For example, if $\mathbf{f}_1$ and $\mathbf{f}_2$ both act on the same rigid body, then their resultant is $\mathbf{f}_1 + \mathbf{f}_2$; if two different bodies have velocities of $\mathbf{v}_1$ and $\mathbf{v}_2$, then the velocity of the second body relative to the first is $\mathbf{v}_2 - \mathbf{v}_1$; and if $\mathbf{f}$ denotes a force of 1 N acting along a particular line in space, then $\alpha \mathbf{f}$ denotes a force of α N acting along the same line.

2.2.5 Scalar Product

A scalar product is defined between any two spatial vectors, provided that one of them is a motion and the other a force. Given any $\mathbf{m} \in \mathbf{M}^6$ and $\mathbf{f} \in \mathbf{F}^6$, the scalar product can be written either $\mathbf{f} \cdot \mathbf{m}$ or $\mathbf{m} \cdot \mathbf{f}$, and expresses the work done by a force $\mathbf{f}$ acting on a body with motion $\mathbf{m}$. Expressions like $\mathbf{f} \cdot \mathbf{f}$ and $\mathbf{m} \cdot \mathbf{m}$ are not defined. If $\underline{\mathbf{m}}$ and $\underline{\mathbf{f}}$ are coordinate vectors representing $\mathbf{m}$ and $\mathbf{f}$ in the same coordinate system, then

$$\mathbf{m} \cdot \mathbf{f} = \underline{\mathbf{m}}^T \underline{\mathbf{f}}. \quad (2.5)$$

2.2.6 Coordinate Transforms

Motion and force vectors obey different transformation rules. Let A and B be two coordinate frames, each defining a coordinate system of the same name; and let $\underline{\mathbf{m}}_A$, $\underline{\mathbf{m}}_B$, $\underline{\mathbf{f}}_A$, and $\underline{\mathbf{f}}_B$ be coordinate vectors representing the spatial vectors $\mathbf{m} \in \mathbf{M}^6$ and $\mathbf{f} \in \mathbf{F}^6$ in A and B coordinates, respectively. The transformation rules are then

$$\underline{\mathbf{m}}_B = {}^B\mathbf{X}_A \underline{\mathbf{m}}_A \quad (2.6)$$

and

$$\underline{\mathbf{f}}_B = {}^B\mathbf{X}_A^F \underline{\mathbf{f}}_A, \quad (2.7)$$

where ${}^B\mathbf{X}_A$ and ${}^B\mathbf{X}_A^F$ are the coordinate transformation matrices from A to B for motion and force vectors,

respectively. These matrices are related by the identity

$${}^B\mathbf{X}_A^F \equiv ({}^B\mathbf{X}_A)^{-T} \equiv ({}^A\mathbf{X}_B)^T. \quad (2.8)$$

Suppose that the position and orientation of frame A relative to frame B is described by a position vector ${}^B\mathbf{p}_A$ and a 3×3 rotation matrix ${}^B\mathbf{R}_A$, as described in Sect. 1.2. The formula for ${}^B\mathbf{X}_A$ is then

$$\begin{aligned} {}^B\mathbf{X}_A &= \begin{pmatrix} \mathbf{1} & \mathbf{0} \\ S({}^B\mathbf{p}_A) & \mathbf{1} \end{pmatrix} \begin{pmatrix} {}^B\mathbf{R}_A & \mathbf{0} \\ \mathbf{0} & {}^B\mathbf{R}_A \end{pmatrix} \\ &= \begin{pmatrix} {}^B\mathbf{R}_A & \mathbf{0} \\ S({}^B\mathbf{p}_A){}^B\mathbf{R}_A & {}^B\mathbf{R}_A \end{pmatrix}, \end{aligned} \quad (2.9)$$

and its inverse is

$${}^A\mathbf{X}_B = \begin{pmatrix} {}^A\mathbf{R}_B & \mathbf{0} \\ \mathbf{0} & {}^A\mathbf{R}_B \end{pmatrix} \begin{pmatrix} \mathbf{1} & \mathbf{0} \\ -S({}^B\mathbf{p}_A) & \mathbf{1} \end{pmatrix}. \quad (2.10)$$

The quantity $S(\underline{\mathbf{p}})$ is the skew-symmetric matrix that satisfies $S(\underline{\mathbf{p}})\underline{\mathbf{v}} = \underline{\mathbf{p}} \times \underline{\mathbf{v}}$ for any 3-D vector $\underline{\mathbf{v}}$. It is defined by the equation

$$S(\underline{\mathbf{p}}) = \begin{pmatrix} 0 & -p_z & p_y \\ p_z & 0 & -p_x \\ -p_y & p_x & 0 \end{pmatrix}. \quad (2.11)$$

2.2.7 Vector Products

There are two vector (cross) products defined on spatial vectors. The first takes two motion-vector arguments, and produces a motion-vector result. It is defined by the formula

$$\begin{aligned} \underline{\mathbf{m}}_1 \times \underline{\mathbf{m}}_2 &= \begin{pmatrix} \underline{\mathbf{m}}_1 \\ \underline{\mathbf{m}}_{1O} \end{pmatrix} \times \begin{pmatrix} \underline{\mathbf{m}}_2 \\ \underline{\mathbf{m}}_{2O} \end{pmatrix} \\ &= \begin{pmatrix} \underline{\mathbf{m}}_1 \times \underline{\mathbf{m}}_2 \\ \underline{\mathbf{m}}_1 \times \underline{\mathbf{m}}_{2O} + \underline{\mathbf{m}}_{1O} \times \underline{\mathbf{m}}_2 \end{pmatrix}. \end{aligned} \quad (2.12)$$

The second takes a motion vector as left-hand argument and a force vector as right-hand argument, and produces a force-vector result. It is defined by the formula

$$\begin{aligned} \underline{\mathbf{m}} \times \underline{\mathbf{f}} &= \begin{pmatrix} \underline{\mathbf{m}} \\ \underline{\mathbf{m}}_O \end{pmatrix} \times \begin{pmatrix} \underline{\mathbf{f}} \\ \underline{\mathbf{f}}_O \end{pmatrix} \\ &= \begin{pmatrix} \underline{\mathbf{m}} \times \underline{\mathbf{f}}_O + \underline{\mathbf{m}}_O \times \underline{\mathbf{f}} \\ \underline{\mathbf{m}} \times \underline{\mathbf{f}} \end{pmatrix}. \end{aligned} \quad (2.13)$$

These products arise in differentiation formulae.

It is possible to define a spatial cross-product operator, in analogy with (2.11), as follows:

$$S(\underline{\mathbf{m}}) = \begin{pmatrix} S(\underline{\mathbf{m}}) & \mathbf{0} \\ S(\underline{\mathbf{m}}_O) & S(\underline{\mathbf{m}}) \end{pmatrix}, \quad (2.14)$$

in which case

$$\underline{\mathbf{m}}_1 \times \underline{\mathbf{m}}_2 = S(\underline{\mathbf{m}}_1) \underline{\mathbf{m}}_2, \quad (2.15)$$

but

$$\underline{\mathbf{m}} \times \underline{\mathbf{f}} = -S(\underline{\mathbf{m}})^T \underline{\mathbf{f}}. \quad (2.16)$$

Observe that $S(\underline{\mathbf{m}})$ maps motion vectors to motion vectors, but $S(\underline{\mathbf{m}})^T$ maps force vectors to force vectors.

2.2.8 Differentiation

The derivative of a spatial vector is defined by

$$\frac{d}{dx} \mathbf{s}(x) = \lim_{\delta x \rightarrow 0} \frac{\mathbf{s}(x + \delta x) - \mathbf{s}(x)}{\delta x}, \quad (2.17)$$

where $\mathbf{s}$ here stands for any spatial vector. The derivative is a spatial vector of the same kind (motion or force) as that being differentiated.

The formula for differentiating a spatial coordinate vector in a moving coordinate system is

$$\left(\frac{d}{dt} \mathbf{s} \right)_A = \frac{d}{dt} \underline{\mathbf{s}}_A + \underline{\mathbf{v}}_A \times \underline{\mathbf{s}}_A, \quad (2.18)$$

where $\mathbf{s}$ is any spatial vector, ds/dt is the time derivative of $\mathbf{s}$, A is the moving coordinate system, $(ds/dt)_A$ is the coordinate vector that represents ds/dt in A coordinates, $\underline{\mathbf{s}}_A$ is the coordinate vector that represents $\mathbf{s}$ in A coordinates, $d\underline{\mathbf{s}}_A/dt$ is the time derivative of $\underline{\mathbf{s}}_A$ (which is the componentwise derivative, since $\underline{\mathbf{s}}_A$ is a coordinate vector), and $\underline{\mathbf{v}}_A$ is the velocity of the A coordinate frame, expressed in A coordinates.

The time derivative of a spatial vector that changes only because it is moving is given by

$$\frac{d}{dt} \mathbf{s} = \mathbf{v} \times \mathbf{s}, \quad (2.19)$$

where $\mathbf{v}$ is the velocity of $\mathbf{s}$. This formula is useful for differentiating quantities that do not change in their own right, but are attached to moving rigid bodies (e.g., joint axis vectors).

2.2.9 Acceleration

Spatial acceleration is defined as the rate of change of spatial velocity. Unfortunately, this means that spatial

acceleration differs from the classical textbook definition of rigid-body acceleration, which we shall call *classical acceleration*. Essentially, the difference can be summarized as follows:

$$\underline{\mathbf{a}} = \begin{pmatrix} \dot{\underline{\omega}} \\ \dot{\underline{\mathbf{v}}}_O \end{pmatrix} \quad \text{and} \quad \underline{\mathbf{a}}' = \begin{pmatrix} \dot{\underline{\omega}} \\ \dot{\underline{\mathbf{v}}}'_O \end{pmatrix}, \quad (2.20)$$

where $\underline{\mathbf{a}}$ is the spatial acceleration, $\underline{\mathbf{a}}'$ is the classical acceleration, $\underline{\mathbf{v}}_O$ is the derivative of $\underline{\mathbf{v}}_O$ taking O to be fixed in space, and $\underline{\mathbf{v}}'_O$ is the derivative of $\underline{\mathbf{v}}_O$ taking O to be fixed in the body. The two accelerations are related by

$$\underline{\mathbf{a}}' = \underline{\mathbf{a}} + \begin{pmatrix} \mathbf{0} \\ \underline{\omega} \times \underline{\mathbf{v}}_O \end{pmatrix}. \quad (2.21)$$

If $\mathbf{r}$ is a position vector giving the position of the body-fixed point at O relative to any fixed point, then

$$\begin{aligned} \mathbf{v}_O &= \dot{\mathbf{r}}, \\ \dot{\mathbf{v}}_O &= \ddot{\mathbf{r}}, \\ \dot{\mathbf{v}}_O &= \ddot{\mathbf{r}} - \underline{\omega} \times \mathbf{v}_O. \end{aligned} \quad (2.22)$$

The practical difference is that spatial accelerations are easier to use. For example, if the bodies B_1 and B_2 have velocities of $\mathbf{v}_1$ and $\mathbf{v}_2$, respectively, and $\mathbf{v}_{\text{rel}}$ is the relative velocity of B_2 with respect to B_1 , then

$$\mathbf{v}_2 = \mathbf{v}_1 + \mathbf{v}_{\text{rel}}.$$

The relationship between their spatial accelerations is obtained simply by differentiating the velocity formula:

$$\frac{d}{dt} (\mathbf{v}_2 = \mathbf{v}_1 + \mathbf{v}_{\text{rel}}) \Rightarrow \underline{\mathbf{a}}_2 = \underline{\mathbf{a}}_1 + \underline{\mathbf{a}}_{\text{rel}}.$$

Observe that spatial accelerations are composed by addition, exactly like velocities. There are no Coriolis or centrifugal terms to worry about. This is a significant improvement on the formulae for composing classical accelerations, such as those in [2.2, 13, 14].

2.2.10 Spatial Momentum

Suppose that a rigid body has a mass of m , a center of mass at C , and a rotational inertia of $\bar{\mathbf{I}}^{\text{cm}}$ about C (Fig. 2.2). If this body is moving with a spatial velocity of $\underline{\mathbf{v}}_C = (\underline{\omega}^T \underline{\mathbf{v}}_C^T)^T$, then its linear momentum is $\mathbf{h} = m\mathbf{v}_C$, and its intrinsic angular momentum is $\mathbf{h}_C = \bar{\mathbf{I}}^{\text{cm}}\underline{\omega}$. Its moment of momentum about a general point, O , is $\mathbf{h}_O = \mathbf{h}_C + \mathbf{c} \times \mathbf{h}$, where $\mathbf{c} = \overrightarrow{OC}$. We can assemble these vectors into a spatial momentum vector as follows:

$$\underline{\mathbf{h}}_C = \begin{pmatrix} \mathbf{h}_C \\ \mathbf{h} \end{pmatrix} = \begin{pmatrix} \bar{\mathbf{I}}^{\text{cm}} \underline{\omega} \\ m \underline{\mathbf{v}}_C \end{pmatrix} \quad (2.23)$$

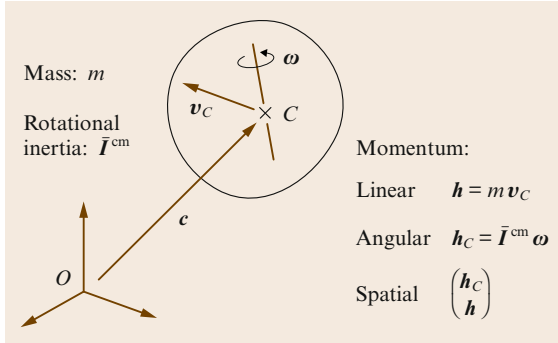


Fig. 2.2 Spatial momentum

and

$$\underline{\mathbf{h}}_O = \begin{pmatrix} \underline{\mathbf{h}}_O \\ \underline{\mathbf{h}} \end{pmatrix} = \begin{pmatrix} \mathbf{1} & S(\mathbf{c}) \\ \mathbf{0} & \mathbf{1} \end{pmatrix} \underline{\mathbf{h}}_C. \quad (2.24)$$

Spatial momentum is a force vector, and transforms accordingly.

2.2.11 Spatial Inertia

The spatial momentum of a rigid body is the product of its spatial inertia and velocity:

$$\mathbf{h} = \mathbf{I} \mathbf{v}, \quad (2.25)$$

where $\mathbf{I}$ is the spatial inertia. Expressed in Plücker coordinates at C , we have

$$\mathbf{h}_C = \mathbf{I}_C \mathbf{v}_C, \quad (2.26)$$

which implies

$$\mathbf{I}_C = \begin{pmatrix} \bar{I}^{\text{cm}} & \mathbf{0} \\ \mathbf{0} & m\mathbf{1} \end{pmatrix}. \quad (2.27)$$

This is the general formula for the spatial inertia of a rigid body expressed at its center of mass. To express it at another point, O , we proceed as follows. From (2.24), (2.26), and (2.27)

$$\begin{aligned} \underline{\mathbf{h}}_O &= \begin{pmatrix} \mathbf{1} & S(\mathbf{c}) \\ \mathbf{0} & \mathbf{1} \end{pmatrix} \begin{pmatrix} \bar{I}^{\text{cm}} & \mathbf{0} \\ \mathbf{0} & m\mathbf{1} \end{pmatrix} \underline{\mathbf{v}}_C \\ &= \begin{pmatrix} \mathbf{1} & S(\mathbf{c}) \\ \mathbf{0} & \mathbf{1} \end{pmatrix} \begin{pmatrix} \bar{I}^{\text{cm}} & \mathbf{0} \\ \mathbf{0} & m\mathbf{1} \end{pmatrix} \begin{pmatrix} \mathbf{1} & \mathbf{0} \\ S(\mathbf{c})^T & \mathbf{1} \end{pmatrix} \underline{\mathbf{v}}_O \\ &= \begin{pmatrix} \bar{I}^{\text{cm}} + mS(\mathbf{c})S(\mathbf{c})^T & mS(\mathbf{c}) \\ mS(\mathbf{c})^T & m\mathbf{1} \end{pmatrix} \underline{\mathbf{v}}_O; \end{aligned}$$

but we also have that $\underline{\mathbf{h}}_O = \mathbf{I}_O \underline{\mathbf{v}}_O$, so

$$\mathbf{I}_O = \begin{pmatrix} \bar{I}^{\text{cm}} + mS(\mathbf{c})S(\mathbf{c})^T & mS(\mathbf{c}) \\ mS(\mathbf{c})^T & m\mathbf{1} \end{pmatrix}. \quad (2.28)$$

This equation can also be written

$$\mathbf{I}_O = \begin{pmatrix} \bar{I}_O & mS(\mathbf{c}) \\ mS(\mathbf{c})^T & m\mathbf{1} \end{pmatrix}, \quad (2.29)$$

where

$$\bar{I}_O = \bar{I}^{\text{cm}} + mS(\mathbf{c})S(\mathbf{c})^T \quad (2.30)$$

is the rotational inertia of the rigid body about O .

Spatial inertia matrices are symmetric and positive-definite. In the general case, 21 numbers are required to specify a spatial inertia (e.g., for an articulated-body or operational-space inertia); but a rigid-body inertia needs only 10 parameters: the mass, the coordinates of the center of mass, and the six independent elements of either $\bar{I}^{\text{cm}}$ or $\bar{I}_O$.

The transformation rule for spatial inertias is

$$\mathbf{I}_B = {}^B\mathbf{X}_A^F \mathbf{I}_A {}^A\mathbf{X}_B, \quad (2.31)$$

where A and B are any two coordinate systems. In practice, we often need to calculate $\mathbf{I}_A$ from $\mathbf{I}_B$, given only ${}^B\mathbf{X}_A$. The formula for this transformation is

$$\mathbf{I}_A = ({}^B\mathbf{X}_A)^T \mathbf{I}_B {}^B\mathbf{X}_A. \quad (2.32)$$

If two bodies, having inertias $\mathbf{I}_1$ and $\mathbf{I}_2$, are rigidly connected to form a single composite body, then the inertia of the composite, $\mathbf{I}_{\text{tot}}$, is the sum of the inertias of its parts:

$$\mathbf{I}_{\text{tot}} = \mathbf{I}_1 + \mathbf{I}_2. \quad (2.33)$$

This single equation takes the place of three equations in the traditional 3-D vector approach: one to compute the composite mass, one to compute the composite center of mass, and one to compute the composite rotational inertia. If a rigid body with inertia $\mathbf{I}$ is moving with a velocity of $\mathbf{v}$, then its kinetic energy is

$$T = \frac{1}{2} \mathbf{v} \cdot \mathbf{I} \mathbf{v}. \quad (2.34)$$

If a rigid body, B , is part of a larger system, then it is possible to define an apparent-inertia matrix for B , which describes the relationship between a force acting on B and its resulting acceleration, taking into account the effects of the other bodies in the system. Such quantities are called articulated-body inertias. If B happens to be the end-effector of a robot, then its apparent inertia is called an operational-space inertia.

2.2.12 Equation of Motion

The spatial equation of motion states that the net force acting on a rigid body equals its rate of change of momentum:

$$\mathbf{f} = \frac{d}{dt}(\mathbf{I} \mathbf{v}) = \mathbf{I} \mathbf{a} + \dot{\mathbf{I}} \mathbf{v}.$$

It can be shown that the expression $\dot{\mathbf{I}}\mathbf{v}$ evaluates to $(\mathbf{v} \times \mathbf{I}\mathbf{v})$ [2.8, 15], so the equation of motion can be written

$$\mathbf{f} = \mathbf{I}\mathbf{a} + \mathbf{v} \times \mathbf{I}\mathbf{v}. \quad (2.35)$$

This single equation incorporates both Newton's and Euler's equations of motion for a rigid body. To verify this, we can recover them as follows. Expressing (2.35) at the body's center of mass, and using (2.16), (2.14), and (2.22), we have

$$\begin{aligned} \begin{pmatrix} \underline{\mathbf{n}}_C \\ \underline{\mathbf{f}} \end{pmatrix} &= \begin{pmatrix} \bar{\mathbf{I}}^{\text{cm}} & \mathbf{0} \\ \mathbf{0} & m\mathbf{1} \end{pmatrix} \begin{pmatrix} \underline{\dot{\boldsymbol{\omega}}} \\ \underline{\dot{\mathbf{v}}}_C \end{pmatrix} \\ &\quad - \begin{pmatrix} S(\underline{\boldsymbol{\omega}})^T & S(\underline{\mathbf{v}}_C)^T \\ \mathbf{0} & S(\underline{\boldsymbol{\omega}})^T \end{pmatrix} \begin{pmatrix} \bar{\mathbf{I}}^{\text{cm}}\underline{\boldsymbol{\omega}} \\ m\underline{\mathbf{v}}_C \end{pmatrix} \\ &= \begin{pmatrix} \bar{\mathbf{I}}^{\text{cm}} & \mathbf{0} \\ \mathbf{0} & m\mathbf{1} \end{pmatrix} \begin{pmatrix} \underline{\dot{\boldsymbol{\omega}}} \\ \underline{\ddot{\mathbf{c}}} - \underline{\boldsymbol{\omega}} \times \underline{\mathbf{v}}_C \end{pmatrix} + \begin{pmatrix} \underline{\boldsymbol{\omega}} \times \bar{\mathbf{I}}^{\text{cm}}\underline{\boldsymbol{\omega}} \\ m\underline{\boldsymbol{\omega}} \times \underline{\mathbf{v}}_C \end{pmatrix} \\ &= \begin{pmatrix} \bar{\mathbf{I}}^{\text{cm}}\underline{\dot{\boldsymbol{\omega}}} + \underline{\boldsymbol{\omega}} \times \bar{\mathbf{I}}^{\text{cm}}\underline{\boldsymbol{\omega}} \\ m\underline{\ddot{\mathbf{c}}} \end{pmatrix}. \end{aligned} \quad (2.36)$$

2.2.13 Computer Implementation

The easiest way to implement spatial vector arithmetic on a computer is to start with an existing matrix arithmetic tool, like MATLAB® and write (or download from the Web) routines to do the following:

1. calculate $S(\mathbf{m})$ from $\mathbf{m}$ according to (2.14)
2. compose X from R and p according to (2.9)
3. compose I from m , c , and $\bar{I}^{\text{cm}}$ according to (2.28)

All other spatial arithmetic operations can be performed using standard matrix arithmetic routines. However, some additional routines could usefully be added to this list, such as:

- routines to calculate R from various other representations of rotation
- routines to convert between spatial and 4×4 matrix quantities

This is the recommended approach whenever human productivity is more important than computational efficiency.

If greater efficiency is required, then a more elaborate spatial arithmetic library must be used, in which

1. a dedicated data structure is defined for each kind of spatial quantity, and
2. a suite of calculation routines are provided, each implementing a spatial arithmetic operation by means of an efficient formula.

Some examples of suitable data structures and efficient formulae are shown in Table 2.1. Observe that the suggested data structures for rigid-body inertias and Plücker transforms contain only a third as many numbers as the 6×6 matrices they represent. The efficient arithmetic formulae listed in this table offer cost savings ranging from a factor of 1.5 to a factor of 6 relative to the use of general 6×6 and 6×1 matrix arithmetic. Even more efficient formulae can be found in [2.16].

Table 2.1 Summary of spatial vector notation

Spatial quantities:	
$\mathbf{v}$	velocity of a rigid body
$\mathbf{a}$	spatial acceleration of a rigid body ($\mathbf{a} = \dot{\mathbf{v}}$)
$\mathbf{a}'$	classical description of rigid-body acceleration expressed as a 6-D vector
$\mathbf{f}$	force acting on a rigid body
$\mathbf{I}$	inertia of a rigid body
X	Plücker coordinate transform for motion vectors
X^F	Plücker coordinate transform for force vectors ($X^F = X^{-T}$)
${}^B X_A$	Plücker transform from A coordinates to B coordinates
$\mathbf{m}$	a generic motion vector (any element of M^6)
3-D quantities:	
O	coordinate system origin
$\mathbf{r}$	position of the body-fixed point at O relative to any fixed point in space
$\boldsymbol{\omega}$	angular velocity of a rigid body
$\mathbf{v}_O$	linear velocity of the body-fixed point at O ($\mathbf{v}_O = \dot{\mathbf{r}}$)
$\dot{\boldsymbol{\omega}}$	angular acceleration of a rigid body
$\dot{\mathbf{v}}_O$	the derivative of $\mathbf{v}_O$ taking O to be fixed in space
$\mathbf{v}'_O$	the derivative of $\mathbf{v}_O$ taking O to be fixed in the body; the classical acceleration of the body-fixed point at O ($\mathbf{v}'_O = \ddot{\mathbf{r}}$)
$\mathbf{f}$	linear force acting on a rigid body, or the resultant of a system of linear forces
$\mathbf{n}_O$	moment about O of a linear force or system of linear forces
m	mass of a rigid body
$\mathbf{c}$	position of a rigid body's centre of mass, measured relative to O
$\mathbf{h}$	first moment of mass of a rigid body, $\mathbf{h} = m\mathbf{c}$; can also denote linear momentum
$\bar{I}^{\text{cm}}$	moment of inertia about a body's centre of mass
$\bar{I}$	moment of inertia about O
${}^B R_A$	orthonormal rotation matrix transforming from A coordinates to B coordinates
${}^A p_B$	location of the origin of B coordinates relative to the origin of A coordinates, expressed in A coordinates

Table 2.1 (continued)

Equations:			
$\mathbf{v} = \begin{pmatrix} \omega \\ v_O \end{pmatrix}$	$\mathbf{a} = \begin{pmatrix} \dot{\omega} \\ \dot{v}_O \end{pmatrix} = \begin{pmatrix} \dot{\omega} \\ \ddot{r} - \omega \times \dot{r} \end{pmatrix}$		
$\mathbf{f} = \begin{pmatrix} n_O \\ f \end{pmatrix}$	$\mathbf{a}' = \begin{pmatrix} \dot{\omega} \\ \dot{v}_O \end{pmatrix} = \begin{pmatrix} \dot{\omega} \\ \ddot{r} \end{pmatrix} = \mathbf{a} + \begin{pmatrix} 0 \\ \omega \times v_O \end{pmatrix}$		
$\mathbf{I} = \begin{pmatrix} \bar{\mathbf{I}} & S(h) \\ S(h)^T & m\mathbf{1} \end{pmatrix}$	$= \begin{pmatrix} \bar{\mathbf{I}}^{\text{cm}} + mS(c)S(c)^T & mS(c) \\ mS(c)^T & m\mathbf{1} \end{pmatrix}$		
${}^B\mathbf{X}_A = \begin{pmatrix} {}^B\mathbf{R}_A & 0 \\ {}^B\mathbf{R}_A S({}^A\mathbf{p}_B)^T & {}^B\mathbf{R}_A \end{pmatrix}$	$= \begin{pmatrix} {}^B\mathbf{R}_A & 0 \\ S({}^B\mathbf{p}_A) {}^B\mathbf{R}_A & {}^B\mathbf{R}_A \end{pmatrix}$		
$\mathbf{v} \cdot \mathbf{f} = \mathbf{f} \cdot \mathbf{v} = \mathbf{v}^T \mathbf{f} = \omega \cdot \mathbf{n}_O + v_O \cdot f$			
$\mathbf{v} \times \mathbf{m} = \begin{pmatrix} \omega \times \mathbf{m} \\ v_O \times \mathbf{m} + \omega \times \mathbf{m}_O \end{pmatrix}$	$= \begin{pmatrix} S(\omega) & 0 \\ S(v_O) & S(\omega) \end{pmatrix} \begin{pmatrix} \mathbf{m} \\ \mathbf{m}_O \end{pmatrix}$		
$\mathbf{v} \times \mathbf{f} = \begin{pmatrix} \omega \times \mathbf{n}_O + v_O \times f \\ \omega \times f \end{pmatrix}$	$= \begin{pmatrix} S(\omega) & S(v_O) \\ 0 & S(\omega) \end{pmatrix} \begin{pmatrix} \mathbf{n}_O \\ f \end{pmatrix}$		
Compact computer representations:			
Mathematical object	Size	Computer representation	Size
$\begin{pmatrix} \omega \\ v_O \end{pmatrix}$	6×1	$(\omega; v_O)$	3+3
$\begin{pmatrix} n_O \\ f \end{pmatrix}$	6×1	$(n_O; f)$	3+3
$\begin{pmatrix} \bar{\mathbf{I}} & S(h) \\ S(h)^T & m\mathbf{1} \end{pmatrix}$	6×6	$(m; h; \bar{\mathbf{I}})$	1+3+9
$\begin{pmatrix} \mathbf{R} & 0 \\ \mathbf{R}S(p)^T & \mathbf{R} \end{pmatrix}$	6×6	$(\mathbf{R}; p)$	9+3

2.2.14 Summary

2.3 Canonical Equations

The equations of motion of a robot mechanism are usually presented in one of two canonical forms: the joint-space formulation,

$$\mathbf{H}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \boldsymbol{\tau}_g(\mathbf{q}) = \boldsymbol{\tau}, \quad (2.37)$$

or the operational-space formulation,

$$\boldsymbol{\Lambda}(\mathbf{x})\dot{\mathbf{v}} + \boldsymbol{\mu}(\mathbf{x}, \mathbf{v}) + \boldsymbol{\rho}(\mathbf{x}) = \mathbf{f}. \quad (2.38)$$

These equations show the functional dependencies explicitly: $\mathbf{H}$ is a function of $\mathbf{q}$, $\boldsymbol{\Lambda}$ is a function of $\mathbf{x}$,

Table 2.1 (continued)

Efficient spatial arithmetic formulae:	
Expression	Computed value
$\mathbf{X}\mathbf{v}$	$(\mathbf{R}\omega; \mathbf{R}(v_O - p \times \omega))$
$\mathbf{X}^F \mathbf{f}$	$(\mathbf{R}(n_O - p \times f); \mathbf{R}f)$
$\mathbf{X}^{-1}$	$(\mathbf{R}^T; -\mathbf{R}p)$
$\mathbf{X}^{-1}\mathbf{v}$	$(\mathbf{R}^T\omega; \mathbf{R}^T v_O + p \times \mathbf{R}^T\omega)$
$(\mathbf{X}^F)^{-1}\mathbf{f}$	$(\mathbf{R}^T n_O + p \times \mathbf{R}^T f; \mathbf{R}^T f)$
$\mathbf{X}_1 \mathbf{X}_2$	$(\mathbf{R}_1 \mathbf{R}_2; p_2 + \mathbf{R}_2^T p_1)$
$\mathbf{I}_1 + \mathbf{I}_2$	$(m_1 + m_2; h_1 + h_2; \bar{\mathbf{I}}_1 + \bar{\mathbf{I}}_2)$
$\mathbf{I}\mathbf{v}$	$(\bar{\mathbf{I}}\omega + h \times v_O; m v_O - h \times \omega)$
$\mathbf{X}^T \mathbf{I} \mathbf{X}$	$(m; \mathbf{R}^T h + m p; \mathbf{R}^T \bar{\mathbf{I}} \mathbf{R} - S(p)S(\mathbf{R}^T h) - S(\mathbf{R}^T h + m p)S(p))$
For meaning of $\mathbf{X}^T \mathbf{I} \mathbf{X}$ see (2.32)	

Spatial vectors are 6-D vectors that combine the linear and angular aspects of rigid-body motion, resulting in a compact notation that is very suitable for describing dynamics algorithms. To avoid a few name clashes with 3-D vectors, we have used bold upright letters to denote spatial vectors, while tensors are still denoted by italics. In the sections that follow, upright letters will be used to denote both spatial vectors and vectors that are concatenations of other vectors, like $\dot{\mathbf{q}}$.

Table 2.1 presents a summary of the spatial quantities and operators introduced in this section, together with the formulae that define them in terms of 3-D quantities and operators. It also presents data structures and formulae for efficient computer implementation of spatial arithmetic. This table should be read in conjunction with Tables 1.5 and 1.6, which shows how to compute the orientation, position, and spatial velocities for a variety of joint types. Note that ${}^j\mathbf{R}_i$ and ${}^j\mathbf{p}_i$ in these tables correspond to ${}^B\mathbf{R}_A^T$ and ${}^A\mathbf{p}_B$, respectively, when read in conjunction with Table 2.1.

and so on. Once these dependencies are understood, they are usually omitted. In (2.38), $\mathbf{x}$ is a vector of operational-space coordinates, while $\mathbf{v}$ and $\mathbf{f}$ are spatial vectors denoting the velocity of the end-effector and the external force acting on it. If the robot is redundant, then the coefficients of this equation must be defined as functions of $\mathbf{q}$ and $\dot{\mathbf{q}}$ rather than $\mathbf{x}$ and $\mathbf{v}$.

These two equations are further explained below, along with a description of the Lagrange formulation of (2.37), and the impulsive equations of motion for impact.

2.3.1 Joint-Space Formulation

The symbols $\mathbf{q}$, $\dot{\mathbf{q}}$, $\ddot{\mathbf{q}}$, and $\boldsymbol{\tau}$ denote n -dimensional vectors of joint position, velocity, acceleration and force variables, respectively, where n is the number of degrees of motion freedom of the robot mechanism. $\mathbf{H}$ is an $n \times n$ symmetric, positive-definite matrix, and is called the generalized, or joint-space, inertia matrix (JSIM). $\mathbf{C}$ is an $n \times n$ matrix such that $\mathbf{C} \dot{\mathbf{q}}$ is the vector of Coriolis and centrifugal terms (collectively known as *velocity product* terms); and $\boldsymbol{\tau}_g$ is the vector of gravity terms. More terms can be added to this equation, as required, to account for other dynamical effects (e.g., viscous friction). The effects of a force $\mathbf{f}$ exerted on the mechanism at the end-effector can be accounted for by adding the term $\mathbf{J}^T \mathbf{f}$ to the right side of (2.37), where $\mathbf{J}$ is the Jacobian of the end-effector (Sect. 1.8.1).

$\mathbf{q}$ specifies the coordinates of a point in the mechanism's configuration space. If the mechanism is a kinematic tree (see Sect. 2.4), then $\mathbf{q}$ contains every joint variable in the mechanism, otherwise it contains only an independent subset. The elements of $\mathbf{q}$ are generalized coordinates. Likewise, the elements of $\dot{\mathbf{q}}$, $\ddot{\mathbf{q}}$, and $\boldsymbol{\tau}$ are generalized velocities, accelerations, and forces.

2.3.2 Lagrange Formulation

Various methods exist for deriving the terms in (2.37). The two that are most commonly used in robotics are the Newton–Euler formulation and the Lagrange formulation. The former works directly with Newton's and Euler's equations for a rigid body, which are contained within the spatial equation of motion, (2.35). This formulation is especially amenable to the development of efficient recursive algorithms for dynamics computations, such as those described in Sections 2.5 and 2.6.

The Lagrange formulation proceeds via the Lagrangian of the robot mechanism,

$$L = T - U, \quad (2.39)$$

where T and U are the total kinetic and potential energy, respectively, of the mechanism. The kinetic energy is given by

$$T = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{H} \dot{\mathbf{q}}. \quad (2.40)$$

The dynamic equations of motion can then be developed using Lagrange's equation for each generalized coordinate:

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = \tau_i. \quad (2.41)$$

The resulting equation can be written in scalar form:

$$\sum_{j=1}^n H_{ij} \ddot{q}_j + \sum_{j=1}^n \sum_{k=1}^n C_{ijk} \dot{q}_j \dot{q}_k + \tau_{gi} = \tau_i, \quad (2.42)$$

which shows the structure of the velocity-product terms. C_{ijk} are known as Christoffel symbols of the first type, and are given by

$$C_{ijk} = \frac{1}{2} \left(\frac{\partial H_{ij}}{\partial q_k} + \frac{\partial H_{ik}}{\partial q_j} - \frac{\partial H_{jk}}{\partial q_i} \right). \quad (2.43)$$

They are functions of only the position variables, q_i . The elements of $\mathbf{C}$ in (2.37) can be defined as

$$C_{ij} = \sum_{k=1}^n C_{ijk} \dot{q}_k. \quad (2.44)$$

However, $\mathbf{C}$ is not unique, and other definitions are possible.

With the choice of $\mathbf{C}$ given in (2.44), it is possible to show that the matrix $\mathbf{N}$, given by

$$\mathbf{N}(\mathbf{q}, \dot{\mathbf{q}}) = \dot{\mathbf{H}}(\mathbf{q}) - 2\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}), \quad (2.45)$$

is skew-symmetric [2.17]. Thus, for any $n \times 1$ vector $\boldsymbol{\alpha}$,

$$\boldsymbol{\alpha}^T \mathbf{N}(\mathbf{q}, \dot{\mathbf{q}}) \boldsymbol{\alpha} = 0. \quad (2.46)$$

This property is quite useful in control, especially when considering $\boldsymbol{\alpha} = \dot{\mathbf{q}}$, which gives

$$\dot{\mathbf{q}}^T \mathbf{N}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} = 0. \quad (2.47)$$

By applying the principle of conservation of energy, it can be shown that (2.47) holds for any choice of the matrix $\mathbf{C}$ [2.17, 18].

2.3.3 Operational-Space Formulation

In (2.38), $\mathbf{x}$ is a 6-D vector of operational-space coordinates giving the position and orientation of the robot's end-effector; $\mathbf{v}$ is the velocity of the end-effector; and $\mathbf{f}$ is the force exerted on the end-effector. $\mathbf{x}$ is typically a list of Cartesian coordinates, and Euler angles or quaternion components, and is related to $\mathbf{v}$ via a differential equation of the form

$$\dot{\mathbf{x}} = \mathbf{E}(\mathbf{x}) \mathbf{v}. \quad (2.48)$$

$\mathbf{A}$ is the operational-space inertia matrix, which is the apparent inertia of the end-effector taking into account the effect of the rest of the robot's mechanism (i.e., it is an articulated-body inertia). $\boldsymbol{\mu}$ and $\boldsymbol{\rho}$ are vectors of velocity-product and gravity terms, respectively.

Operational space (also known as task space) is the space in which high-level motion and force commands are issued and executed. The operational-space formulation is therefore particularly useful in the context of motion and force control systems (Sect. 6.2 and Sect. 7.2). Equation (2.38) can be generalized to operational spaces with dimensions other than six, and to operational spaces that incorporate the motions of more than one end-effector [2.19].

The terms in (2.37) and (2.38) are related by the following formulae

$$\mathbf{v} = \mathbf{J}\dot{\mathbf{q}}, \quad (2.49)$$

$$\dot{\mathbf{v}} = \mathbf{J}\ddot{\mathbf{q}} + \dot{\mathbf{J}}\dot{\mathbf{q}}, \quad (2.50)$$

$$\boldsymbol{\tau} = \mathbf{J}^T \mathbf{f}, \quad (2.51)$$

$$\boldsymbol{\Lambda} = (\mathbf{J}\mathbf{H}^{-1}\mathbf{J}^T)^{-1}, \quad (2.52)$$

$$\boldsymbol{\mu} = \boldsymbol{\Lambda}(\mathbf{J}\mathbf{H}^{-1}\mathbf{C}\dot{\mathbf{q}} - \dot{\mathbf{J}}\dot{\mathbf{q}}), \quad (2.53)$$

and

$$\boldsymbol{\rho} = \boldsymbol{\Lambda}\mathbf{J}\mathbf{H}^{-1}\boldsymbol{\tau}_g. \quad (2.54)$$

These equations assume that $m \leq n$ (m is the dimension of operational-space coordinates), and that the Jacobian $\mathbf{J}$ has full rank. More details can be found in [2.20].

2.3.4 Impact Model

If a robot strikes a rigid body in its environment, then an impulsive force arises at the moment of impact and causes a step change in the robot's velocity. Let us assume that the impact occurs between the end effector and a rigid body in the environment, and that a spatial

impulse of $\mathbf{f}'$ is exerted on the end-effector. This impulse causes a step change of $\Delta\mathbf{v}$ in the end-effector's velocity; and the two are related by the operational-space equation of impulsive motion [2.21],

$$\boldsymbol{\Lambda}\Delta\mathbf{v} = \mathbf{f}'. \quad (2.55)$$

In joint space, the equation of impulsive motion for a robot mechanism is

$$\mathbf{H}\Delta\dot{\mathbf{q}} = \boldsymbol{\tau}', \quad (2.56)$$

where $\boldsymbol{\tau}'$ and $\Delta\dot{\mathbf{q}}$ denote the joint-space impulse and velocity change, respectively. In the case of a collision involving the robot's end-effector, we have

$$\boldsymbol{\tau}' = \mathbf{J}^T \mathbf{f}' \quad (2.57)$$

and

$$\Delta\mathbf{v} = \mathbf{J}\Delta\dot{\mathbf{q}}, \quad (2.58)$$

which follow from (2.51) and (2.49). Equations (2.55)–(2.57) imply that

$$\Delta\dot{\mathbf{q}} = \bar{\mathbf{J}}\Delta\mathbf{v}, \quad (2.59)$$

where $\bar{\mathbf{J}}$ is the inertia-weighted pseudoinverse of $\mathbf{J}$ and is given by

$$\bar{\mathbf{J}} = \mathbf{H}^{-1}\mathbf{J}^T\boldsymbol{\Lambda}. \quad (2.60)$$

$\bar{\mathbf{J}}$ is also known as the dynamically consistent generalized inverse of the Jacobian matrix [2.20]. Note that the expression $\boldsymbol{\Lambda}\mathbf{J}\mathbf{H}^{-1}$, which appears in (2.53) and (2.54), is equal to $\bar{\mathbf{J}}^T$ since $\mathbf{H}$ and $\boldsymbol{\Lambda}$ are both symmetric. Although we have introduced $\bar{\mathbf{J}}$ in the context of impulsive dynamics, it is more typically used in normal (i.e., nonimpulsive) dynamics equations.

2.4 Dynamic Models of Rigid-Body Systems

A basic rigid-body model of a robot mechanism has four components: a connectivity graph, link and joint geometry parameters, link inertia parameters, and a set of joint models. To this model, one can add various force-producing elements, such as springs, dampers, joint friction, actuators, and drives. The actuators and drives, in particular, may have quite elaborate dynamic models of their own. It is also possible to add extra motion freedoms to model elasticity in the joint bearings or links (Chap. 13). This section describes a basic model. More on this topic can be found in books such as [2.3, 8, 22].

2.4.1 Connectivity

A connectivity graph is an undirected graph in which each node represents a rigid body and each arc represents a joint. The graph must be connected; and exactly one node represents a fixed base or reference frame. If the graph represents a mobile robot (i.e., a robot that is not connected to a fixed base), then it is necessary to introduce a fictitious 6-DOF joint between the fixed base and any one body in the mobile robot. The chosen body is then known as a *floating base*. If a single graph is to represent a collection of mobile robots, then each

robot has its own floating base, and each floating base has its own 6-DOF joint. Note that a 6-DOF joint imposes no constraints on the two bodies it connects, so the introduction of a 6-DOF joint alters the connectivity of the graph without altering the physical properties of the system it represents.

In graph-theory terminology, a loop is an arc that connects a node to itself, and a cycle is a closed path that does not traverse any arc more than once. In the connectivity graph of a robot mechanism, loops are not allowed, and cycles are called kinematic loops. A mechanism that contains kinematic loops is called a closed-loop mechanism; and a mechanism that does not is called an open-loop mechanism or a kinematic tree. Every closed-loop mechanism has a spanning tree, which defines an open-loop mechanism, and every joint that is not in the spanning tree is called a loop-closing joint. The joints in the tree are called tree joints.

The fixed base serves as the root node of a kinematic tree, and the root node of any spanning tree on a closed-loop mechanism. A kinematic tree is said to be branched if at least one node has at least two children, and unbranched otherwise. An unbranched kinematic tree is also called a kinematic chain, and a branched tree can be called a branched kinematic chain. A typical industrial robot arm, without a gripper, is a kinematic chain, while a typical humanoid robot is a kinematic tree with a floating base.

In a system containing N_B moving bodies and N_J joints, where N_J includes the 6-DOF joints mentioned above, the bodies and joints are numbered as follows. First, the fixed base is numbered body 0. The other bodies are then numbered from 1 to N_B in any order such that each body has a higher number than its parent. If the system contains kinematic loops then one must first choose a spanning tree, and commit to that choice, since the identity of a body's parent is determined by the spanning tree. This style of numbering is called a *regular numbering scheme*.

Having numbered the bodies, we number the tree joints from 1 to N_B such that joint i connects body i to its parent. The loop-closing joints, if any, are then numbered from $N_B + 1$ to N_J in any order. Each loop-closing joint k closes one independent kinematic loop, and we number the loops from 1 to N_L (where $N_L = N_J - N_B$ is the number of independent loops) such that loop l is the one closed by joint $k = N_B + l$. Kinematic loop l is the unique cycle in the graph that traverses joint k , but does not traverse any other loop-closing joint.

For an unbranched kinematic tree, these rules produce a unique numbering in which the bodies are

numbered consecutively from base to tip, and the joints are numbered such that joint i connects bodies i and $i - 1$. In all other cases, regular numberings are not unique.

Although the connectivity graph is undirected, it is necessary to assign a direction to each joint for the purpose of defining joint velocity and force. This is necessary for both tree joints and loop-closing joints. Specifically, a joint is said to connect from one body to another. We may call them the predecessor $p(i)$ and successor $s(i)$ for joint i , respectively. Joint velocity is then defined as the velocity of the successor relative to the predecessor; and joint force is defined as a force acting on the successor. It is standard practice (but not a necessity) for all tree joints to connect from the parent to the child.

The connectivity of a kinematic tree, or the spanning tree on a closed-loop mechanism, is described by an N_B -element array of parent body numbers, where the i -th element $p(i)$ is the parent of body i . Note that the parent

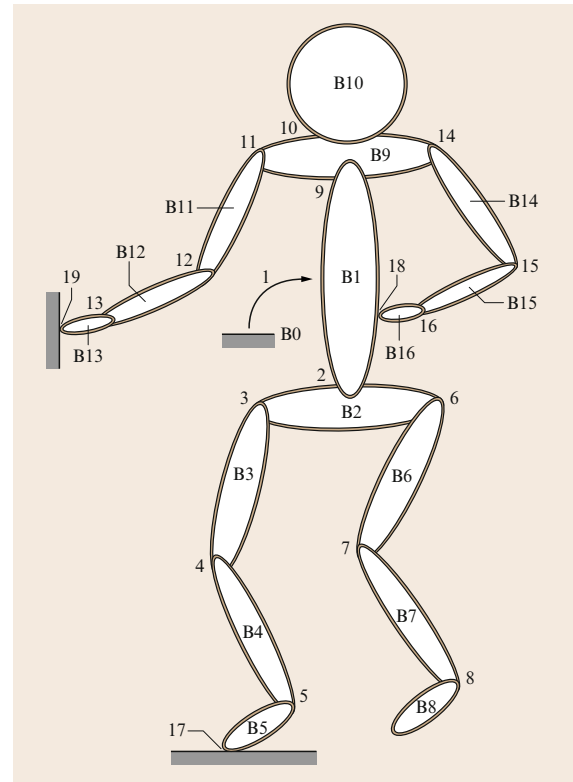


Fig. 2.3 Humanoid robot example. Note: to distinguish between body numbers and joint numbers in this figure, body numbers are preceded by a 'B' for clarity

Table 2.2 Loop-closing joints and roots of loops for the humanoid example

Loop l	Loop-closing joint k	$p(k)$	$s(k)$	Root
1	17	0	5	0
2	18	16	1	1
3	19	0	13	0

$p(i)$ for body i is also the predecessor $p(i)$ for joint i , and thus the common notation. Many algorithms rely on the property $p(i) < i$ to perform their calculations in the correct order. The set of all body numbers for the children of body i , $c(i)$, is also useful in many recursive algorithms.

The connectivity data for kinematic loops may be described in a variety of ways. A representation that facilitates use in recursive algorithms includes the following conventions. Loop-closing joint k joins bodies $p(k)$ (the predecessor) and $s(k)$ (the successor). The set $LR(i)$ for body i gives the numbers of the loops for which body i is the root. Using the property $p(i) < i$ for bodies in the spanning tree, the root of a loop is chosen as the body with the lowest number. In addition, the set $LB(i)$ for body i gives the numbers of the loops to which body i belongs but is not the root.

An example of a closed-loop system is given in Fig. 2.3. The system consists of a humanoid mechanism with topologically varying contacts, with the environment and within the mechanism, which form closed loops. The system has $N_B = 16$ moving bodies and $N_J = 19$ joints with $N_L = N_J - N_B = 3$ loops. The main body (1) is considered to be a *floating base* for this mobile robot system. It is connected to the fixed base (0) through a fictitious 6-DOF joint (1). To complete the example, the loop-closing joint and the body numbers $p(k)$ and $s(k)$ as well as the root body for each loop are given in Table 2.2. The body-based sets $c(i)$ and $LB(i)$ are given in Table 2.3. Note that $LR(0) = \{1, 3\}$ and $LR(1) = \{2\}$ and all other LR sets are null for this example.

2.4.2 Link Geometry

When two bodies are connected by a joint, a complete description of the connection consists of a description of the joint itself, and the locations of two coordinate frames, one in each body, which specify where in each body the joint is located. If there are N_J joints in the system, then there are a total of $2N_J$ joint-attachment frames. One half of these frames are identified with the

Table 2.3 Body-based sets for the humanoid example

Body i	$c(i)$	$LB(i)$	Body i	$c(i)$	$LB(i)$
0	1				
1	2, 9	1, 3	9	10, 11, 14	2, 3
2	3, 6	1	10		
3	4	1	11	12	3
4	5	1	12	13	3
5		1	13		3
6	7		14	15	2
7	8		15	16	2
8			16		2

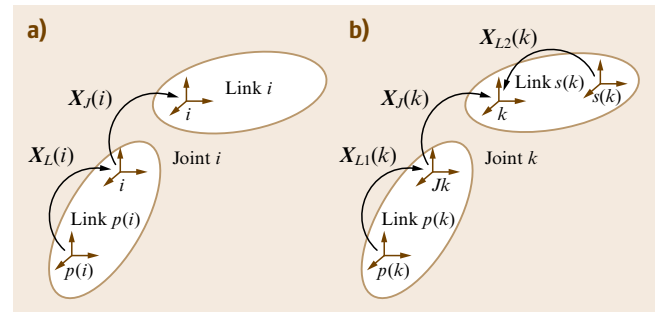
numbers 1 to N_J , and the remainder with the labels $J1$ to JN_J . Each joint i connects from frame Ji to frame i . For joints 1 to N_B (i.e., the tree joints), frame i is rigidly attached to body i . For joints $N_B + 1$ to N_J , frame k for loop-closing joint k will be rigidly attached to body $s(k)$. The second coordinate frame Ji is attached to the predecessor $p(i)$ for each joint i , whether it is a tree joint or a loop-closing joint. Coordinate frame Ji provides a base frame for joint i in that the joint rotation and/or translation is defined relative to this frame.

Figure 2.4 shows the coordinate frames and transforms associated with each joint in the system. The overall transform from frame $p(i)$ coordinates to frame i coordinates for a tree joint is given by:

$${}^iX_{p(i)} = {}^iX_{Ji} {}^{Ji}X_{p(i)} = X_J(i)X_L(i). \quad (2.61)$$

The transform $X_L(i)$ is a fixed link transform which sets the base frame Ji of joint i relative to $p(i)$. It may be used to transform spatial motion vectors from $p(i)$ to Ji coordinates. The transform $X_J(i)$ is a variable joint transform which completes the transformation across joint i from Ji to i coordinates.

Similarly, the overall transform from frame $p(k)$ coordinates to frame k coordinates for a loop-closing joint


Fig. 2.4a,b Coordinate frames and transforms associated with a tree joint (a) and a loop-closing joint (b)

is given by:

$${}^kX_{p(k)} = {}^kX_{Jk} {}^JX_{p(k)} = X_J(k) X_{L1}(k). \quad (2.62)$$

An additional transform $X_{L2}(k)$ is defined from frame $s(k)$ coordinates to frame k coordinates and is given by:

$$X_{L2}(k) = {}^kX_{s(k)}. \quad (2.63)$$

Link and joint geometry data can be specified in a variety of different ways. The most common method is to use Denavit–Hartenberg parameters [2.23]. However, standard Denavit–Hartenberg parameters are not completely general, and are insufficient for describing the geometry for a branched kinematic tree, or for a mechanism containing certain kinds of multi-DOF joints. A modified form of Denavit–Hartenberg parameters [2.2] is used for single-DOF joints in this Handbook (Sect. 1.4). The parameters have been extended for branched kinematic trees [2.22] and closed-loop mechanisms.

2.4.3 Link Inertias

The link inertia data consists of the masses, positions of centers of mass, and rotational inertias of each link in the mechanism. The inertia parameters for link i are expressed in coordinate frame i , and are therefore constants.

2.4.4 Joint Models

The relationship between connected links is described using the general joint model of *Roberson* and *Schwertassek* [2.3]. For a kinematic tree or spanning tree on a closed-loop mechanism, an $n_i \times 1$ vector, $\dot{\mathbf{q}}_i$, relates the velocity of link i to the velocity of its parent, link $p(i)$, where n_i is the number of degrees of freedom at the joint connecting the two links. For a loop-closing joint in a closed-loop mechanism, the relationship is between the velocity of link $s(i)$ (the successor) and the velocity of link $p(i)$ (the predecessor). In either case, the relationship is between the velocity of coordinate frames i and Ji .

Let $\mathbf{v}_{\text{rel}}$ and $\mathbf{a}_{\text{rel}}$ denote the velocity and acceleration across joint i , that is, the velocity and acceleration of link $s(i)$ relative to $p(i)$. The free modes of the joint are represented by the $6 \times n_i$ matrix Φ_i , such that $\mathbf{v}_{\text{rel}}$ and $\mathbf{a}_{\text{rel}}$ are given as follows:

$$\mathbf{v}_{\text{rel}} = \Phi_i \dot{\mathbf{q}}_i \quad (2.64)$$

and

$$\mathbf{a}_{\text{rel}} = \Phi_i \ddot{\mathbf{q}}_i + \dot{\Phi}_i \dot{\mathbf{q}}_i, \quad (2.65)$$

where Φ_i and $\dot{\Phi}_i$ depend on the type of joint [2.3]. The matrix Φ_i has full column rank, so we can define a complementary matrix, Φ_i^c , such that the 6×6 matrix $(\Phi_i \Phi_i^c)$ is invertible. We can regard the columns of this matrix as forming a basis on $\mathbb{M}^6$ such that the first n_i basis vectors define the directions in which motion is allowed, and the remaining $6 - n_i = n_i^c$ vectors define directions in which motion is not allowed. Thus, Φ_i^c represents the constrained modes of joint i .

The force transmitted across joint i from its predecessor to its successor, $\mathbf{f}_i$, is given as follows:

$$\mathbf{f}_i = (\Psi_i \Psi_i^c) \begin{pmatrix} \boldsymbol{\tau}_i \\ \boldsymbol{\lambda}_i \end{pmatrix}, \quad (2.66)$$

where $\boldsymbol{\tau}_i$ is the $n_i \times 1$ vector of applied forces along the free modes, $\boldsymbol{\lambda}_i$ is the $(6 - n_i) \times 1$ vector of constraint forces, and Ψ_i and Ψ_i^c are computed as follows:

$$(\Psi_i \Psi_i^c) = (\Phi_i \Phi_i^c)^{-T}. \quad (2.67)$$

For most common joint types, it is possible to choose Φ_i and Φ_i^c such that the matrix $(\Phi_i \Phi_i^c)$ is numerically orthonormal, so that $(\Psi_i \Psi_i^c)$ is numerically equal to $(\Phi_i \Phi_i^c)$. Note that (2.67) implies the following relationships: $(\Psi_i)^T \Phi_i = \mathbf{1}_{n_i \times n_i}$, $(\Psi_i)^T \Phi_i^c = \mathbf{0}_{n_i \times (6 - n_i)}$, $(\Psi_i^c)^T \Phi_i = \mathbf{0}_{(6 - n_i) \times n_i}$, and $(\Psi_i^c)^T \Phi_i^c = \mathbf{1}_{(6 - n_i) \times (6 - n_i)}$. When applied to (2.66), the following useful relationship results:

$$\boldsymbol{\tau}_i = \Phi_i^T \mathbf{f}_i. \quad (2.68)$$

The value of $\dot{\Phi}_i$ in (2.65) depends on the type of joint. The general formula is

$$\dot{\Phi}_i = \dot{\Phi}_i + \mathbf{v}_i \times \Phi_i, \quad (2.69)$$

where $\mathbf{v}_i$ is the velocity of link i , and $\dot{\Phi}_i$ is the apparent derivative of Φ_i , as seen by an observer moving with link i , and is given by

$$\dot{\Phi}_i = \frac{\partial \Phi_i}{\partial \mathbf{q}_i} \dot{\mathbf{q}}_i. \quad (2.70)$$

For most common joint types, $\dot{\Phi}_i = \mathbf{0}$.

Single-DOF joints ($n_i = 1$) are especially straightforward to work with when using the Denavit–Hartenberg convention. Motion is chosen along (prismatic) or about (revolute) the $\hat{\mathbf{z}}_i$ coordinate axis. In this case, $\Phi_i = (000001)^T$ for a prismatic joint and $\Phi_i = (001000)^T$ for a revolute joint. Also, $\Phi_i = \mathbf{0}$.

The fictitious 6-DOF joint for a floating base for a mobile robot is also handled relatively easily. For this case, $\Phi_i = \mathbf{1}$ (6×6 identity matrix) and $\dot{\Phi}_i = \mathbf{0}$.

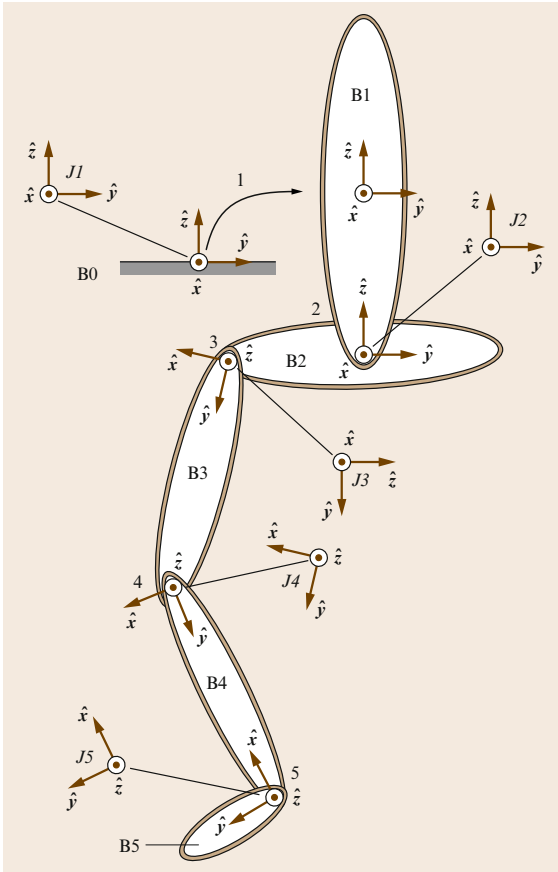


Fig. 2.5 Coordinate frames for the first five links and joints of the humanoid robot example

The revolute joint and floating-base joint, as well as the universal joint ($n_i = 2$) and spherical joint ($n_i = 3$) are illustrated in the example in the next section. For additional details on joint kinematics, see Sect. 1.3.

2.4.5 Example System

In order to illustrate the conventions used for the link and joint models, coordinate frames are attached to the first five links (bodies) and fixed base of the humanoid robot as shown in Fig. 2.5. Note that frame J_i is attached to link $p(i) = i - 1$ for each of the five joints. For this example, the origin of frame J_1 is set coincident with the origin of frame 0, and the origins of frames J_2 , J_3 , J_4 , and J_5 are coincident with the origins of frames 2, 3, 4, and 5, respectively. Note that J_1 could be set at any position/orientation on the fixed base (B0) to permit the most convenient representation for the motion of the

Table 2.4 Number of degrees of freedom (n_i), fixed rotation (${}^J R_{p(i)}$) and position (${}^{p(i)} p_{Ji}$) from frame $p(i)$ to base frame J_i for joint i of the example system. Note that $2l_i$ is the nominal length of link i along its long axis

Joint	n_i	${}^J R_{p(i)}$	${}^{p(i)} p_{Ji}$
1	6	$\mathbf{1}_{3 \times 3}$	$\mathbf{0}_{3 \times 1}$
2	1	$\mathbf{1}_{3 \times 3}$	$\begin{pmatrix} 0 \\ 0 \\ -l_1 \end{pmatrix}$
3	3	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix}$	$\begin{pmatrix} 0 \\ -l_2 \\ 0 \end{pmatrix}$
4	1	$\mathbf{1}_{3 \times 3}$	$\begin{pmatrix} 0 \\ 2l_3 \\ 0 \end{pmatrix}$
5	2	$\begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 2l_4 \\ 0 \end{pmatrix}$

floating base (B1) relative to the fixed base. Also, the origin of J_2 could be set anywhere along $\hat{z}_2$.

The number of degrees of freedom, and fixed rotation and position of the base frames J_i for each joint of the example system, are given in Table 2.4. The rotation ${}^J R_{p(i)}$ transforms 3-D vectors in $p(i)$ coordinates to J_i coordinates. The position ${}^{p(i)} p_{Ji}$ is the vector giving the position of the origin O_{Ji} relative to $O_{p(i)}$, expressed in $p(i)$ coordinates. The spatial transform $\mathbf{X}_{L(i)} = {}^J X_{p(i)}$ may be composed from these 3-D quantities through the equation for ${}^B X_A$ in Table 2.1. The humanoid has a floating base, the torso, a revolute joint between the torso and pelvis (about $\hat{z}_2$), a spherical joint at the hip, a revolute joint at the knee, and a universal joint at the ankle. As shown in Fig. 2.5, the leg is slightly bent and the foot is turned out to the side ($\approx 90^\circ$ rotation about $\hat{y}_3$ at the hip).

The free modes, velocity variables, and position variables for all of the joint types in the humanoid are given in Tables 1.5 and 1.6. The expressions for ${}^j R_i$ and ${}^j p_i$ in these tables give ${}^i R_{Ji}^T$ and ${}^J p_i$, respectively, through which the joint transform $\mathbf{X}_{J(i)} = {}^i X_{Ji}$ may be composed. Revolute joints follow the Denavit–Hartenberg convention with rotation about the $\hat{z}_i$ axis. The ankle has a pitch rotation of α_5 about the $\hat{z}_{J5}$ axis followed by a roll rotation of β_5 about the $\hat{y}_5$ axis (see the Z–Y–X Euler angle definitions in Table 1.1). The hip is modeled as a ball-and-socket, spherical joint. To avoid the singularities that are associated with Euler angles, the quaternion ϵ_i may be used

to represent the orientation at the hip. The relationship between the quaternion rate $\dot{\epsilon}_i$ and the relative rotation rate $\omega_{i \text{ rel}}$ is given in (1.7) of the Handbook.

The floating base uses the position of the torso 0p_1 and quaternion ϵ_1 for its position and orientation state variables, respectively. The position of the torso may

be computed by integrating the velocity for the link, as expressed in fixed base coordinates: ${}^0v_1 = {}^0R_1 v_1$, where v_1 is the velocity of the torso in moving coordinates.

Note that $\Phi_i = 0$ for all joints except the universal joint. Since the components of $\hat{z}_{J5}$ in link 5 coordinates vary with $\beta_5, \dot{\beta}_5 \neq 0$. See Sect. 1.3 of the Handbook for further details of the joint kinematics.

2.5 Kinematic Trees

The dynamics of a kinematic tree is simpler, and easier to calculate, than the dynamics of a closed-loop mechanism. Indeed, many algorithms for closed-loop mechanisms work by first calculating the dynamics of a spanning tree, and then subjecting it to the loop-closure constraints.

This section describes the following dynamics algorithms for kinematic trees: the recursive Newton–Euler algorithm (RNEA) for inverse dynamics, the articulated-body algorithm (ABA) for forward dynamics, the composite-rigid-body algorithm (CRBA) for calculating the joint-space inertia matrix (JSIM), and two algorithms to calculate the operational-space inertia matrix (OSIM).

2.5.1 The Recursive Newton–Euler Algorithm

This is an $O(n)$ algorithm for calculating the inverse dynamics of a fixed-base kinematic tree, and is based on the very efficient RNEA of Luh et al. [2.4]. A floating-base version can be found in [2.8, 15]. Given the joint position and velocity variables, this algorithm calculates the applied joint torque/force variables required to produce a given set of joint accelerations.

The link velocities and accelerations are first computed through an outward recursion from the fixed base to the leaf links of the tree. The resultant forces on each link are computed using the Newton–Euler equations (2.35) during this recursion. A second, inward recursion uses the force balance equations at each link to compute the spatial force across each joint and the value of each joint torque/force variable. The key step for computational efficiency is to refer most quantities to local link coordinates. Also, the effects of gravity on each link are efficiently included in the equations by accelerating the base of the mechanism upward.

The calculation proceeds in four steps, as follows, with two steps in each of the two recursions.

Step 1

Calculate the velocity and acceleration of each link in turn, starting with the known velocity and acceleration of the fixed base, and working towards the tips (i. e., the leaf nodes in the connectivity graph).

The velocity of each link in a kinematic tree is given by the recursive formula

$$\mathbf{v}_i = \mathbf{v}_{p(i)} + \Phi_i \dot{\mathbf{q}}_i, \quad (\mathbf{v}_0 = \mathbf{0}), \quad (2.71)$$

where $\mathbf{v}_i$ is the velocity of link i , Φ_i is the motion matrix of joint i , and $\dot{\mathbf{q}}_i$ is the vector of joint velocity variables for joint i .

The equivalent formula for accelerations is obtained by differentiating (2.71), giving

$$\mathbf{a}_i = \mathbf{a}_{p(i)} + \Phi_i \ddot{\mathbf{q}}_i + \dot{\Phi}_i \dot{\mathbf{q}}_i, \quad (\mathbf{a}_0 = \mathbf{0}) \quad (2.72)$$

where $\mathbf{a}_i$ is the acceleration of link i , and $\ddot{\mathbf{q}}_i$ is the vector of joint acceleration variables.

The effect of a uniform gravitational field on the mechanism can be simulated by initializing $\mathbf{a}_0$ to $-\mathbf{a}_g$ instead of zero, where $\mathbf{a}_g$ is the gravitational acceleration vector. In this case, $\mathbf{a}_i$ is not the true acceleration of link i , but the sum of its true acceleration and $-\mathbf{a}_g$.

Step 2

Calculate the equation of motion for each link. This step computes the forces required to cause the accelerations calculated in step 1. The equation of motion for link i is

$$\mathbf{f}_i^a = \mathbf{I}_i \mathbf{a}_i + \mathbf{v}_i \times \mathbf{I}_i \mathbf{v}_i, \quad (2.73)$$

where $\mathbf{I}_i$ is the spatial inertia of link i , and $\mathbf{f}_i^a$ is the net force acting on link i .

Step 3

Calculate the spatial force across each joint. Referring to Fig. 2.6, the net force acting on link i is

$$\mathbf{f}_i^a = \mathbf{f}_i^e + \mathbf{f}_i - \sum_{j \in c(i)} \mathbf{f}_j,$$

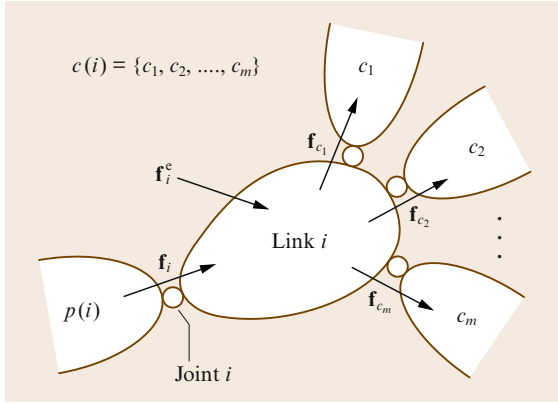


Fig. 2.6 Forces acting on link i

where $\mathbf{f}_i$ is the force transmitted across joint i , $\mathbf{f}_i^e$ is the sum of all relevant external forces acting on link i , and $c(i)$ is the set of children of link i . Rearranging this equation gives the following recursive formula for calculating the joint forces:

$$\mathbf{f}_i = \mathbf{f}_i^a - \mathbf{f}_i^e + \sum_{j \in c(i)} \mathbf{f}_j, \quad (2.74)$$

where i iterates from N_B to 1.

$\mathbf{f}_i^e$ may include contributions from springs, dampers, force fields, contact with the environment, and so on, but its value is assumed to be known, or at least to be calculable from known quantities. If gravity has not been simulated by a fictitious base acceleration, then

Table 2.5 Coordinate-free recursive Newton–Euler algorithm (RNEA) for inverse dynamics

```

v0 = 0
a0 = -ag
for i = 1 to NB do
    vi = vp(i) + Φi q̇i
    ai = ap(i) + Φi q̈i + Φ̇i q̇i
    fi = Ii ai + vi × Ii vi - fie
end
for i = NB to 1 do
    τi = ΦiT fi
    if p(i) ≠ 0 then
        fp(i) = fp(i) + fi
    end
end
end

```

the gravitational force acting on link i must be included in $\mathbf{f}_i^e$.

Step 4

Calculate the joint force variables, τ_i . By definition, they are given by the equation

$$\tau_i = \Phi_i^T \mathbf{f}_i. \quad (2.75)$$

Coordinate-Free Algorithm

Equations (2.71)–(2.75) imply the algorithm shown in Table 2.5, which is the coordinate-free version of the RNEA. This is the simplest form of the algorithm, and it is suitable for mathematical analysis and related purposes. However, it is not suitable for numerical computation because a numerical version must use coordinate vectors.

Link-Coordinates Algorithm

In general, we say that an algorithm is implemented in link coordinates if a coordinate system is defined for each link, and the calculations pertaining to link i are performed in the coordinate system associated with link i . The alternative is to implement the algorithm in absolute coordinates, in which case all calculations are performed in a single coordinate system, typically that of the base link. In practice, the RNEA is more computationally efficient when implemented in link coordinates, and the same is true of most other dynamics algorithms.

To convert the RNEA to link coordinates, we first examine the equations to see which ones involve quantities from more than one link. Equations (2.73) and (2.75) each involve quantities pertaining to link i only, and therefore need no modification. Such equations are said to be local to link i . The remaining equations involve quantities from more than one link, and therefore require the insertion of coordinate transformation matrices. The modified versions of (2.71), (2.72), and (2.74) are

$$\mathbf{v}_i = {}^i X_{p(i)} \mathbf{v}_{p(i)} + \Phi_i \dot{\mathbf{q}}_i, \quad (2.76)$$

$$\mathbf{a}_i = {}^i X_{p(i)} \mathbf{a}_{p(i)} + \Phi_i \ddot{\mathbf{q}}_i + \dot{\Phi}_i \dot{\mathbf{q}}_i, \quad (2.77)$$

and

$$\mathbf{f}_i = \mathbf{f}_i^a - {}^i X_0^{F0} \mathbf{f}_i^e + \sum_{j \in c(i)} {}^i X_j^F \mathbf{f}_j. \quad (2.78)$$

Equation (2.78) assumes that external forces are expressed in absolute (i. e., link 0) coordinates.

The complete algorithm is shown in Table 2.6. The function `jtype` returns the type code for joint i ; the

Table 2.6 Recursive Newton–Euler algorithm using spatial vectors

```

inputs:  $\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}, model, {}^0\mathbf{f}_i^e$ 
output:  $\boldsymbol{\tau}$ 
model data:  $N_B, jtype(i), p(i), X_L(i), I_i$ 

 $\mathbf{v}_0 = \mathbf{0}$ 
 $\mathbf{a}_0 = -\mathbf{a}_g$ 
for  $i = 1$  to  $N_B$  do
     $X_J(i) = xjcalc(jtype(i), q_i)$ 
     ${}^iX_{p(i)} = X_J(i) X_L(i)$ 
    if  $p(i) \neq 0$  then
         ${}^iX_0 = {}^iX_{p(i)} {}^{p(i)}X_0$ 
    end
     $\Phi_i = pcalc(jtype(i), q_i)$ 
     $\Phi_{c_i} = pdcalc(jtype(i), q_i, \dot{q}_i)$ 
     $\mathbf{v}_i = {}^iX_{p(i)} \mathbf{v}_{p(i)} + \Phi_i \dot{q}_i$ 
     $\zeta_i = \Phi_{c_i} \dot{q}_i + \mathbf{v}_i \times \Phi_i \dot{q}_i$ 
     $\mathbf{a}_i = {}^iX_{p(i)} \mathbf{a}_{p(i)} + \Phi_i \ddot{q}_i + \zeta_i$ 
     $\mathbf{f}_i = I_i \mathbf{a}_i + \mathbf{v}_i \times I_i \mathbf{v}_i - {}^iX_0^{-T} {}^0\mathbf{f}_i^e$ 
end
for  $i = N_B$  to  $1$  do
     $\boldsymbol{\tau}_i = \Phi_i^T \mathbf{f}_i$ 
    if  $p(i) \neq 0$  then
         $\mathbf{f}_{p(i)} = \mathbf{f}_{p(i)} + {}^iX_{p(i)}^T \mathbf{f}_i$ 
    end
end

```

function `xjcalc` calculates the joint transformation matrix for the specified type of joint; and the functions `pcalc` and `pdcalc` calculate Φ_i and Φ_{c_i} . The formulae used by these functions for a variety of joint types can be found in Tables 1.5 and 1.6. In the general case, both `pcalc` and `pdcalc` are needed. However, for most common joint types, Φ_i is a known constant in link coordinates, and Φ_{c_i} is therefore zero. If it is known in advance that all joints will have this property, then the algorithm can be simplified accordingly. The quantities I_i and $X_L(i)$ are known constants in link coordinates, and are part of the data structure describing the robot mechanism.

The last assignment in the first loop initializes each $\mathbf{f}_i$ to the expression $\mathbf{f}_i^a - {}^iX_0^T {}^0\mathbf{f}_i^e$ (using the identity ${}^iX_0^F = {}^iX_0^{-T}$). The summation on the right-hand side of (2.78) is then performed in the second loop. This al-

Table 2.7 Recursive Newton–Euler algorithm in 3-D vectors, for revolute joints only

```

inputs:  $\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}, model$ 
output:  $\boldsymbol{\tau}$ 
model data:  $N_B, p(i), \mathbf{R}_L(i), {}^{p(i)}\mathbf{p}_i, m_i, \mathbf{c}_i, \bar{I}_i^{\text{cm}}$ 

 $\boldsymbol{\omega}_0 = \mathbf{0}$ 
 $\dot{\boldsymbol{\omega}}_0 = \mathbf{0}$ 
 $\dot{\mathbf{v}}_0 = -\dot{\mathbf{v}}_g$ 
for  $i = 1$  to  $N_B$  do
     ${}^i\mathbf{R}_{p(i)} = \text{rotz}(q_i) \mathbf{R}_L(i)$ 
     $\boldsymbol{\omega}_i = {}^i\mathbf{R}_{p(i)} \boldsymbol{\omega}_{p(i)} + \hat{\mathbf{z}}_i \dot{q}_i$ 
     $\dot{\boldsymbol{\omega}}_i = {}^i\mathbf{R}_{p(i)} \dot{\boldsymbol{\omega}}_{p(i)} + ({}^i\mathbf{R}_{p(i)} \boldsymbol{\omega}_{p(i)}) \times \hat{\mathbf{z}}_i \dot{q}_i + \hat{\mathbf{z}}_i \ddot{q}_i$ 
     $\dot{\mathbf{v}}_i = {}^i\mathbf{R}_{p(i)} (\dot{\mathbf{v}}_{p(i)} + \dot{\boldsymbol{\omega}}_{p(i)} \times {}^{p(i)}\mathbf{p}_i$ 
         $\quad + \boldsymbol{\omega}_{p(i)} \times \boldsymbol{\omega}_{p(i)} \times {}^{p(i)}\mathbf{p}_i)$ 
     $\mathbf{f}_i = m_i (\dot{\mathbf{v}}_i + \dot{\boldsymbol{\omega}}_i \times \mathbf{c}_i + \boldsymbol{\omega}_i \times \boldsymbol{\omega}_i \times \mathbf{c}_i)$ 
     $\mathbf{n}_i = \bar{I}_i^{\text{cm}} \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times \bar{I}_i^{\text{cm}} \boldsymbol{\omega}_i + \mathbf{c}_i \times \mathbf{f}_i$ 
end
for  $i = N_B$  to  $1$  do
     $\boldsymbol{\tau}_i = \hat{\mathbf{z}}_i^T \mathbf{n}_i$ 
    if  $p(i) \neq 0$  then
         $\mathbf{f}_{p(i)} = \mathbf{f}_{p(i)} + {}^i\mathbf{R}_{p(i)}^T \mathbf{f}_i$ 
         $\mathbf{n}_{p(i)} = \mathbf{n}_{p(i)} + {}^i\mathbf{R}_{p(i)}^T \mathbf{n}_i + {}^{p(i)}\mathbf{p}_i \times {}^i\mathbf{R}_{p(i)}^T \mathbf{f}_i$ 
    end
end

```

gorithm includes code to calculate iX_0 , which is used to transform the external forces to link coordinates. If there are no external forces, then this code can be omitted. If there is only a single external force (e.g., a force at the end-effector of a robot arm) then this code can be replaced with code that transforms the external force vector successively from one link coordinate system to the next, using ${}^iX_{p(i)}$.

Note: although the phrase ‘link coordinates’ suggests that we are using moving coordinate frames, the algorithm is in fact implemented in stationary coordinates that happen to coincide with the moving coordinates at the current instant.

3-D Vector RNEA

The original version of the RNEA was developed and expressed using 3-D vectors (e.g. [2.2,4]). Table 2.7 shows a special case of this algorithm, in which the joints are assumed to be revolute, and the joint axes are assumed

to coincide with the z axes of the link coordinate systems. (Without these assumptions, the equations would be a lot longer.) It also assumes that the external forces are zero.

In this algorithm, $\dot{\mathbf{v}}'_g$ is the linear acceleration due to gravity, expressed in base (link 0) coordinates; `rotz` computes a rotation matrix representing a rotation of the coordinate frame about the z axis; $\mathbf{R}_L(i)$ is the rotational component of $\mathbf{X}_L(i)$; $\mathbf{R}_{p(i)}$ is the rotational component of $\mathbf{X}_{p(i)}$; `pcalc` and `pdcalc` are not used because Φ_i is the known constant $(\hat{\mathbf{z}}^T \mathbf{0}^T)^T$; $\dot{\mathbf{v}}'_i$ is the linear acceleration of the origin of link i coordinates (O_i), and is the linear component of the classical acceleration of link i ; ${}^{p(i)}\mathbf{p}_i$ is the position of O_i relative to $O_{p(i)}$ expressed in $p(i)$ coordinates; and m_i , $\mathbf{c}_i$, and $\mathbf{I}_i^{\text{cm}}$ are the inertia parameters of link i . (See Table 2.1 for the equations relating these 3-D quantities to the corresponding spatial quantities.)

At first sight, the 3-D vector algorithm looks significantly different from the spatial vector algorithm. Nevertheless, it can be obtained directly from the spatial vector algorithm simply by expanding the spatial vectors to their 3-D components, restricting the joint type to revolute, converting spatial accelerations to classical accelerations (i.e., replacing each instance of $\dot{\mathbf{v}}_i$ with $\dot{\mathbf{v}}'_i - \boldsymbol{\omega}_i \times \mathbf{v}_i$ as per (2.22)), and applying some 3-D vector identities to bring the equations into the form shown in the table. The conversion from spatial to classical acceleration has one interesting side-effect: $\mathbf{v}_i$ cancels out of the equation of motion, and therefore does not need to be calculated. As a result, the 3-D version of the algorithm has a slight speed advantage over the spatial version.

2.5.2 The Articulated-Body Algorithm

The **ABA** is an $O(N_B)$ algorithm for calculating the forward dynamics of a kinematic tree. However, under normal circumstances, $O(N_B) = O(n)$, so we shall refer to it as an $O(n)$ algorithm. The **ABA** was developed by Featherstone [2.1] and is an example of a constraint-propagation algorithm. Given the joint position, velocity, and applied torque/force variables, this algorithm calculates the joint accelerations. With the joint accelerations determined, numerical integration may be used to provide a simulation of the mechanism.

The key concept in the **ABA** is illustrated in Fig. 2.7. The subtree rooted at link i interacts with the rest of the kinematic tree only through a force $\mathbf{f}_i$ that is transmitted across joint i . Suppose we break the tree at this point, and consider only the motion of the subtree subject to an

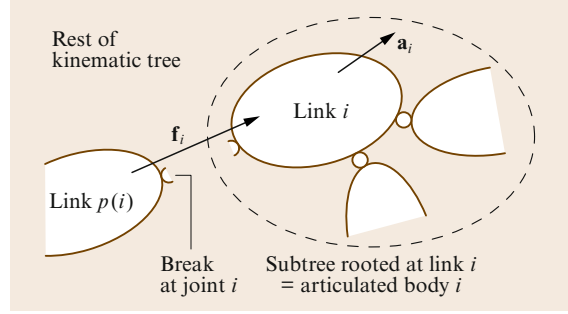


Fig. 2.7 Definition of articulated body i

unknown force, $\mathbf{f}_i$, acting on link i . It is possible to show that the acceleration of link i is related to the applied force according to the equation

$$\mathbf{f}_i = \mathbf{I}_i^A \mathbf{a}_i + \mathbf{p}_i^A, \quad (2.79)$$

where $\mathbf{I}_i^A$ is called the articulated-body inertia of link i in the subtree (which we can now call an articulated body), and $\mathbf{p}_i^A$ is the associated bias force, which is the force required to produce zero acceleration in link i . Note that $\mathbf{p}_i^A$ depends on the velocities of the individual bodies in the articulated body. Equation (2.79) takes into account the complete dynamics of the subtree. Thus, if we happened to know the correct value of $\mathbf{f}_i$, then (2.79) would immediately give us the correct acceleration of link i .

The reason we are interested in the quantities $\mathbf{I}_i^A$ and $\mathbf{p}_i^A$ is that they allow us to calculate $\ddot{\mathbf{q}}_i$ from $\mathbf{a}_{p(i)}$, which in turn allows us to calculate $\mathbf{a}_i$, which then allows us to calculate more joint accelerations, and so on. Combining (2.79) with (2.75) and (2.72) gives

$$\boldsymbol{\tau}_i = \Phi_i^T \mathbf{f}_i = \Phi_i^T (\mathbf{I}_i^A (\mathbf{a}_{p(i)} + \Phi_i \ddot{\mathbf{q}}_i + \dot{\Phi}_i \dot{\mathbf{q}}_i) + \mathbf{p}_i^A),$$

which can be solved for $\ddot{\mathbf{q}}_i$ to give

$$\ddot{\mathbf{q}}_i = \mathbf{D}_i (\mathbf{u}_i - \mathbf{U}_i^T \mathbf{a}_{p(i)}), \quad (2.80)$$

where

$$\begin{aligned} \mathbf{U}_i &= \mathbf{I}_i^A \Phi_i, \\ \mathbf{D}_i &= (\Phi_i^T \mathbf{U}_i)^{-1} = (\Phi_i^T \mathbf{I}_i^A \Phi_i)^{-1}, \\ \mathbf{u}_i &= \boldsymbol{\tau}_i - \mathbf{U}_i^T \zeta_i - \Phi_i^T \mathbf{p}_i^A \end{aligned}$$

and

$$\zeta_i = \dot{\Phi}_i \dot{\mathbf{q}}_i = \dot{\Phi}_i \dot{\mathbf{q}}_i + \mathbf{v}_i \times \Phi_i \dot{\mathbf{q}}_i.$$

$\mathbf{a}_i$ can then be calculated via (2.72).

It turns out that the articulated-body inertias and bias forces can be calculated efficiently via the recursive formulae

$$\mathbf{I}_i^A = \mathbf{I}_i + \sum_{j \in c(i)} (\mathbf{I}_j^A - \mathbf{U}_j \mathbf{D}_j \mathbf{U}_j^T) \quad (2.81)$$

and

$$\mathbf{p}_i^A = \mathbf{p}_i + \sum_{j \in c(i)} (\mathbf{p}_j^A + \mathbf{I}_j^A \boldsymbol{\zeta}_j + \mathbf{U}_j \mathbf{D}_j \mathbf{u}_j), \quad (2.82)$$

where

$$\mathbf{p}_i = \mathbf{v}_i \times \mathbf{I}_i \mathbf{v}_i - \mathbf{f}_i^e.$$

These formulae are obtained by examining the relationship between $\mathbf{f}_i$ and $\mathbf{a}_i$ in Fig. 2.7 and assuming that $\mathbf{I}_j^A$

Table 2.8 Articulated-body algorithm for forward dynamics

```

inputs:  $\mathbf{q}, \dot{\mathbf{q}}, \boldsymbol{\tau}, model, {}^0\mathbf{f}_i^e$ 
output:  $\ddot{\mathbf{q}}$ 
model data:  $N_B, jtype(i), p(i), X_L(i), \mathbf{I}_i$ 

 $\mathbf{v}_0 = \mathbf{0}$ 
 $\mathbf{a}_0 = -\mathbf{a}_g$ 
for  $i = 1$  to  $N_B$  do
     $X_J(i) = xjcalc(jtype(i), \mathbf{q}_i)$ 
     ${}^iX_{p(i)} = X_J(i) X_L(i)$ 
    if  $p(i) \neq 0$  then
         ${}^iX_0 = {}^iX_{p(i)} {}^{p(i)}X_0$ 
    end
     $\Phi_i = pcalc(jtype(i), \mathbf{q}_i)$ 
     $\Phi_{c_i} = pdcalc(jtype(i), \mathbf{q}_i, \dot{\mathbf{q}}_i)$ 
     $\mathbf{v}_i = {}^iX_{p(i)} \mathbf{v}_{p(i)} + \Phi_i \dot{\mathbf{q}}_i$ 
     $\boldsymbol{\zeta}_i = \Phi_{c_i} \dot{\mathbf{q}}_i + \mathbf{v}_i \times \Phi_i \dot{\mathbf{q}}_i$ 
     $\mathbf{I}_i^A = \mathbf{I}_i$ 
     $\mathbf{p}_i^A = \mathbf{v}_i \times \mathbf{I}_i \mathbf{v}_i - {}^iX_0^{-T} {}^0\mathbf{f}_i^e$ 
end
for  $i = N_B$  to  $1$  do
     $\mathbf{U}_i = \mathbf{I}_i^A \Phi_i$ 
     $\mathbf{D}_i = (\Phi_i^T \mathbf{U}_i)^{-1}$ 
     $\mathbf{u}_i = \boldsymbol{\tau}_i - \mathbf{U}_i^T \boldsymbol{\zeta}_i - \Phi_i^T \mathbf{p}_i^A$ 
    if  $p(i) \neq 0$  then
         $\mathbf{I}_{p(i)}^A = \mathbf{I}_{p(i)}^A + {}^iX_{p(i)}^T (\mathbf{I}_i^A - \mathbf{U}_i \mathbf{D}_i \mathbf{U}_i^T) {}^iX_{p(i)}$ 
         $\mathbf{p}_{p(i)}^A = \mathbf{p}_{p(i)}^A + {}^iX_{p(i)}^T (\mathbf{p}_i^A + \mathbf{I}_i^A \boldsymbol{\zeta}_i + \mathbf{U}_i \mathbf{D}_i \mathbf{u}_i)$ 
    end
end
for  $i = 1$  to  $N_B$  do
     $\mathbf{a}_i = {}^iX_{p(i)} \mathbf{a}_{p(i)}$ 
     $\ddot{\mathbf{q}}_i = \mathbf{D}_i (\mathbf{u}_i - \mathbf{U}_i^T \mathbf{a}_i)$ 
     $\mathbf{a}_i = \mathbf{a}_i + \Phi_i \ddot{\mathbf{q}}_i + \boldsymbol{\zeta}_i$ 
end

```

and $\mathbf{p}_j^A$ are already known for every $j \in c(i)$. For more details see [2.1, 8, 15, 24].

The complete algorithm is shown in Table 2.8. It is expressed in link coordinates, as per the RNEA in Table 2.6. It makes a total of three passes through the kinematic tree. The first pass iterates from the base out to the tips; it calculates the link velocities using (2.76), the velocity-product term $\boldsymbol{\zeta}_i = \Phi_i \dot{\mathbf{q}}_i$, and it initializes the variables $\mathbf{I}_i^A$ and $\mathbf{p}_i^A$ to the values $\mathbf{I}_i$ and $\mathbf{p}_i (= \mathbf{v}_i \times \mathbf{I}_i \mathbf{v}_i - {}^iX_0^T {}^0\mathbf{f}_i^e)$, respectively. The second pass iterates from the tips back to the base; it calculates the articulated-body inertia and bias force for each link using (2.81) and (2.82). The third pass iterates from the base to the tips; it calculates the link and joint accelerations using (2.80) and (2.77).

2.5.3 The Composite-Rigid-Body Algorithm

The CRBA is an algorithm for calculating the joint-space inertia matrix (JSIM) of a kinematic tree. The most common use for the CRBA is as part of a forward dynamics algorithm. It first appeared as method 3 in [2.5].

Forward dynamics, in joint space, is the task of calculating $\ddot{\mathbf{q}}$ from $\mathbf{q}, \dot{\mathbf{q}}$, and $\boldsymbol{\tau}$. Starting from (2.37), the most obvious way to proceed is to calculate $\mathbf{H}$ and $\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$, and then solve the linear equation

$$\mathbf{H}\ddot{\mathbf{q}} = \boldsymbol{\tau} - (\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g) \quad (2.83)$$

for $\ddot{\mathbf{q}}$. If the mechanism is a kinematic tree, then $\mathbf{H}$ and $\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$ can be computed in $O(n^2)$ and $O(n)$ operations, respectively, and (2.83) can be solved in $O(n^3)$ operations. Algorithms that take this approach are therefore known collectively as $O(n^3)$ algorithms. However, this figure of $O(n^3)$ should be regarded as the worst-case complexity, since the actual complexity depends on the amount of branching in the tree [2.25]. Furthermore, even in the worst case, the n^3 term has a small coefficient, and does not dominate until approximately $n = 60$.

$\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$ can be calculated using an inverse dynamics algorithm. If $ID(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$ is the result of an inverse dynamics calculation with arguments $\mathbf{q}, \dot{\mathbf{q}}$ and $\ddot{\mathbf{q}}$, then

$$ID(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = \boldsymbol{\tau} = \mathbf{H}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g,$$

so

$$\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g = ID(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{0}). \quad (2.84)$$

Thus, the value of $\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$ for a kinematic tree can be calculated efficiently using the RNEA with $\ddot{\mathbf{q}} = \mathbf{0}$.

The key concept in the CRBA is to note that the JSIM only depends on the joint positions, and not their rates. The CRBA makes the simplifying assumption that the

rate at each joint is zero. By also assuming that gravity is zero, $\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$ is eliminated from (2.83). Furthermore, for a revolute joint, a unit of joint acceleration applied at the j -th joint produces the j -th column of the **JSIM**. This partitions the mechanism into two composite rigid bodies connected by the j -th joint, and simplifies the dynamics considerably. This concept has been generalized so that the **CRBA** may be applied to any joint type within a kinematic tree structure.

It can be shown that the general form of the **JSIM** for a kinematic tree is

$$H_{ij} = \begin{cases} \Phi_i^T I_i^C \Phi_j & \text{if } i \in c^*(j) \\ \Phi_i^T I_j^C \Phi_j & \text{if } j \in c^*(i) \\ \mathbf{0} & \text{otherwise,} \end{cases} \quad (2.85)$$

where $c^*(i)$ is the set of links in the subtree rooted at link i , including link i itself, and

$$I_i^C = \sum_{j \in c^*(i)} I_j. \quad (2.86)$$

Table 2.9 Composite-rigid-body algorithm for calculating the **JSIM**

inputs: *model*, *RNEA* partial results

output: *H*

model data: N_B , $p(i)$, I_i

RNEA data: Φ_i , ${}^iX_{p(i)}$

$H = \mathbf{0}$

for $i = 1$ **to** N_B **do**

$I_i^C = I_i$

end

for $i = N_B$ **to** 1 **do**

$F = I_i^C \Phi_i$

$H_{ii} = \Phi_i^T F$

if $p(i) \neq 0$ **then**

$I_{p(i)}^C = I_{p(i)}^C + {}^iX_{p(i)}^T I_i^C {}^iX_{p(i)}$

end

$j = i$

while $p(j) \neq 0$ **do**

$F = {}^jX_{p(j)}^T F$

$j = p(j)$

$H_{ij} = F^T \Phi_j$

$H_{ji} = H_{ij}^T$

end

end

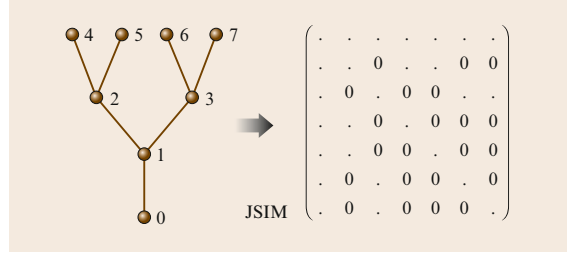


Fig. 2.8 Branch-induced sparsity: branches in the kinematic tree cause certain elements in the **JSIM** to be zero

See [2.8, 15]. In fact, I_i^C is the inertia of the composite rigid body formed by the rigid assembly of all the links in $c^*(i)$, and this is where the algorithm gets its name.

Equations (2.85) and (2.86) are the basis of the algorithm shown in Table 2.9, which is the **CRBA** in link coordinates. This algorithm assumes that the matrices ${}^iX_{p(i)}$ and Φ_i have already been calculated, e.g., during the calculation of $\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$. If this is not the case, then the relevant lines from Table 2.6 can be inserted into the first loop. The matrix F is a local variable. The first step, $H = \mathbf{0}$, can be omitted if there are no branches in the tree.

Having calculated $\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$ and H , the final step is to solve (2.83) for $\ddot{\mathbf{q}}$. This can be done using a standard Cholesky or LDL^T factorization. Note that H can be highly ill-conditioned [2.26], reflecting an underlying ill-conditioning of the kinematic tree itself, so it is recommended to use double-precision arithmetic for every step in the forward dynamics calculation. (This advice applies also to the **ABA**.)

Exploiting Sparsity

Equation (2.85) implies that some elements of H will automatically be zero if there are branches in the kinematic tree. An example of this effect is shown in Fig. 2.8. Observe that nearly half of the elements are zero. It is possible to exploit this sparsity using the factorization algorithms described in [2.25]. Depending on the amount of branching in the tree, the sparse algorithms can run many times faster than the standard algorithms.

2.5.4 Operational-Space Inertia Matrix

Two different algorithms are presented to calculate the **OSIM**. The first is an $O(n^3)$ algorithm that uses the basic definition of the **OSIM** along with efficient factorization of the **JSIM**. The second is an $O(n)$ algorithm that is based on efficient solution of the forward dynamics problem.

Algorithm Using Basic Definition

If a robot has a relatively small number of freedoms (e.g., six), then the most efficient method for calculating the **OSIM** is via (2.52). The procedure is as follows.

1. Calculate $\mathbf{H}$ via the **CRBA**.
2. Factorize $\mathbf{H}$ into $\mathbf{H} = \mathbf{L}\mathbf{L}^T$ (Cholesky factorization).
3. Use back-substitution to calculate $\mathbf{Y} = \mathbf{L}^{-1} \mathbf{J}^T$.
4. $\mathbf{A}^{-1} = \mathbf{Y}^T \mathbf{Y}$.
5. Factorize $\mathbf{A}^{-1}$ (optional).

The final step is only possible if the end-effector has a full six **DOFs**, and is only necessary if the application requires $\mathbf{A}$ rather than $\mathbf{A}^{-1}$. In the second step, an **LDL^T** factorization can be used instead of Cholesky, or one can use one of the efficient factorizations described in [2.25] for branched kinematic trees.

The other terms in (2.38) can be calculated via (2.53) and (2.54). In particular, (2.38) can be rewritten in the form

$$\dot{\mathbf{v}} + \mathbf{A}^{-1}(\mathbf{x})[\boldsymbol{\mu}(\mathbf{x}, \mathbf{v}) + \boldsymbol{\rho}(\mathbf{x})] = \mathbf{A}^{-1}(\mathbf{x}) \mathbf{f}, \quad (2.87)$$

and the quantity $\mathbf{A}^{-1}(\boldsymbol{\mu} + \boldsymbol{\rho})$ can be calculated from the formula

$$\mathbf{A}^{-1}(\boldsymbol{\mu} + \boldsymbol{\rho}) = \mathbf{J}\mathbf{H}^{-1}(\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g) - \dot{\mathbf{J}}\dot{\mathbf{q}}. \quad (2.88)$$

The term $\dot{\mathbf{J}}\dot{\mathbf{q}}$ is the velocity-product acceleration of the end-effector (2.50). It is calculated as a by-product of calculating $\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$ via the **RNEA** (2.84). Specifically, $\dot{\mathbf{J}}\dot{\mathbf{q}} = \mathbf{a}_{ee} - \mathbf{a}_0$, where $\mathbf{a}_{ee}$ is the calculated acceleration of the end-effector (expressed in the same coordinates as $\dot{\mathbf{v}}$) and $\mathbf{a}_0$ is the acceleration of the base ($-\mathbf{a}_g$).

$O(n)$ Algorithm

For a sufficiently large value of n , it becomes more efficient to use an $O(n)$ algorithm. Several such algorithms can be found in [2.27–29]. In this section, a more straightforward algorithm is given, which is based on an $O(n)$ calculation of the joint-space forward dynamics problem, e.g., via the **ABA**. It is a variation of the unit force method [2.28] and computes the inverse of the **OSIM**.

Starting with (2.87), observe that $\mathbf{A}^{-1}$ is a function of position only, and certain terms in the dynamic equations can be neglected without affecting its value. Specifically, if the joint rates, $\dot{\mathbf{q}}$, joint forces, $\boldsymbol{\tau}$, and gravitational forces are all set to zero, the value of $\mathbf{A}$ will remain unchanged. Under these conditions,

$$\dot{\mathbf{v}} = \mathbf{A}^{-1} \mathbf{f}. \quad (2.89)$$

Let us define $\hat{\mathbf{e}}_i$ to be a 6-D coordinate vector with a 1 in the i -th coordinate and zeros elsewhere. If we set

$\mathbf{f} = \hat{\mathbf{e}}_i$ in (2.89), then $\dot{\mathbf{v}}$ will equal column i of $\mathbf{A}^{-1}$. Let us also define the function $FD(i, j, \mathbf{q}, \dot{\mathbf{q}}, \mathbf{a}_0, \boldsymbol{\tau}, \mathbf{f})$, which performs a forward-dynamics calculation and returns the true acceleration of link i (i.e., $\mathbf{a}_i - \mathbf{a}_0$), expressed in the same coordinates as $\mathbf{f}$ (typically the base coordinates). The arguments $\mathbf{q}$, $\dot{\mathbf{q}}$ and $\boldsymbol{\tau}$ set the values of the joint position, velocity, and force variables, while j and $\mathbf{f}$ specify that an external force of $\mathbf{f}$ is to be applied to link j . The argument $\mathbf{a}_0$ specifies a fictitious base acceleration to include gravitational effects, and is set to either $\mathbf{0}$ or $-\mathbf{a}_g$.

With these definitions, we have

$$(\mathbf{A}^{-1})^i = FD(ee, ee, \mathbf{q}, \mathbf{0}, \mathbf{0}, \mathbf{0}, \hat{\mathbf{e}}_i) \quad (2.90)$$

and

$$\mathbf{A}^{-1}(\boldsymbol{\mu} + \boldsymbol{\rho}) = -FD(ee, ee, \mathbf{q}, \dot{\mathbf{q}}, -\mathbf{a}_g, \boldsymbol{\tau}, \mathbf{0}), \quad (2.91)$$

where $(\mathbf{A}^{-1})^i$ is column i of $\mathbf{A}^{-1}$, and ee is the body number of the end-effector. It therefore follows that the coefficients of (2.87) can be calculated using the algorithm shown in Table 2.10. This algorithm is $O(n)$.

The efficiency of the algorithm may be increased significantly when computing $\mathbf{A}^{-1}$ by noting that: (1) $\mathbf{v}_i$, $\boldsymbol{\zeta}_i$, and $\boldsymbol{\tau}_i$ in the **ABA** calculation (Table 2.8) may be set to zero, and (2) $\mathbf{I}_i^A$ and the quantities that depend upon it ($\mathbf{U}_i$ and $\mathbf{D}_i$) need only be computed once, since they do not vary with the applied force. Also, note that the algorithm may be applied to multiple end-effectors by modifying FD to accept a list of end-effector body numbers in its first argument, and return a composite vector containing the accelerations of all the specified bodies. The algorithm in Table 2.10 is then enclosed in a **for** loop that controls the second argument to FD , and iterates over all the end-effector body numbers [2.19]. However, with several end-effectors, the force propagation method of *Lilly* [2.28] should probably be used for enhanced computational efficiency. If there are m end-effectors, then the complexity is $O(mn)$.

Table 2.10 Algorithm to compute the inverse of the operational-space inertia matrix and other terms

for $j = 1$ to 6 **do**

$\dot{\mathbf{v}}^j = FD(ee, ee, \mathbf{q}, \mathbf{0}, \mathbf{0}, \mathbf{0}, \hat{\mathbf{e}}_j)$

end

$\mathbf{A}^{-1} = [\dot{\mathbf{v}}^1 \ \dot{\mathbf{v}}^2 \ \dots \ \dot{\mathbf{v}}^6]$

$\mathbf{A}^{-1}(\boldsymbol{\mu} + \boldsymbol{\rho}) = -FD(ee, ee, \mathbf{q}, \dot{\mathbf{q}}, -\mathbf{a}_g, \boldsymbol{\tau}, \mathbf{0})$

2.6 Kinematic Loops

All of the algorithms in the last section were for kinematic trees. In this section, a final algorithm is provided for the forward dynamics of closed-loop systems. The algorithm supplements the dynamic equations of motion for a spanning tree of the closed-loop system, with the loop-closure constraint equations. Three different methods are given to solve the resulting linear system of equations. An efficient algorithm is given to compute the loop-closure constraints.

Systems with closed kinematic loops exhibit more complicated dynamics than kinematic trees. For example:

1. The degree of motion freedom of a kinematic tree is fixed, but that of a closed-loop system can vary.
2. The degree of instantaneous motion freedom is always the same as the degree of finite motion freedom in a kinematic tree, but they can be different in a closed-loop system.
3. Every force in a kinematic tree can be determined, but some forces in a closed-loop system can be indeterminate. This occurs whenever a closed-loop system is overconstrained.

Two examples of these phenomena are shown in Fig. 2.9. The mechanism in Fig. 2.9a has no finite motion freedom, but it has two degrees of infinitesimal motion freedom. The mechanism in Fig. 2.9b has one degree of freedom when $\theta \neq 0$, but if $\theta = 0$ then the two arms, *A* and *B*, are able to move independently, and the mechanism has two degrees of freedom. Moreover, at the boundary between these two motion regimes, the mechanism has three degrees of infinitesimal motion freedom. Both these mechanisms are planar, and are therefore overconstrained.

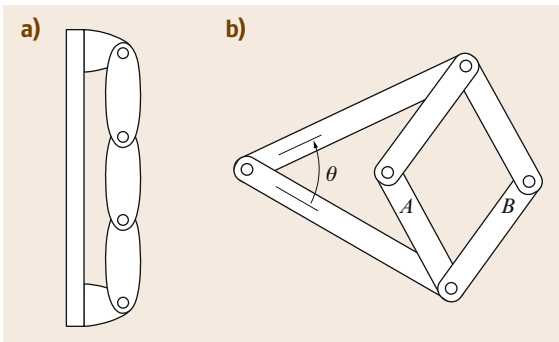


Fig. 2.9 Pathological closed-loop systems

As a result, the out-of-plane components of the joint constraint forces are indeterminate. This kind of indeterminacy has no effect on the motions of these mechanisms, but it does complicate the calculation of their dynamics.

2.6.1 Formulation of Closed-Loop Algorithm

A closed-loop system can be modeled as a spanning tree subject to a set of loop-closure constraint forces. If

$$H\ddot{\mathbf{q}} + C\dot{\mathbf{q}} + \boldsymbol{\tau}_g = \boldsymbol{\tau}$$

is the equation of motion of the spanning tree on its own, then the equation of motion for the closed-loop system is

$$H\ddot{\mathbf{q}} + C\dot{\mathbf{q}} + \boldsymbol{\tau}_g = \boldsymbol{\tau} + \boldsymbol{\tau}^a + \boldsymbol{\tau}^c, \quad (2.92)$$

where $\boldsymbol{\tau}^a$ and $\boldsymbol{\tau}^c$ are vectors of loop-closure active and constraint forces, respectively, expressed in the generalized force coordinates of the spanning tree. $\boldsymbol{\tau}^a$ is a known quantity, and $\boldsymbol{\tau}^c$ is unknown. $\boldsymbol{\tau}^a$ comes from the force elements acting at the loop-closing joints (springs, dampers and actuators). If there are no such force elements, then $\boldsymbol{\tau}^a = \mathbf{0}$.

The loop-closure constraints restrict the motion of the spanning tree. At the acceleration level, these constraints can be expressed in the form of a linear equation,

$$\mathbf{L}\ddot{\mathbf{q}} = \mathbf{1}, \quad (2.93)$$

where $\mathbf{L}$ is an $n^c \times n$ matrix. n^c is the number of constraints due to the loop-closing joints, and is given by the formula

$$n^c = \sum_{k=N_B+1}^{N_J} n_k^c, \quad (2.94)$$

where n_k^c is the number of constraints imposed by joint *k*. If $\text{rank}(\mathbf{L}) < n^c$ then the loop-closure constraints are linearly dependent, and the closed-loop mechanism is overconstrained. The mobility of a closed-loop system (i.e., its degree of motion freedom) is given by the formula

$$\text{mobility} = n - \text{rank}(\mathbf{L}). \quad (2.95)$$

Given a constraint equation in the form of (2.93), it follows that the constraint forces can be expressed in the form

$$\boldsymbol{\tau}^c = \mathbf{L}^T \boldsymbol{\lambda}, \quad (2.96)$$

where $\boldsymbol{\lambda} = (\lambda_{N_B+1}^T \cdots \lambda_{N_J}^T)^T$ is an $n^c \times 1$ vector of unknown constraint-force variables (or Lagrange multipliers). If the mechanism is overconstrained, then $\mathbf{L}^T$ will have a null space, and the component of $\boldsymbol{\lambda}$ lying in this null space will be indeterminate.

It is often possible to identify redundant constraints in advance. For example, if a kinematic loop is known to be planar, then the out-of-plane loop-closure constraints are redundant. In these circumstances, it is advantageous to remove the corresponding rows of $\mathbf{L}$ and elements of $\mathbf{I}$ and $\boldsymbol{\lambda}$. The removed elements of $\boldsymbol{\lambda}$ can be assigned a value of zero.

Combining (2.92), (2.93), and (2.96) produces the following equation of motion for a closed-loop system:

$$\begin{pmatrix} \mathbf{H} & \mathbf{L}^T \\ \mathbf{L} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \ddot{\mathbf{q}} \\ -\boldsymbol{\lambda} \end{pmatrix} = \begin{pmatrix} \boldsymbol{\tau} + \boldsymbol{\tau}^a - (\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g) \\ \mathbf{I} \end{pmatrix}. \quad (2.97)$$

The system matrix is symmetric, but indefinite. If $\mathbf{L}$ has full rank, then the system matrix will be nonsingular, otherwise it will be singular, and one or more elements of $\boldsymbol{\lambda}$ will be indeterminate.

Equation (2.97) can be solved in any of the following ways:

1. solve it directly for $\ddot{\mathbf{q}}$ and $\boldsymbol{\lambda}$
2. solve for $\boldsymbol{\lambda}$ first, and then use the result to solve for $\ddot{\mathbf{q}}$
3. solve (2.93) for $\ddot{\mathbf{q}}$, substitute the result into (2.92), eliminate the unknown constraint forces, and solve for the remaining unknowns

Method 1 is the simplest, but generally also the least efficient. This method is appropriate when the system matrix is nonsingular. As the size of the system matrix is $(n + n^c) \times (n + n^c)$, this method is $O((n + n^c)^3)$.

Method 2 is particularly useful if $n \gg n^c$, and offers the opportunity to use $O(n)$ algorithms on the spanning tree [2.6]. From (2.97),

$$\mathbf{L}\mathbf{H}^{-1}\mathbf{L}^T\boldsymbol{\lambda} = \mathbf{I} - \mathbf{L}\mathbf{H}^{-1}[\boldsymbol{\tau} + \boldsymbol{\tau}^a - (\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g)]. \quad (2.98)$$

This equation can be formulated in $O(n(n^c)^2)$ operations via $O(n)$ algorithms, and solved in $O((n^c)^3)$. Once $\boldsymbol{\lambda}$ is known, $\boldsymbol{\tau}^c$ can be calculated via (2.96) in $O(nn^c)$ operations, and (2.92) solved by an $O(n)$ al-

gorithm; so the total complexity is $O(n(n^c)^2 + (n^c)^3)$. If $\mathbf{L}$ is rank deficient, then $\mathbf{L}\mathbf{H}^{-1}\mathbf{L}^T$ will be singular; but it is still a positive-semidefinite matrix, and presents a slightly easier factorization problem than a singular instance of the indefinite system matrix in (2.97).

Method 3 is useful if $n - n^c$ is small, or if $\mathbf{L}$ is expected to be rank deficient. Equation (2.93) is solved using a special version of Gaussian elimination (or similar procedure), which is equipped with a numerical rank test, and which is designed to solve underdetermined systems. The solution is an equation of the form

$$\ddot{\mathbf{q}} = \mathbf{K}\mathbf{y} + \ddot{\mathbf{q}}_0,$$

where $\ddot{\mathbf{q}}_0$ is any particular solution to (2.93), $\mathbf{K}$ is an $n \times (n - \text{rank}(\mathbf{L}))$ matrix with the property $\mathbf{L}\mathbf{K} = \mathbf{0}$, and $\mathbf{y}$ is a vector of $n - \text{rank}(\mathbf{L})$ unknowns. (Typically, $\mathbf{y}$ is a linearly independent subset of the elements of $\ddot{\mathbf{q}}$.) Substituting this expression for $\ddot{\mathbf{q}}$ into (2.92), and pre-multiplying both sides by $\mathbf{K}^T$ to eliminate $\boldsymbol{\tau}^c$, produces

$$\mathbf{K}^T \mathbf{H} \mathbf{K} \mathbf{y} = \mathbf{K}^T (\boldsymbol{\tau} + \boldsymbol{\tau}^a - (\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g) - \mathbf{H}\ddot{\mathbf{q}}_0). \quad (2.99)$$

This method also has cubic complexity, but it can be the most efficient if $n - n^c$ is small. It is also reported to be more stable than method 1 [2.30].

2.6.2 Closed-Loop Algorithm

Algorithms for calculating $\mathbf{H}$ and $\mathbf{C}\dot{\mathbf{q}} + \boldsymbol{\tau}_g$ can be found in Sections 2.5.3 and 2.5.1, respectively, which just leaves $\mathbf{L}$, $\mathbf{I}$ and $\boldsymbol{\tau}^a$. To keep things simple, we will assume that all loop-closing joints are zero-DOF joints. There is no loss of generality with this assumption: one simply breaks open the loops by cutting links instead of joints (see Fig. 2.10). However, there may be some loss of efficiency. With this assumption, we only need to calculate $\mathbf{L}$ and $\mathbf{I}$, since $\boldsymbol{\tau}^a = \mathbf{0}$.

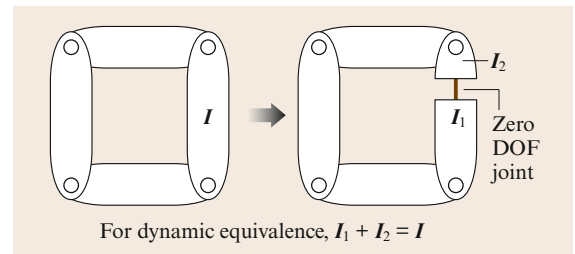


Fig. 2.10 Inserting a zero-DOF joint in preparation for cutting the loop open at that joint

Loop Constraints

In the general case, the velocity constraint equation for loop k is

$$(\Psi_k^c)^T (\mathbf{v}_{s(k)} - \mathbf{v}_{p(k)}) = \mathbf{0}, \quad (2.100)$$

and the acceleration constraint is

$$(\Psi_k^c)^T (\mathbf{a}_{s(k)} - \mathbf{a}_{p(k)}) + (\Psi_k^c)^T (\mathbf{v}_{s(k)} - \mathbf{v}_{p(k)}) = \mathbf{0}. \quad (2.101)$$

However, if every loop-closing joint has zero DOF, then these equations simplify to

$$\mathbf{v}_{s(k)} - \mathbf{v}_{p(k)} = \mathbf{0} \quad (2.102)$$

and

$$\mathbf{a}_{s(k)} - \mathbf{a}_{p(k)} = \mathbf{0}. \quad (2.103)$$

Let us define a loop Jacobian, $\mathbf{J}_k$, with the property that

$$\mathbf{v}_{s(k)} - \mathbf{v}_{p(k)} = \mathbf{J}_k \dot{\mathbf{q}}. \quad (2.104)$$

$\mathbf{J}_k$ is a $6 \times n$ matrix defined by the formula

$$\mathbf{J}_k = (e_{1k} \Phi_1 \cdots e_{N_B k} \Phi_{N_B}), \quad (2.105)$$

where

$$e_{ik} = \begin{cases} +1 & \text{if } s(k) \in c^*(i) \text{ and } p(k) \notin c^*(i), \\ -1 & \text{if } p(k) \in c^*(i) \text{ and } s(k) \notin c^*(i), \\ 0 & \text{otherwise.} \end{cases}$$

In other words, $e_{ik} = +1$ if joint i lies on the path to $s(k)$ but not the path to $p(k)$; $e_{ik} = -1$ if joint i lies on the path to $p(k)$ but not the path to $s(k)$; and $e_{ik} = 0$ if joint i lies on both paths or on neither.

The loop acceleration constraint can now be written

$$\begin{aligned} \mathbf{0} &= \mathbf{a}_{s(k)} - \mathbf{a}_{p(k)} \\ &= \mathbf{J}_k \ddot{\mathbf{q}} + \dot{\mathbf{J}}_k \dot{\mathbf{q}} \\ &= \mathbf{J}_k \ddot{\mathbf{q}} + \mathbf{a}_{s(k)}^{\text{vp}} - \mathbf{a}_{p(k)}^{\text{vp}}, \end{aligned} \quad (2.106)$$

where $\mathbf{a}_i^{\text{vp}}$ is the velocity-product acceleration of link i , which is the acceleration it would have if $\ddot{\mathbf{q}}$ were zero. The velocity-product acceleration of every link is calculated during the calculation of the vector $\mathbf{C}\dot{\mathbf{q}} + \mathbf{\tau}_g$ (2.84). If the RNEA is used to calculate $\mathbf{C}\dot{\mathbf{q}} + \mathbf{\tau}_g$, then

$\mathbf{a}_i^{\text{vp}}$ will be the value of $\mathbf{a}_i$ calculated by the RNEA with its acceleration argument set to zero.

The matrices $\mathbf{L}$ and $\mathbf{I}$ can now be expressed as follows:

$$\mathbf{L} = \begin{pmatrix} \mathbf{L}_{N_B+1} \\ \vdots \\ \mathbf{L}_{N_J} \end{pmatrix} \quad \text{and} \quad \mathbf{I} = \begin{pmatrix} \mathbf{I}_{N_B+1} \\ \vdots \\ \mathbf{I}_{N_J} \end{pmatrix}, \quad (2.107)$$

where

$$\mathbf{L}_k = \mathbf{J}_k \quad (2.108)$$

and

$$\mathbf{I}_k = \mathbf{a}_{p(k)}^{\text{vp}} - \mathbf{a}_{s(k)}^{\text{vp}}. \quad (2.109)$$

Constraint Stabilization

In practice, it is necessary to stabilize loop-closure constraints, or they will simply fly apart during simulation because of numerical integration errors. The standard technique is due to *Baumgarte* [2.3, 7, 31], and consists of replacing each constraint equation of the form

$$a_e = 0$$

with one of the form

$$a_e + K_v v_e + K_p p_e = 0,$$

where a_e , v_e , and p_e are the acceleration, velocity, and position errors, respectively, and K_v and K_p are positive constants. Typically, one chooses a time constant, t_c , according to how quickly one wants the position and velocity errors to decay. K_p and K_v are then given by the formulae $K_v = 2/t_c$ and $K_p = 1/t_c^2$. However, there is no good rule for choosing t_c . If t_c is too long, then loop-constraint errors accumulate faster than they decay; if t_c is too short, then the equations of motion become excessively stiff, causing a loss of numerical integration accuracy. A reasonable value for a large, slow industrial robot is $t_c = 0.1$, while a smaller, faster robot might need $t_c = 0.01$.

To incorporate stabilization terms into the loop constraint equation, we replace (2.109) with

$$\mathbf{I}_k = \mathbf{a}_{p(k)}^{\text{vp}} - \mathbf{a}_{s(k)}^{\text{vp}} - K_v (\mathbf{v}_{s(k)} - \mathbf{v}_{p(k)}) - K_p \mathbf{p}_{ek}, \quad (2.110)$$

where $\mathbf{p}_{ek}$ is a vector representing the position error in loop k . In absolute coordinates (i. e., link 0 coordinates), $\mathbf{p}_{ek}$ is given by

$$\mathbf{p}_{ek} = \mathbf{x_to_vec}(^0X_{p(k)} X_{L1}^{-1}(k) X_{L2}(k)^{s(k)} X_0), \quad (2.111)$$

where the $X_{L1}(k)$ and $X_{L2}(k)$ transforms are defined in (2.62) and (2.63), and shown in Fig. 2.4 for joint k , and $\mathbf{x_to_vec}(^B X_A)$ computes a vector approximating the displacement from frame A to frame B , assuming this displacement to be infinitesimal. $\mathbf{x_to_vec}$ can be defined as

$$\mathbf{x_to_vec}(X) = \frac{1}{2} \begin{pmatrix} X_{23} - X_{32} \\ X_{31} - X_{13} \\ X_{12} - X_{21} \\ X_{53} - X_{62} \\ X_{61} - X_{43} \\ X_{42} - X_{51} \end{pmatrix}. \quad (2.112)$$

Algorithm

Table 2.11 shows an algorithm for calculating $\mathbf{L}$ and $\mathbf{l}$ for the special case when all the loop-closing joints have zero DOF. It combines simplicity with good performance by transforming every quantity that is needed to formulate the loop-closure constraints into a single coordinate system, in this case absolute (link 0) coordinates, so that no further transforms are needed.

The first loop calculates the transforms from absolute to link coordinates, and uses them to transform Φ_i to absolute coordinates. Only the Φ_i that are needed in the loop-closure constraints are transformed.

The second loop calculates the nonzero elements of $\mathbf{L}$ (which can be sparse), according to (2.105). The inner while loop terminates on the root of the loop, which is the highest-numbered common ancestor of links $p(k)$ and $s(k)$. It could be the fixed base if they have no other common ancestor. The second loop ends with the calculation of $\mathbf{l}$, in absolute coordinates, according to (2.110).

Table 2.11 Algorithm to calculate loop-closure constraints

```

inputs: model, RNEA partial results
outputs:  $\mathbf{L}$ ,  $\mathbf{l}$ 
model data:  $N_B$ ,  $p(i)$ ,  $N_J$ ,  $p(k)$ ,  $s(k)$ ,  $LB(i)$ ,
 $X_{L1}(k)$ ,  $X_{L2}(k)$ ,  $K_p$ ,  $K_v$ 
RNEA data:  $\Phi_i$ ,  ${}^iX_{p(i)}$ ,  $\mathbf{v}_{p(k)}$ ,  $\mathbf{v}_{s(k)}$ ,  $\mathbf{a}_{p(k)}^{vp}$ ,  $\mathbf{a}_{s(k)}^{vp}$ 
for  $i = 1$  to  $N_B$  do
  if  $p(i) \neq 0$  then
     ${}^iX_0 = {}^iX_{p(i)} {}^{p(i)}X_0$ 
  end
  if  $LB(i) \neq \text{null}$  then
     ${}^0\Phi_i = {}^iX_0^{-1} \Phi_i$ 
  end
end
 $\mathbf{L} = \mathbf{0}$ 
for  $k = N_B + 1$  to  $N_J$  do
   $i = p(k)$ 
   $j = s(k)$ 
  while  $i \neq j$  do
    if  $i > j$  then
       $L_{k,i} = -{}^0\Phi_i$ 
       $i = p(i)$ 
    else
       $L_{k,j} = {}^0\Phi_j$ 
       $j = p(j)$ 
    end
  end
   $\mathbf{a}_e = {}^{s(k)}X_0^{-1} \mathbf{a}_{s(k)}^{vp} - {}^{p(k)}X_0^{-1} \mathbf{a}_{p(k)}^{vp}$ 
   $\mathbf{v}_e = {}^{s(k)}X_0^{-1} \mathbf{v}_{s(k)} - {}^{p(k)}X_0^{-1} \mathbf{v}_{p(k)}$ 
   $\mathbf{p}_e = \mathbf{x\_to\_vec}({}^{p(k)}X_0^{-1} X_{L1}^{-1}(k) X_{L2}(k)^{s(k)} X_0)$ 
   $\mathbf{l}_k = -\mathbf{a}_e - K_v \mathbf{v}_e - K_p \mathbf{p}_e$ 
end

```

2.7 Conclusions and Further Reading

This chapter has presented the fundamentals of rigid-body dynamics as they apply to robot mechanisms. It has covered the following topics: the spatial vector algebra, which provides a concise notation for describing and implementing dynamics equations and algorithms; the canonical equations of motion that are most frequently used in robotics; how to construct a dynamic model of

a robot; and several efficient model-based algorithms for calculating inverse dynamics, forward dynamics, and the joint-space and operational-space inertia matrices.

There are many topics in dynamics that have not been mentioned in this chapter, but can be found in later chapters of this handbook. The dynamics of robots with elastic links and joints is covered in Chap. 13; the prob-

lem of identifying the parameters of a dynamic model is covered in Chap. 14; the dynamics of physical contact between a robot and the objects in its environment is described in Chap. 27; and the dynamics of robots with floating bases is described in Chap. 45.

We conclude this chapter by noting that a brief history of robot dynamics can be found in [2.32], and that a more extensive treatment of robot dynamics can be found in books such as [2.8, 10, 15, 28, 33–35]. Finally, some suggestions for further reading are listed below.

2.7.1 Multibody Dynamics

Robot dynamics can be regarded as a subset (or a specific application) of the broader discipline of multibody dynamics. Books on multibody dynamics include [2.3, 14, 31, 36–41]. Of course, multibody dynamics is, in turn, a subset of classical mechanics; and the mathematical foundations of the subject can be found in any good book on classical mechanics, such as [2.13].

2.7.2 Alternative Representations

This chapter has used spatial vectors to express the equations of motion. There are various alternatives to the use of spatial vectors: other kinds of 6-D vector, 3-D vectors, 4×4 matrices, and the spatial operator algebra. All 6-D vector formalisms are similar, but are not exactly the same. The main alternatives to spatial vectors are: screws [2.10–12], motors [2.42], Lie algebras [2.12, 43], and ad hoc notations. (An ad hoc notation is one in which 3-D vectors are grouped into pairs for the purpose of reducing the volume of algebra.) Three-dimensional vectors are the formalism used in most classical mechanics and multibody texts, and are also a precursor to 6-D vector and 4×4 matrix formalisms. 4×4 matrices are popular in robotics because they are very useful for kinematics. However, they are not so useful for dynamics. 4×4 matrix formulations of dynamics can be found in [2.33, 44, 45]. The spatial operator algebra was developed at the Jet Propulsion Laboratory (JPL) by Rodriguez, Jain, and others. It uses $6N$ -dimensional vectors and $6N \times 6N$ matrices, the latter regarded as linear operators. Examples of this notation can be found in [2.46–48].

2.7.3 Alternative Formulations

This chapter used a vectorial formulation of the equations of motion that is usually called the Newtonian or Newton–Euler formulation. The main alternative is

the Lagrangian formulation, in which the equations of motion are obtained via Lagrange’s equation. Examples of the Lagrangian formulation can be found in [2.9, 10, 17, 49, 50]. *Kane’s* method has also been applied in robotics [2.51, 52].

2.7.4 Efficiency

Because of the need for real-time implementation, especially in control, the robotics community has focused on the problem of computational efficiency. For *inverse dynamics*, the $O(n)$ recursive Newton–Euler algorithm (RNEA) of Luh et al. [2.4] remains the most important algorithm. Further improvements to the algorithm are given in [2.53, 54]. For *forward dynamics*, the two algorithms presented in this chapter remain the most important for computational considerations: the $O(n)$ articulated-body algorithm (ABA) developed by Featherstone [2.1] and the $O(n^3)$ algorithm based on the composite-rigid-body algorithm (CRBA) of Walker and Orin [2.5]. Improvements were made in the ABA over the years [2.15, 16, 24] so that it was more efficient than the CRBA-based algorithm for decreasingly smaller values of n . However, recent application of the CRBA to branched kinematic trees [2.25] and robotic systems with motion-controlled appendages [2.55] continue to show the viability of the CRBA approach.

For the *joint-space inertia matrix*, the CRBA [2.5] is the most important algorithm. A number of improvements and modifications have been made over the years to increase its computational efficiency [2.15, 56–58]. For the *operational-space inertia matrix*, efficient $O(n)$ algorithms have been developed [2.27–29] and applied to increasingly complex systems [2.19].

2.7.5 Accuracy

Concerns can arise over the numerical accuracy of a dynamics algorithm, the accuracy of a simulation (i.e., numerical integration accuracy), or the accuracy of a dynamic model. The numerical accuracy of dynamics algorithms has received relatively little attention compared with efficiency. The RNEA, CRBA, and ABA have all been tested for accuracy on a large variety of rigid-body systems, but the same cannot be said of most other algorithms. Rigid-body systems are often ill-conditioned, in the sense that a small change in the applied force (or a model parameter) can produce a large change in the resulting acceleration. This phenomenon was studied by Featherstone [2.26], who discovered that the ill-conditioning gets worse with in-

creasing body count, and that it can grow in proportion to $O(n^4)$ in the worst case. Other publications on this topic include [2.8, 30, 59, 60].

2.7.6 Software Packages

A number of software packages have been developed to provide dynamic simulation capabilities for multibody systems, and in particular, robotic systems. Several have been written in MATLAB® for ease of integration with other analysis, control, and simulation programs. While many of these are open source, some are offered at a relatively low cost to the user. They differ in their capabilities in a variety of ways including: speed, topologies and joint models supported, accuracy, underlying dynamic formulation and associated order of complexity, user interface, graphics support, numerical integration routines, integration with other code, application support, and cost. Among those commonly cited are: Adams [2.61], Autolev [2.62], DynaMechs [2.63], Open Dynamics Engine [2.64], Robotics Studio [2.65], Robotics Toolbox [2.66], SD/FAST [2.67], SimMechanics [2.68], and Webots [2.69].

2.7.7 Symbolic Simplification

The technique of symbolic simplification takes a general-purpose dynamics algorithm, and applies it symbolically to a specific dynamic model. The result is a list of assignment statements detailing what the algorithm would have done if it had been executed for real. This list is then inspected and pruned of all unnecessary calculations, and the remainder is output to a text file in the form of computer source code. This code can run as much as ten times faster than the original general-purpose algorithm, but it is specific to one dynamic model. Both Autolev [2.62] and SD/FAST [2.67] use this technique. Other publications on symbolic simplification for dynamics include [2.70–75].

2.7.8 Algorithms for Parallel Computers

In order to speed up the common dynamics computations, a number of algorithms have been developed for parallel and pipelined computers. For *inverse dynamics*, early work focused on speeding up the $O(n)$ RNEA on up to n processors [2.76, 77] while subsequent work resulted in $O(\log_2 n)$ algorithms [2.78, 79]. For the $O(n^2)$ CRBA to compute the *joint-space inertia matrix*, early work resulted in $O(\log_2 n)$ algorithms for n processors to compute the composite-rigid-body inertias and diagonal elements of the matrix [2.80, 81]. Subsequent work resulted in $O(\log_2 n)$ algorithms for $O(n^2)$ processors to compute the entire matrix [2.82, 83]. For *forward dynamics*, speedup was obtained for a multiple manipulator system on a parallel/pipelined supercomputer [2.84]. The first $O(\log_2 n)$ algorithm for n processors was developed for an unbranched serial chain [2.85]. More recent work has focused on $O(\log_2 n)$ algorithms for more complex structures [2.60, 86, 87].

2.7.9 Topologically-Varying Systems

There are many robot mechanisms whose topology varies over time because of a change of contact conditions, especially with the environment. In legged vehicles, use of a compliant ground-contact model to compute the contact forces reduced the closed-loop structure to a tree structure [2.88]. However, for cases in which the contacts are very stiff, numerical integration problems may result. In more recent work [2.35, 89] in which hard contact constraints are assumed, an efficient method was used to reduce the large number of coordinate variables from that which may be necessary in general-purpose motion analysis systems [2.38]. Also, they were able to automatically identify the variables as the structure varied and developed a method for computing the velocity boundary conditions after configuration changes [2.35, 89].

References

- 2.1 R. Featherstone: The Calculation of Robot Dynamics using Articulated-Body Inertias, *Int. J. Robot. Res.* **2**(1), 13–30 (1983)
- 2.2 J.J. Craig: *Introduction to Robotics: Mechanics and Control*, 3rd edn. (Pearson Prentice Hall, Upper Saddle River, NJ 2005)
- 2.3 R.E. Roberson, R. Schwertassek: *Dynamics of Multi-body Systems* (Springer-Verlag, Berlin/Heidelberg/New York 1988)
- 2.4 J.Y.S. Luh, M.W. Walker, R.P.C. Paul: On-Line Computational Scheme for Mechanical Manipulators, *Trans. ASME J. Dyn. Syst. Measur. Control* **102**(2), 69–76 (1980)

- 2.5 M.W. Walker, D.E. Orin: Efficient Dynamic Computer Simulation of Robotic Mechanisms, *Trans. ASME J. Dyn. Syst. Measur. Control* **104**, 205–211 (1982)
- 2.6 D. Baraff: Linear-Time Dynamics using Lagrange Multipliers, *Proc. SIGGRAPH '96* (New Orleans 1996) pp. 137–146
- 2.7 J. Baumgarte: Stabilization of Constraints and Integrals of Motion in Dynamical Systems, *Comput. Methods Appl. Mech. Eng.* **1**, 1–16 (1972)
- 2.8 R. Featherstone: *Rigid Body Dynamics Algorithms* (Springer, Berlin, Heidelberg 2007)
- 2.9 R.M. Murray, Z. Li, S.S. Sastry: *A Mathematical Introduction to Robotic Manipulation* (CRC, Boca Raton, FL 1994)
- 2.10 J. Angeles: *Fundamentals of Robotic Mechanical Systems*, 2nd edn. (Springer-Verlag, New York 2003)
- 2.11 R.S. Ball: *A Treatise on the Theory of Screws* (Cambridge Univ. Press, London 1900), Republished (1998)
- 2.12 J.M. Selig: *Geometrical Methods in Robotics* (Springer, New York 1996)
- 2.13 D.T. Greenwood: *Principles of Dynamics* (Prentice-Hall, Englewood Cliffs, NJ 1988)
- 2.14 F.C. Moon: *Applied Dynamics* (Wiley, New York 1998)
- 2.15 R. Featherstone: *Robot Dynamics Algorithms* (Kluwer Academic, Boston 1987)
- 2.16 S. McMillan, D.E. Orin: Efficient Computation of Articulated-Body Inertias Using Successive Axial Screws, *IEEE Trans. Robot. Autom.* **11**, 606–611 (1995)
- 2.17 L. Sciavicco, B. Siciliano: *Modeling and Control of Robot Manipulators*, 2nd edn. (Springer, London 2000)
- 2.18 J. Slotine, W. Li: On the Adaptive Control of Robot Manipulators, *Int. J. Robot. Res.* **6**(3), 49–59 (1987)
- 2.19 K.S. Chang, O. Khatib: Operational Space Dynamics: Efficient Algorithms for Modeling and Control of Branching Mechanisms. In: *Proc. of IEEE International Conference on Robotics and Automation* (San Francisco 2000) pp. 850–856
- 2.20 O. Khatib: A Unified Approach to Motion and Force Control of Robot Manipulators: The Operational Space Formulation, *IEEE J. Robot. Autom.* **3**(1), 43–53 (1987)
- 2.21 Y.F. Zheng, H. Hemami: Mathematical Modeling of a Robot Collision with its Environment, *J. Robot. Syst.* **2**(3), 289–307 (1985)
- 2.22 W. Khalil, E. Dombre: *Modeling, Identification and Control of Robots* (Taylor & Francis, New York 2002)
- 2.23 J. Denavit, R.S. Hartenberg: A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices, *J. Appl. Mech.* **22**, 215–221 (1955)
- 2.24 H. Brandl, R. Johanni, M. Otter: A Very Efficient Algorithm for the Simulation of Robots and Similar Multibody Systems Without Inversion of the Mass Matrix. In: *Proc. of IFAC/IFIP/IMACS International Symposium on Theory of Robots*, (Vienna 1986)
- 2.25 R. Featherstone: Efficient Factorization of the Joint Space Inertia Matrix for Branched Kinematic Trees, *Int. J. Robot. Res.* **24**(6), 487–500 (2005)
- 2.26 R. Featherstone: An Empirical Study of the Joint Space Inertia Matrix, *Int. J. Robot. Res.* **23**(9), 859–871 (2004)
- 2.27 K. Kreutz-Delgado, A. Jain, G. Rodriguez: Recursive Formulation of Operational Space Control, *Proc. of IEEE International Conference on Robotics and Automation* (Sacramento, CA April 1991) pp. 1750–1753
- 2.28 K.W. Lilly: *Efficient Dynamic Simulation of Robotic Mechanisms* (Kluwer Academic, Norwell, MA 1993)
- 2.29 K.W. Lilly, D.E. Orin: Efficient O(N) Recursive Computation of the Operational Space Inertia Matrix, *IEEE Trans. Syst. Man Cybern.* **23**(5), 1384–1391 (1993)
- 2.30 R.E. Ellis, S.L. Ricker: Two Numerical Issues in Simulating Constrained Robot Dynamics, *IEEE Trans. Syst. Man Cybern.* **24**(1), 19–27 (1994)
- 2.31 J. Wittenburg: *Dynamics of Systems of Rigid Bodies* (B.G. Teubner, Stuttgart 1977)
- 2.32 R. Featherstone, D.E. Orin: Robot Dynamics: Equations and Algorithms. In: *Proc. of IEEE International Conference on Robotics and Automation*, (San Francisco, April 2000) pp. 826–834
- 2.33 C.A. Balafoutis, R.V. Patel: *Dynamic Analysis of Robot Manipulators: A Cartesian Tensor Approach* (Kluwer Academic, Boston 1991)
- 2.34 L.W. Tsai: *Robot Analysis and Design: The Mechanics of Serial and Parallel Manipulators* (Wiley, New York 1999)
- 2.35 K. Yamane: *Simulating and Generating Motions of Human Figures* (Springer, Berlin 2004)
- 2.36 F.M.L. Amirouche: *Fundamentals of Multibody Dynamics: Theory and Applications* (Birkhäuser, Boston 2006)
- 2.37 M.G. Coutinho: *Dynamic Simulations of Multibody Systems* (Springer, New York 2001)
- 2.38 E.J. Haug: *Computer Aided Kinematics and Dynamics of Mechanical Systems* (Allyn and Bacon, Boston, MA 1989)
- 2.39 R.L. Huston: *Multibody Dynamics* (Butterworths, Boston 1990)
- 2.40 A.A. Shabana: *Computational Dynamics*, 2nd edn. (Wiley, New York 2001)
- 2.41 V. Stejskal, M. Valášek: *Kinematics and Dynamics of Machinery* (Marcel Dekker, New York 1996)
- 2.42 L. Brand: *Vector and Tensor Analysis*, 4th edn. (Wiley/Chapman and Hall, New York/London 1953)
- 2.43 F.C. Park, J.E. Bobrow, S.R. Ploen: A Lie Group Formulation of Robot Dynamics, *Int. J. Robot. Res.* **14**(6), 609–618 (1995)
- 2.44 M.E. Kahn, B. Roth: The Near Minimum-time Control of Open-loop Articulated Kinematic Chains, *J. Dyn. Syst. Measur. Control* **93**, 164–172 (1971)
- 2.45 J.J. Uicker: Dynamic Force Analysis of Spatial Linkages, *Trans. ASME J. Appl. Mech.* **34**, 418–424 (1967)

- 2.46 A. Jain: Unified Formulation of Dynamics for Serial Rigid Multibody Systems, *J. Guid. Control Dyn.* **14**(3), 531–542 (1991)
- 2.47 G. Rodriguez: Kalman Filtering, Smoothing, and Recursive Robot Arm Forward and Inverse Dynamics, *IEEE J. Robot. Autom.* **RA-3**(6), 624–639 (1987)
- 2.48 G. Rodriguez, A. Jain, K. Kreutz-Delgado: A Spatial Operator Algebra for Manipulator Modelling and Control, *Int. J. Robot. Res.* **10**(4), 371–381 (1991)
- 2.49 J.M. Hollerbach: A Recursive Lagrangian Formulation of Manipulator Dynamics and a Comparative Study of Dynamics Formulation Complexity, *IEEE Trans. Syst. Man Cybern.* **SMC-10**(11), 730–736 (1980)
- 2.50 M.W. Spong, S. Hutchinson, M. Vidyasagar: *Robot Modeling and Control* (Wiley, Hoboken, NJ 2006)
- 2.51 K.W. Buffinton: Kane's Method in Robotics. In: *Robotics and Automation Handbook*, ed. by T.R. Kurfess (CRC, Boca Raton, FL 2005), 6–1 to 6–31
- 2.52 T.R. Kane, D.A. Levinson: The Use of Kane's Dynamical Equations in Robotics, *Int. J. Robot. Res.* **2**(3), 3–21 (1983)
- 2.53 C.A. Balafoutis, R.V. Patel, P. Misra: Efficient Modeling and Computation of Manipulator Dynamics Using Orthogonal Cartesian Tensors, *IEEE J. Robot. Autom.* **4**, 665–676 (1988)
- 2.54 X. He, A.A. Goldenberg: An Algorithm for Efficient Computation of Dynamics of Robotic Manipulators. In: *Proc. of Fourth International Conference on Advanced Robotics*, (Columbus, OH, 1989) pp. 175–188
- 2.55 W. Hu, D.W. Marhefka, D.E. Orin: Hybrid Kinematic and Dynamic Simulation of Running Machines, *IEEE Trans. Robot.* **21**(3), 490–497 (2005)
- 2.56 C.A. Balafoutis, R.V. Patel: Efficient Computation of Manipulator Inertia Matrices and the Direct Dynamics Problem, *IEEE Trans. Syst. Man Cybern.* **19**, 1313–1321 (1989)
- 2.57 K.W. Lilly, D.E. Orin: Alternate Formulations for the Manipulator Inertia Matrix, *Int. J. Robot. Res.* **10**, 64–74 (1991)
- 2.58 S. McMillan, D.E. Orin: Forward dynamics of multilegged vehicles using the composite rigid body method, *Proc. IEEE International Conference on Robotics and Automation* (1998) pp. 464–470
- 2.59 U.M. Ascher, D.K. Pai, B.P. Cloutier: Forward Dynamics: Elimination Methods, and Formulation Stiffness in Robot Simulation, *Int. J. Robot. Res.* **16**(6), 749–758 (1997)
- 2.60 R. Featherstone: A Divide-and-Conquer Articulated-Body Algorithm for Parallel $O(\log(n))$ Calculation of Rigid-Body Dynamics. Part 2: Trees, Loops and Accuracy, *Int. J. Robot. Res.* **18**(9), 876–892 (1999)
- 2.61 MSC Software Corporation: *Adams*, [On-line] <http://www.mscsoftware.com/> (Nov. 12 2007)
- 2.62 T. Kane, D. Levinson: *Autolev User's Manual* (Online Dynamics Inc., 2005)
- 2.63 S. McMillan, D.E. Orin, R.B. McGhee: DynaMechs: An Object Oriented Software Package for Efficient Dynamic Simulation of Underwater Robotic Vehicles. In: *Underwater Robotic Vehicles: Design and Control*, ed. by J. Yuh (TSI Press, Albuquerque, NM 1995) pp. 73–98
- 2.64 R. Smith: *Open Dynamics Engine User Guide*, Available online: <http://www.ode.org> (Nov. 12 2007)
- 2.65 Microsoft Corporation: *Robotics Studio* [On-line] <http://www.microsoft.com/robotics> (Nov. 12 2007)
- 2.66 P.I. Corke: A Robotics Toolbox for MATLAB, *IEEE Robot. Autom. Mag.* **3**(1), 24–32 (1996)
- 2.67 M.G. Hollars, D.E. Rosenthal, M.A. Sherman: *SD/FAST User's Manual* (Symbolic Dynamics Inc., 1994)
- 2.68 G.D. Wood, D.C. Kennedy: *Simulating Mechanical Systems in Simulink with SimMechanics* (MathWorks Inc., 2003)
- 2.69 Cyberbotics Ltd.: *Webots User Guide*, Available online: <http://www.cyberbotics.com> (Nov. 8 2007)
- 2.70 I.C. Brown, P.J. Larcombe: A Survey of Customised Computer Algebra Programs for Multibody Dynamic Modelling. In: *The Use of Symbolic Methods in Control System Analysis and Design*, ed. by N. Munro (The Institute of Engineering and Technology, London 1999) pp. 53–77
- 2.71 J.J. Murray, C.P. Neuman: ARM: An algebraic robot dynamic modeling program. In: *Proc. of IEEE International Conference on Robotics and Automation*, Atlanta, Georgia, March (1984) pp. 103–114
- 2.72 J.J. Murray, C.P. Neuman: Organizing Customized Robot Dynamic Algorithms for Efficient Numerical Evaluation, *IEEE Trans. Syst. Man Cybern.* **18**(1), 115–125 (1988)
- 2.73 F.C. Park, J. Choi, S.R. Ploen: Symbolic Formulation of Closed Chain Dynamics in Independent Coordinates, *Mech. Machine Theory* **34**, 731–751 (1999)
- 2.74 M. Vukobratovic, N. Kircanski: Real-time Dynamics of Manipulation Robots. In: *Scientific Fundamentals of Robotics*, Vol. 4 (Springer-Verlag, New York 1985)
- 2.75 J. Wittenburg, U. Wolz: Mesa Verde: A Symbolic Program for Nonlinear Articulated-Rigid-Body Dynamics. In: *ASME Design Engineering Division Conference and Exhibit on Mechanical Vibration and Noise*, Cincinnati, Ohio, ASME Paper No. 85-DET-151, 1–8, September (1985)
- 2.76 J.Y.S. Luh, C.S. Lin: Scheduling of Parallel Computation for a Computer-Controlled Mechanical Manipulator, *IEEE Trans. Syst. Man Cybern.* **12**(2), 214–234 (1982)
- 2.77 D.E. Orin: Pipelined Approach to Inverse Plant Plus Jacobian Control of Robot Manipulators. In: *Proc. of IEEE International Conference on Robotics and Automation*, Atlanta, Georgia, 169–175, March (1984)
- 2.78 R.H. Lathrop: Parallelism in Manipulator Dynamics, *Int. J. Robot. Res.* **4**(2), 80–102 (1985)
- 2.79 C.S.G. Lee, P.R. Chang: Efficient Parallel Algorithm for Robot Inverse Dynamics Computation, *IEEE Trans. Syst. Man Cybern.* **16**(4), 532–542 (1986)
- 2.80 M. Amin-Javaheri, D.E. Orin: Systolic Architectures for the Manipulator Inertia Matrix, *IEEE Trans. Syst. Man Cybern.* **18**(6), 939–951 (1988)

- 2.81 C.S.G. Lee, P.R. Chang: Efficient Parallel Algorithms for Robot Forward Dynamics Computation, IEEE Trans. Syst. Man Cybern. **18**(2), 238–251 (1988)
- 2.82 M. Amin-Javaheri, D.E. Orin: Parallel Algorithms for Computation of the Manipulator Inertia Matrix, Int. J. Robot. Res. **10**(2), 162–170 (1991)
- 2.83 A. Fijany, A.K. Bejczy: A Class of Parallel Algorithms for Computation of the Manipulator Inertia Matrix, IEEE Trans. Robot. Autom. **5**(5), 600–615 (1989)
- 2.84 S. McMillan, P. Sadayappan, D.E. Orin: Parallel Dynamic Simulation of Multiple Manipulator Systems: Temporal Versus Spatial Methods, IEEE Trans. Syst. Man Cybern. **24**(7), 982–990 (1994)
- 2.85 A. Fijany, I. Sharf, G.M.T. D’Eleuterio: Parallel $O(\log N)$ Algorithms for Computation of Manipulator Forward Dynamics, IEEE Trans. Robot. Autom. **11**(3), 389–400 (1995)
- 2.86 R. Featherstone: A Divide-and-Conquer Articulated-Body Algorithm for Parallel $O(\log(n))$ Calculation of Rigid-Body Dynamics. Part 1: Basic Algorithm, Int. J. Robot. Res. **18**(9), 867–875 (1999)
- 2.87 R. Featherstone, A. Fijany: A Technique for Analyzing Constrained Rigid-Body Systems and Its Application to the Constraint Force Algorithm, IEEE Trans. Robot. Autom. **15**(6), 1140–1144 (1999)
- 2.88 P.S. Freeman, D.E. Orin: Efficient Dynamic Simulation of a Quadruped Using a Decoupled Tree-Structured Approach, Int. J. Robot. Res. **10**, 619–627 (1991)
- 2.89 Y. Nakamura, K. Yamane: Dynamics Computation of Structure-Varying Kinematic Chains and Its Application to Human Figures, IEEE Trans. Robot. Autom. **16**(2), 124–134 (2000)